Hochschule für angewandte Wissenschaften Kempten

Automatisierungstechnik und Robotik

Dynamisches Greifen und automatisierte Sichtprüfung mit einem kollaborativen Roboter

Projektarbeit

Verfasser: Niklas Tollmann, Joshua Junt, Felix Krawczyk, Jeremy Zid
Betreuer: Prof. Dr. Tobias Weiser
Abgabe: 30. Oktober 2026

Kurzfassung

In vielen Automatisierungsanwendungen liegen Bauteile nicht an einer festen Position, sondern werden mit wechselnder Lage bereitgestellt. Gleichzeitig soll ihre Qualität möglichst direkt im Prozess geprüft werden. In dieser Projektarbeit wird dazu ein Demonstrator aufgebaut, der beide Aufgaben verbindet: Ein kollaborativer Roboter vom Typ LARA der Firma Neura Robotics greift ein Bauteil, das an beliebiger Position und in beliebiger Orientierung in einem Arbeitsbereich liegt, führt es einer automatischen Sichtprüfung zu und sortiert es anschließend als Gut- oder Schlechtteil aus.

Das System besteht aus mehreren Modulen, die über fest definierte Schnittstellen miteinander verbunden sind. Die Objekterkennung findet den Arbeitsbereich im Bild einer Kamera über vier ArUco-Marker, erzeugt daraus eine entzerrte Draufsicht und bestimmt Position und Orientierung des Bauteils. Eine abgeschrägte Ecke des Bauteils macht die Orientierung dabei eindeutig. Die Robotersteuerung rechnet diese Lage in eine Greifpose um und führt die Greif- und Ablegebewegungen mit reduzierter Geschwindigkeit innerhalb fester Arbeitsraumgrenzen aus. An der Prüfstation liest die Sichtprüfung mit einer seitlichen Kamera eine fünfstellige Seriennummer mittels Texterkennung, misst den Durchmesser eines Durchgangslochs über eine Ellipsenanpassung und prüft, ob eine Kerbe vorhanden ist. Als Alternative zum programmierten Greifen wird ein lernbasierter Ansatz entwickelt: Eine Diffusion Policy lernt aus Demonstrationen, die der Roboter mit gezielt eingespeistem Rauschen selbst fährt, das Bauteil anhand der Bilder einer Kamera am Greifer und der Kamera über dem Arbeitsbereich zu greifen. Sie ist über eine gemeinsame Schnittstelle in den Ablauf eingebunden und übergibt das Bauteil über der Prüfstation an die Robotersteuerung.

Ein zentraler Zustandsautomat steuert den Ablauf aus Erkennen, Greifen, Prüfen und Sortieren, reagiert auf Fehler mit einem Stopp des Roboters und protokolliert jeden Zyklus. Bedient wird das System über eine grafische Oberfläche. Jedes Modul kann mit Platzhaltern, gespeicherten Testbildern, einer Nachbildung der Robotersteuerung, der Simulation des Herstellers oder der realen Hardware betrieben werden, sodass sich die Module unabhängig voneinander entwickeln und schrittweise in Betrieb nehmen lassen. Die eingesetzten Verfahren der Bildverarbeitung sind bewusst einfach und nachvollziehbar gehalten.

Am realen Roboter lief ein erster vollständiger Ablauf durch, noch ohne Greiferbewegung und mit Erkennung und Prüfung auf Testbildern. Die Erprobung an der realen Anlage umfasst wiederholte Durchläufe mit einem Gutteil in zufälliger Lage sowie gezielt fehlerhafte Bauteile ohne Kerbe, mit abweichendem Lochdurchmesser und ohne Seriennummer.

Ergebnisse der Versuche in ein bis zwei Sätzen ergänzen.

Inhaltsverzeichnis

Kurzfassung **II**

Abbildungsverzeichnis **VIII**

Tabellenverzeichnis **X**

1 Einleitung **1**

1.1 Motivation 1
1.2 Aufgabenstellung 2
1.3 Zielsetzung 2
1.4 Abgrenzung 2
1.5 Aufbau der Arbeit 3

2 Vorprojekt: KI-basierte Roboterbahnen **4**

2.1 Aufgabenstellung und Werkzeugumgebung 4
2.2 Kollisionserkennung 5
2.3 Roboter- und Zellsicherheit 6
2.4 Manuelle Bahnplanung 9
2.5 Vorgehen im Projekt 11
2.6 Werkzeuge zur automatischen Bahnplanung 11
2.7 Bedeutung für das vorliegende Projekt 12

3 Grundlagen **14**

3.1 Digitale Bildverarbeitung 14
3.1.1 Digitale Bilder 14
3.1.2 Filterung 14
3.1.3 Schwellwertverfahren 14
3.1.4 Morphologische Operationen 15
3.1.5 Konturen und Bildmomente 16
3.2 Farbräume 16
3.3 Kameramodell und Kalibrierung 18
3.4 Fiducial Marker und Homographie 18
3.4.1 ArUco-Marker 18

3.4.2 Homographie . . . 19
3.5 Koordinatentransformationen . . . 20
3.6 Industrierobotik und Greifen . . . 21
3.6.1 Kinematik und Koordinatensysteme . . . 21
3.6.2 Bewegungsarten . . . 21
3.6.3 Greifer und Greifstrategie . . . 21
3.6.4 Kollaborative Roboter . . . 22
3.7 Optische Messtechnik und Prüfmerkmale . . . 22
3.7.1 Beleuchtung . . . 22
3.7.2 Kantendetektion . . . 22
3.7.3 Detektion von Kreisen und Ellipsen . . . 23
3.7.4 Formmerkmale und Template-Matching . . . 23
3.7.5 Umrechnung in metrische Größen . . . 24
3.8 Texterkennung (OCR) . . . 24
3.9 Lernbasiertes Greifen . . . 25
3.9.1 Greifposen und Policies . . . 25
3.9.2 Imitation Learning und Behavior Cloning . . . 25
3.9.3 Diffusionsmodelle . . . 26
3.9.4 Diffusion Policy . . . 26

4 Systemkonzept 28
4.1 Anforderungen . . . 28
4.2 Hardwareaufbau . . . 29
4.3 Softwarearchitektur . . . 31
4.3.1 Modulstruktur . . . 31
4.3.2 Ablaufsteuerung . . . 32
4.3.3 Schnittstellen . . . 32
4.4 Koordinatensysteme . . . 34
4.5 Test- und Inbetriebnahmestrategie . . . 35

5 Objekterkennung mit der Top-down-Kamera 37
5.1 Bildaufnahme und Kamerakalibrierung . . . 37
5.2 Detektion des Arbeitsbereichs . . . 38
5.3 Segmentierung des Bauteils . . . 39
5.4 Bestimmung der Pose . . . 40
5.5 Transformation in das Roboterkoordinatensystem . . . 40
5.6 Zwischenfazit . . . 42

6 Robotersteuerung 44
6.1 Roboter und Programmierschnittstelle . . . 44

6.2 Greifpose . . . 45
6.3 Greif- und Ablegesequenz . . . 45
6.4 Feste Posen . . . 46
6.5 Kalibrierung zwischen Kamera und Roboter . . . 47
6.6 Sicherheitskonzept . . . 47
6.7 Inbetriebnahme . . . 48
6.8 Zwischenfazit . . . 49

7 Sichtprüfung 50
7.1 Aufbau der Prüfstation . . . 50
7.2 Lokalisierung des Bauteils und Maßstab . . . 51
7.3 Auslesen der Seriennummer . . . 52
7.4 Vermessung des Lochdurchmessers . . . 52
7.5 Prüfung auf Kerbe . . . 53
7.6 Gut-/Schlecht-Entscheidung . . . 53
7.7 Zwischenfazit . . . 55

8 KI-basiertes Greifen 56
8.1 Aufgabe und Abgrenzung . . . 56
8.2 Hardware und Software . . . 57
8.3 Beobachtung und Aktion . . . 58
8.4 Datenaufnahme mit eingespeistem Rauschen . . . 58
8.4.1 Rauschen und asymmetrische Datenpaare . . . 59
8.4.2 Umrechnung und Kollisionsprüfung . . . 59
8.4.3 Objektlagen . . . 61
8.5 Modell und Training . . . 61
8.6 Ausführung . . . 62
8.7 Sicherheitsmaßnahmen . . . 64
8.8 Ausführungsart an der Anlage . . . 65
8.9 Einbindung in das Gesamtsystem . . . 66
8.10 Stand und Ergebnisse . . . 67
8.11 Zwischenfazit . . . 69

9 Integration und Bedienung 70
9.1 Ablauf eines Zyklus . . . 70
9.2 Konfiguration . . . 70
9.3 Bedienoberfläche . . . 72
9.4 Fehlerbehandlung und Sicherheit im Ablauf . . . 75
9.5 Protokollierung . . . 76

10 Evaluation **77**
10.1 Versuchsaufbau und Vorgehen 77
10.2 Ergebnisse Versuchsreihe 1: Wiederholbarkeit 78
10.3 Ergebnisse Versuchsreihe 2: Aussortierung 78
10.4 Vergleich klassischer und KI-basierter Ansatz 78

11 Fazit und Ausblick **80**
11.1 Zusammenfassung 80
11.2 Diskussion 80
11.3 Ausblick 80

Literaturverzeichnis **81**

Abbildungsverzeichnis

2.1 Anlegen eines Kollisionssets: Die Objekte links werden gegen die Objekte rechts geprüft 5
2.2 Hervorgehobene Objekte eines Kollisionssets in einer Roboterzelle . . . 6
2.3 Festlegen des Zellbereichs über Eckpunkte der Grundfläche und Höhengrenzen 7
2.4 Parameter eines Überwachungsraums vom Typ Schutzraum 8
2.5 Darstellung eines Schutzraums (transparenter Quader) um eine Vorrichtung 8
2.6 Festlegen der Werkzeugkugeln über Mittelpunkt und Radius 9
2.7 Greifer mit den umschließenden Werkzeugkugeln 10
2.8 Manuelle Bahnplanung zwischen zwei Vorposen 10

4.1 Aufbau des Demonstrators 29
4.2 Modulstruktur der Software 31
4.3 Zustandsautomat eines Arbeitszyklus. Aus jedem Zustand außer `IDLE` führt ein Fehler in den Zustand `ERROR` (nicht eingezeichnet). 32

5.1 Verarbeitungskette der Objekterkennung 37
5.2 Posenbestimmung mit Fasenerkennung. Oben: Kontur, umschließendes Rechteck und Abstand d_k der Rechteckecke an der Fase. Unten: Ablauf; b_R ist die Breite von R, f ein fester Anteil. 41
5.3 Zwischenergebnisse der Objekterkennung an einem Testfoto: Kamerabild mit den vier erkannten Markern (links) und entzerrte Draufsicht des Arbeitsbereichs mit Kontur (rot), umschließendem Rechteck (grün) und Orientierung (Pfeil zur Fase); oben die Pose im Arbeitsbereich-Koordinatensystem (KS) (rechts). 42

6.1 Greif- und Ablegesequenz: Bewegung in der Seitenansicht (links) und Schritte von `pick` und `place` (rechts). Die Vorposition V liegt um $h = 80\,\text{mm}$ über der Zielpose Z; ab $s = 20\,\text{mm}$ über Z fährt der Roboter langsam. Geprüft wird gegen die Arbeitsraumgrenzen. 46

7.1 Ablauf der Sichtprüfung. Die drei Merkmale werden unabhängig voneinander bestimmt und gemeinsam bewertet. 50

7.2 Seitenansicht eines Testbilds mit dem Ergebnis der Sichtprüfung: Bauteil-Box (grün), Prüfbereiche für Etikett, Loch und Kerbe, angepasste Ellipse am Loch (rot) und erkannte Kerbe (blau). Oben links das Urteil mit Seriennummer, Lochdurchmesser und Kerbe. 51
7.3 Gut-/Schlecht-Entscheidung. Jedes nicht erfüllte Merkmal wird als Grund in G festgehalten. 54

8.1 Handgelenkkamera am Greifer 58
8.2 Ablauf der automatischen Datenaufnahme 60
8.3 Rauschkorridor um die ideale Bahn 61
8.4 Ablauf eines Takts bei der Ausführung der Policy 63
8.5 Spektrum der Beschleunigung am Unterarm je Ausführungsart 65
8.6 Schwingung am Unterarm je Ausführungsart 66
8.7 Zehn Fahrten der Policy in der Simulation 68
8.8 Beobachtung der Policy: Handgelenkkamera (links) und Szenenkamera (rechts) 68

9.1 Aufrufe des Orchestrators an die Module während eines Arbeitszyklus. Greifen und Ablegen an der Prüfposition laufen über die Schnittstelle `Grasper`: entweder direkt über M2 oder mit der Policy von M4, die anschließend über M2 ablegt. 71
9.2 Bedienoberfläche, Reiter „Ablauf“ nach drei Zyklen mit Testfotos und simuliertem Roboter. Oben die Auswahl der Varianten, darunter die Ablaufkette, die Bilder von Erkennung und Prüfung, die Draufsicht auf den Roboterarbeitsraum und die Ergebnistabelle. 74
9.3 Bedienoberfläche, Reiter „M2 Roboter“ mit dem Simulator verbunden: links Verbindung, Status, Override und Tippbetrieb, rechts die Draufsicht und die Seiten für die Inbetriebnahme 74
9.4 Bedienoberfläche, Reiter „M4 Behavior Cloning“, Unterreiter Systemcheck 75

Tabellenverzeichnis

4.1 Funktionale Anforderungen 28
4.2 Nicht-funktionale Anforderungen 29
4.3 Komponenten des Demonstrators 30
4.4 Zustände des Arbeitszyklus und beteiligte Module 33
4.5 Zwischen den Modulen ausgetauschte Datenobjekte 33
4.6 Schnittstellen der Module 34
4.7 Betriebsstufen der Module 36

5.1 Sättigung und Buntheit auf den Testfotos (Median je Fläche, 8-Bit-Werte, fünf Fotos) 39

6.1 Feste Posen des Roboters 47

8.1 Hardware und Software von Modul 4 57
8.2 Beobachtung und Aktion der Policy 58
8.3 Einstellungen der Diffusion Policy 62
8.4 Erprobung der Kette mit Platzhalterbildern 67
8.5 Ersatzstudie zum eingespeisten Rauschen 68

9.1 Wählbare Varianten der Module 72
9.2 Reiter der Bedienoberfläche 73

10.1 Fehlerhafte Bauteile für Versuchsreihe 2 78

Abkürzungsverzeichnis

BC Behavior Cloning

CNN Convolutional Neural Network

Cobot kollaborativer Roboter

CSV Comma-Separated Values

GUI Graphical User Interface

HSV Hue, Saturation, Value

KS Koordinatensystem

LAN Local Area Network

LIN Linearbewegung

OCR Optical Character Recognition

PTP Point-to-Point

RMS Root Mean Square (quadratischer Mittelwert)

ROI Region of Interest

TCP Tool Center Point

USB Universal Serial Bus

VM virtuelle Maschine

1 Einleitung

1.1 Motivation

In vielen Automatisierungsanwendungen liegen Bauteile nicht an einer exakt festgelegten Position, sondern werden von Hand, über ein Förderband oder aus einem vorgelagerten Prozess mit wechselnder Lage bereitgestellt. Ein Roboter, der nur fest programmierte Positionen anfährt, kann solche Teile nicht greifen. Mit einer Kamera, die Position und Orientierung des Bauteils bestimmt, lässt sich die Greifbewegung dagegen für jedes Teil neu berechnen. Dieses dynamische Handhaben macht mechanische Vorrichtungen zur exakten Ausrichtung der Teile überflüssig.

Eine zweite Aufgabe, die sich mit Kameras automatisieren lässt, ist die Qualitätsprüfung. Wird jedes Teil direkt im Prozess geprüft, statt nur stichprobenartig am Ende, lassen sich fehlerhafte Teile sofort aussortieren. Werden Handhabung und Prüfung in einer Zelle kombiniert, übernimmt der Roboter neben dem Transport auch das Zuführen zur Prüfung und das Sortieren nach dem Ergebnis.

Das vorliegende Projekt verbindet beide Aufgaben in einem Demonstrator. Die Aufgabe hat sich die Projektgruppe selbst gestellt, nachdem ein vorheriges Industrieprojekt zu KI-basierten Roboterbahnen vorzeitig beendet wurde (Kapitel 2). Sie ist bewusst industrienah gewählt: Eingesetzt werden ein realer kollaborativer Industrieroboter mit seiner Herstellerschnittstelle, Kameras und Verfahren der klassischen Bildverarbeitung, die bewusst einfach und nachvollziehbar gehalten sind. Im Vordergrund steht, ein durchgängig funktionierendes Gesamtsystem aufzubauen, und nicht, einzelne Verfahren zu optimieren.

Optional: Aussagen zur Bedeutung von flexibler Handhabung und Inline-Prüfung in der Automatisierung mit Quellen belegen (Quellen mit DOI).

Offen: Weiterverwendung im Pflegerobotik-Projekt erwähnen? (Einleitung, Fazit oder Ausblick, siehe Blueprint)

1.2 Aufgabenstellung

Ziel des Projekts ist ein Demonstrator, der ein Bauteil mit beliebiger Position und Orientierung in einem durch vier ArUco-Marker begrenzten Arbeitsbereich erkennt, mit einem Roboter greift und an einer Prüfstation ablegt. Dort wird eine Seriennummer ausgelesen, ein Lochdurchmesser vermessen und das Vorhandensein einer Kerbe geprüft. Abhängig vom Prüfergebnis sortiert der Roboter das Bauteil als Gut- oder Schlechtteil aus.

Das System ist in vier Module gegliedert: die Objekterkennung mit einer Kamera über dem Arbeitsbereich, die Robotersteuerung, die Sichtprüfung mit einer seitlichen Kamera und ein lernbasierter Ansatz zum Greifen. Bei diesem greift eine aus Demonstrationen gelernte Policy das Bauteil anhand der Bilder einer Kamera am Greifer und der Kamera über dem Arbeitsbereich; sie ist eine Alternative zum programmierten Greifen der Robotersteuerung. Ein zentraler Ablauf verbindet die Module zu einem Gesamtsystem.

1.3 Zielsetzung

Das Projekt gilt als erfolgreich, wenn der Ablauf aus Erkennen, Greifen, Prüfen und Sortieren vollständig automatisch durchläuft, ohne dass ein Bediener eingreifen muss. Die Prüfmerkmale dienen dabei der Demonstration: Toleranzen und Grenzwerte sind so gewählt, dass die Funktion der Prüfung gezeigt werden kann, und entsprechen keinen Fertigungsvorgaben.

1.4 Abgrenzung

Um den Umfang des Projekts zu begrenzen, gelten folgende Annahmen und Einschränkungen:

- Im Arbeitsbereich liegt jeweils nur ein Bauteil. Das Vereinzeln von Teilen aus einer ungeordneten Menge wird nicht betrachtet.
- Das Bauteil liegt flach auf dem Tisch. Seine Lage ist damit durch die Position in der Tischebene und einen Drehwinkel vollständig beschrieben, die Greifhöhe ist bekannt.
- Die Anlage ist nicht sicherheitszertifiziert. Sie wird nur unter Aufsicht und mit reduzierter Geschwindigkeit betrieben.

- Der lernbasierte Greifansatz ist eine Alternative zum programmierten Greifen und keine Voraussetzung für den Demonstrator. Er übernimmt nur das Greifen bis über die Prüfposition; alle übrigen Bewegungen führt die Robotersteuerung aus.

1.5 Aufbau der Arbeit

Kapitel 2 fasst die Erfahrungen aus dem Vorprojekt zur Bahnplanung in der Robotersimulation zusammen und ordnet ein, was davon für dieses Projekt relevant ist. Kapitel 3 stellt die Grundlagen der verwendeten Verfahren aus Bildverarbeitung, Messtechnik und Robotik vor. In Kapitel 4 wird das Gesamtkonzept mit Anforderungen, Hardwareaufbau und Softwarearchitektur beschrieben. Die Kapitel 5 bis 8 behandeln die Umsetzung der einzelnen Module. Kapitel 9 zeigt, wie die Module zu einem Gesamtsystem zusammengesetzt und bedient werden. Kapitel 10 beschreibt die Erprobung des Systems, und Kapitel 11 fasst die Ergebnisse zusammen.

2 Vorprojekt: KI-basierte Roboterbahnen

Vor dem vorliegenden Projekt hat die Projektgruppe an einer Aufgabenstellung in der Industrie gearbeitet. Bei der GROB-WERKE GmbH & Co. KG sollte das Thema „Erstellung von KI-basierten Roboterpfaden für die Robotersimulation und Inbetriebnahme“ bearbeitet werden. Nach einer Einarbeitung von acht Tagen vor Ort wurde das Projekt bearbeitet. Während der Bearbeitung wurde die Zusammenarbeit beendet, und die Gruppe hat sich die Aufgabe dieser Arbeit selbst gestellt (Kapitel 1). Die Ergebnisse des Vorprojekts sind deshalb begrenzt.

Dieses Kapitel fasst zusammen, was bis dahin erarbeitet wurde. Es beschreibt die Aufgabe, die Werkzeuge und das Vorgehen bei der Bahnplanung in der Robotersimulation und die ersten Erfahrungen mit Werkzeugen zur automatischen Bahnplanung. Am Ende wird eingeordnet, welche Erkenntnisse für den Demonstrator relevant sind oder eine mögliche Erweiterung wären.

2.1 Aufgabenstellung und Werkzeugumgebung

Bevor eine Roboterzelle aufgebaut wird, werden ihre Bewegungsabläufe in einer Robotersimulation geplant und geprüft. Dafür wird ein digitales Modell der Zelle aus den Konstruktionsdaten verwendet, mit Robotern, Greifern, Vorrichtungen und Schutzeinrichtungen. Die dort erstellten Bahnen werden später auf die realen Roboter übertragen. Je genauer die Bahnen in der Simulation stimmen, desto weniger muss bei der Inbetriebnahme an der Anlage nachgebessert werden.

Bei GROB wird dafür Siemens Process Simulate eingesetzt. Die Bahnen werden dort bisher von Hand erstellt. Das kostet bei großen Zellen mit mehreren Robotern viel Zeit. Ziel des Vorprojekts war es zu untersuchen, ob sich Bahnen mit KI-basierten Werkzeugen automatisch erzeugen lassen und ob diese Bahnen den Anforderungen von GROB genügen. Ergebnis sollte in jedem Fall eine kollisionsfreie und auf die Taktzeit optimierte Roboterbahn sein.

Die Aufgabe wurde auf einen Teil einer bestehenden Zelle eingegrenzt. Betrachtet wurde ein Handhabungsprozess, bei dem ein Roboter Teile zwischen festen Positionen umsetzt. Prozesse wie das Kleben wurden nicht betrachtet. Sie sind deutlich komplexer und für die Fragestellung nicht notwendig. Verwendet wurde ein KUKA-Roboter, weil sich dieser gut in Process Simulate abbilden lässt und einfacher zu handhaben ist.

In der Einarbeitung wurden vier Themen behandelt: Kollisionserkennung, Roboter- und Zellsicherheit, das manuelle Erstellen von Bahnen und erste Werkzeuge zur automatischen Bahnplanung. Sie werden in den folgenden Abschnitten beschrieben, bevor Abschnitt 2.5 das Vorgehen im Projekt zeigt.

Quelle für Siemens Process Simulate (Herstellerdokumentation) ergänzen, falls gewünscht.

2.2 Kollisionserkennung

Die Simulation prüft nicht automatisch jedes Bauteil der Zelle gegen jedes andere. Stattdessen legt der Anwender *Kollisionssets* an. Ein Kollisionsset besteht aus zwei Listen von Objekten: Die Objekte der ersten Liste werden gegen die Objekte der zweiten Liste geprüft (Abb. 2.1). Typischerweise stehen in der ersten Liste der Roboter und sein Greifer. In der zweiten Liste stehen die Vorrichtungen, Werkstücke und Anlagenteile, mit denen er zusammenstoßen könnte. Paare, die sich ohnehin berühren, etwa der Greifer und der Roboterflansch, werden so von der Prüfung ausgenommen. Das spart Rechenzeit und vermeidet Fehlmeldungen.

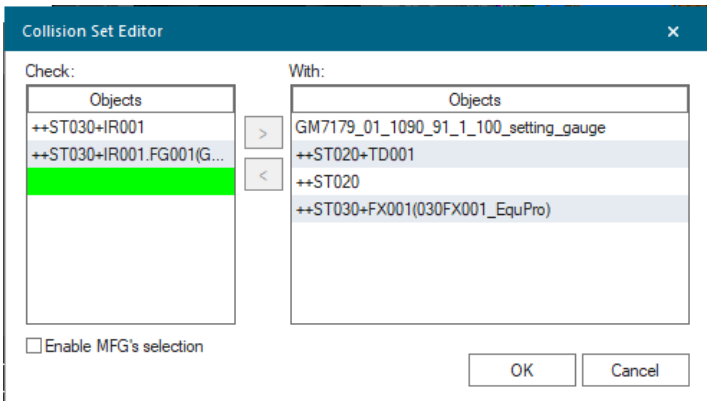


Abbildung 2.1: Anlegen eines Kollisionssets: Die Objekte links werden gegen die Objekte rechts geprüft

Für jedes Kollisionsset lassen sich zwei Grenzwerte einstellen. Der *Beinahe-Abstand* („Near Miss“) meldet, wenn sich zwei Objekte näher als ein festgelegter Abstand kommen, ohne sich zu berühren. Die *zulässige Durchdringung* erlaubt geringe Überschneidungen, zum Beispiel wenn ein Greifer ein Werkstück greift. Während der Simulation hebt die

Software die Objekte eines Sets farbig hervor und listet die kollidierenden Paare auf (Abb. 2.2).

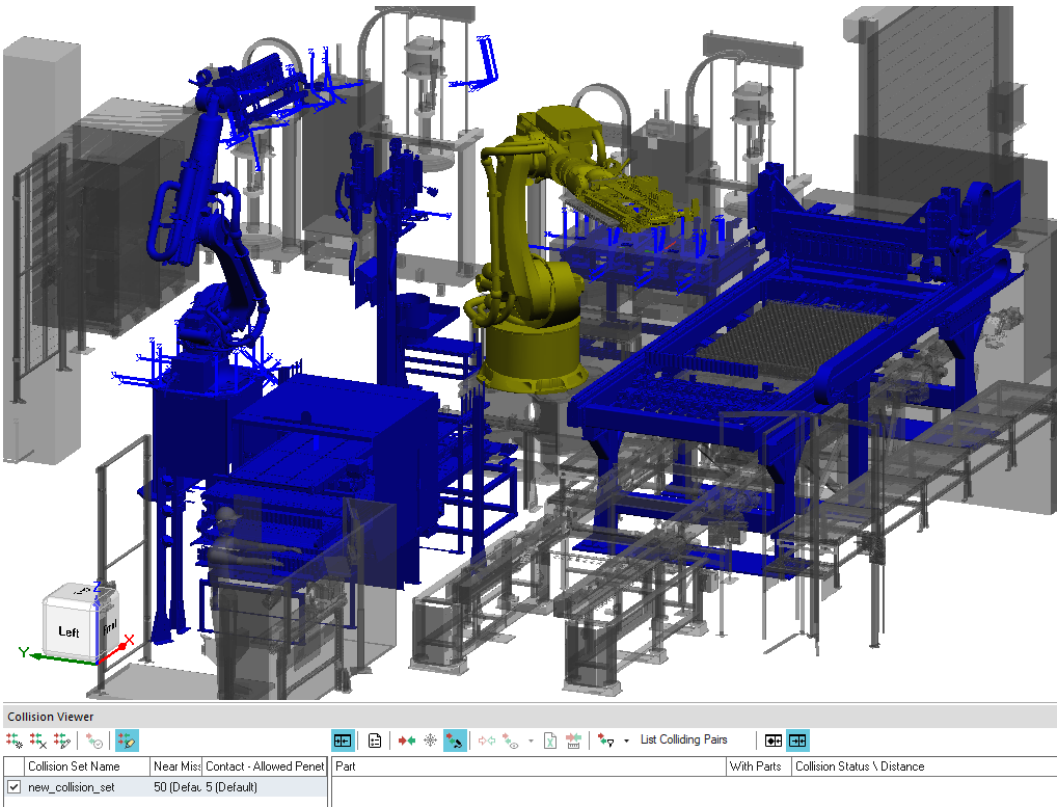


Abbildung 2.2: Hervorgehobene Objekte eines Kollisionssets in einer Roboterzelle

Für die automatische Bahnplanung sind die Kollisionssets die Grundlage. Ein Planungswerkzeug kann nur solche Kollisionen vermeiden, die es auch prüft. Fehlt ein Objekt im Set, kann eine Bahn in der Simulation kollisionsfrei erscheinen und an der realen Anlage trotzdem zu einem Zusammenstoß führen. Jedes Werkzeug zur automatischen Bahnplanung muss die Kollisionssets deshalb berücksichtigen.

2.3 Roboter- und Zellsicherheit

Neben der Kollisionsprüfung in der Simulation überwacht die Sicherheitssteuerung des realen Roboters, dass er bestimmte Bereiche nicht verlässt oder nicht betritt. Diese Bereiche werden ebenfalls in der Simulation festgelegt und dort sichtbar gemacht. In der Einarbeitung wurde dafür eine Erweiterung von Process Simulate für die Sicherheitskonfiguration der eingesetzten KUKA-Roboter verwendet. Drei Elemente wurden behandelt.

Zellbereich. Der Zellbereich beschreibt den Raum, den der Roboter insgesamt nicht verlassen darf. Er wird als Prisma angegeben: eine Grundfläche aus bis zu zehn Eckpunkten in der Ebene und eine untere und obere Höhengrenze (Abb. 2.3).

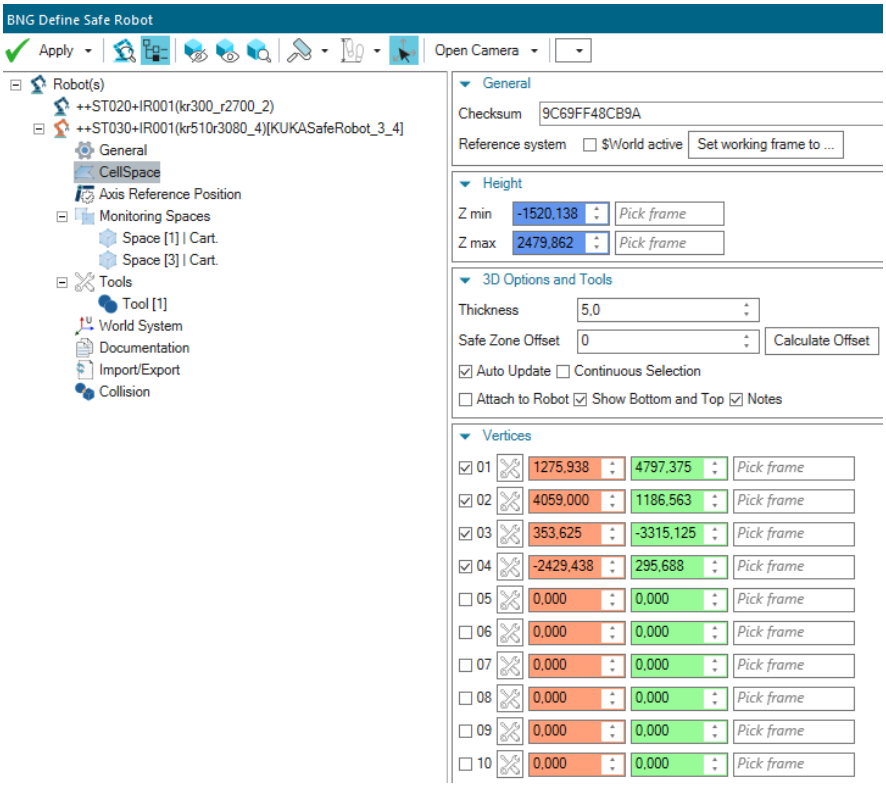


Abbildung 2.3: Festlegen des Zellbereichs über Eckpunkte der Grundfläche und Höhengrenzen

Überwachungsräume. Innerhalb des Zellbereichs lassen sich weitere Räume als Quader festlegen (Abb. 2.4). Ein Quader wird durch ein Bezugskoordinatensystem und die minimalen und maximalen Abstände in jeder Achse beschrieben. Für jeden Raum wird eingestellt, ob der Roboter ihn nicht verlassen darf (Arbeitsraum) oder nicht betreten darf (Schutzraum), ob er immer oder nur unter Bedingungen aktiv ist und ob der Roboter an der Grenze anhält. Ein Schutzraum wird zum Beispiel um eine Vorrichtung gelegt, in die der Roboter nicht eingreifen soll (Abb. 2.5).

Werkzeugkugeln. Damit die Sicherheitssteuerung nicht nur den Tool Center Point (TCP), sondern das ganze Werkzeug überwachen kann, wird der Greifer durch Kugeln angenähert (Abb. 2.6). Jede Kugel ist durch ihren Mittelpunkt relativ zum Roboterflansch und ihren Radius festgelegt. Die Kugeln müssen den Greifer vollständig umschließen. Zu große Kugeln schränken den nutzbaren Arbeitsraum dagegen unnötig ein. Die Steuerung prüft dann für jede Kugel, ob sie einen Überwachungsraum verletzt.

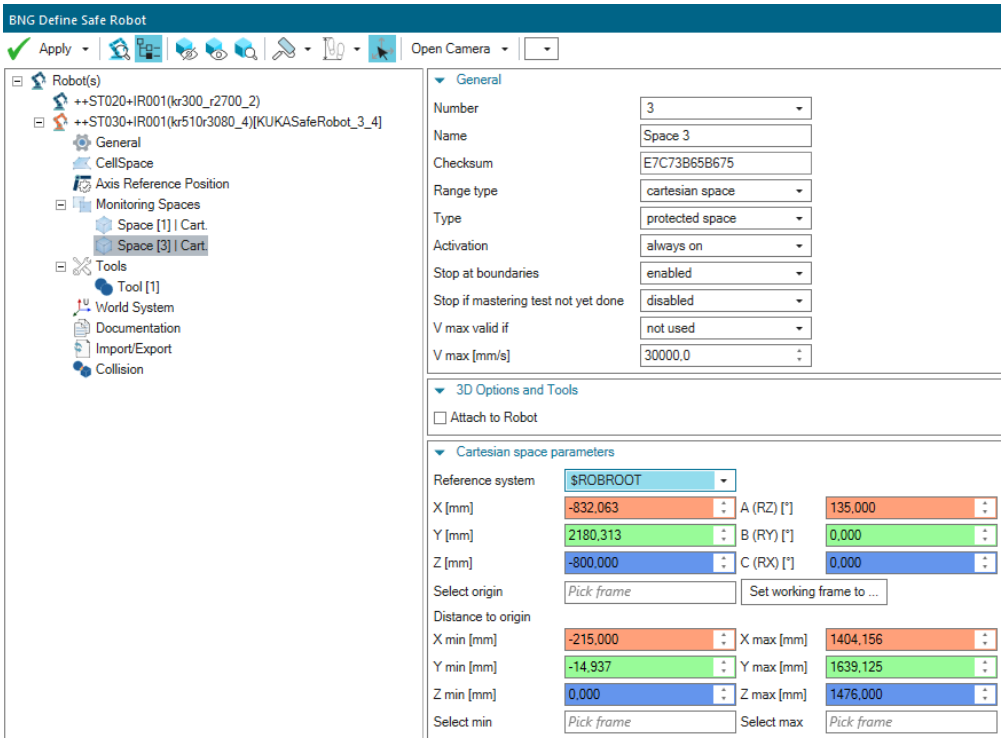


Abbildung 2.4: Parameter eines Überwachungsraums vom Typ Schutzraum

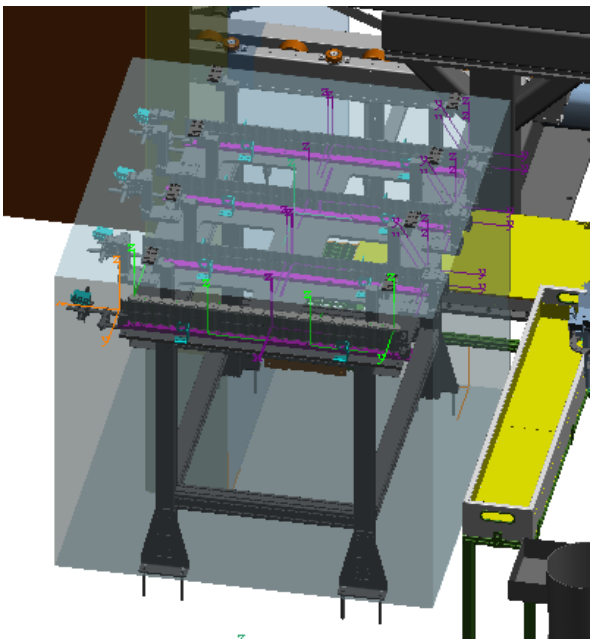


Abbildung 2.5: Darstellung eines Schutzraums (transparenter Quader) um eine Vorrichtung

Kugeln sind dafür gut geeignet, weil sich der Abstand einer Kugel zu einer Fläche mit wenig Rechenaufwand bestimmen lässt.

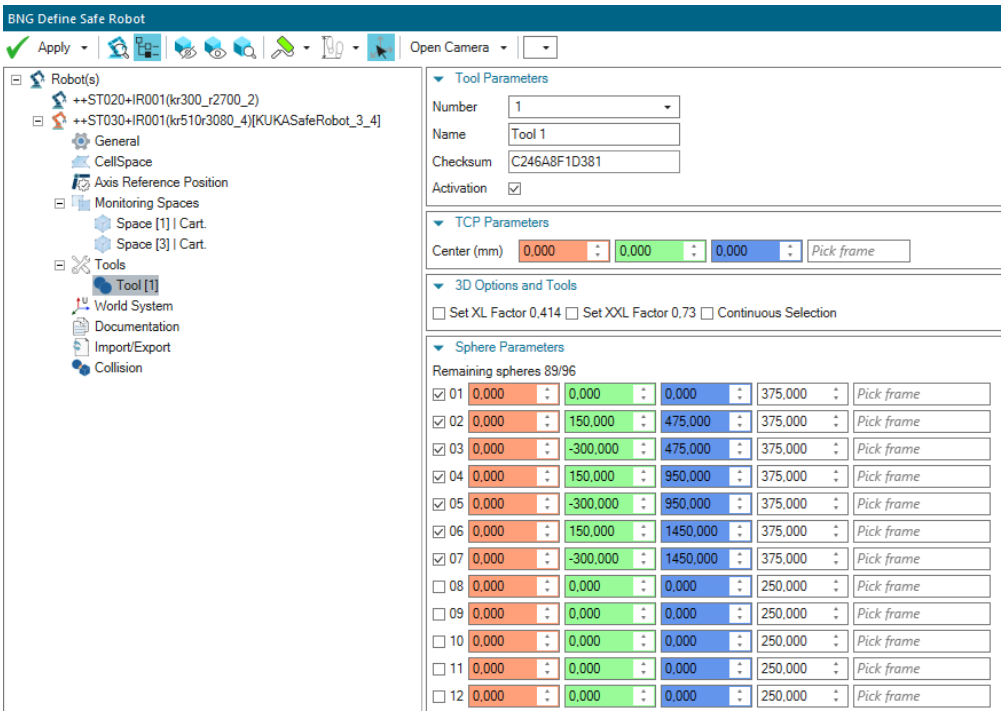


Abbildung 2.6: Festlegen der Werkzeugkugeln über Mittelpunkt und Radius

2.4 Manuelle Bahnplanung

Bei GROB werden die Bahnen nach einem festen Vorgehen erstellt. Gegeben sind die Startpose und die Zielpose, zum Beispiel eine Entnahme- und eine Ablageposition. Zu beiden Posen wird zuerst eine Vorpose erzeugt. Von der Vorpose aus fährt der Roboter mit einer Linearbewegung (LIN) in die Start- bzw. Zielpose, wie es auch in Abschnitt 3.6 beschrieben ist.

Die Verbindung zwischen den beiden Vorposen wird schrittweise aufgebaut (Abb. 2.8). Der Roboter wird in der Simulation in Gelenkkoordinaten von der einen Vorpose zur anderen verfahren, bis eine Kollision auftritt. An dieser Stelle wird er aus der Kollision herausbewegt, möglichst durch das Verfahren nur einer Achse. Die so gefundene Stellung wird als Zwischenpunkt gespeichert, und die Bahn wird von dort aus erneut betrachtet. Das wird wiederholt, bis die gesamte Bahn kollisionsfrei ist.

Ziel dieses Vorgehens ist, die Bewegungen der einzelnen Achsen klein zu halten. Oft wurde außerdem gewünscht, dass die Achsen 4 und 6 bei 0° bleiben. Das ist nicht zwingend, vermeidet aber unnötige Drehungen der Handachsen und damit eine starke

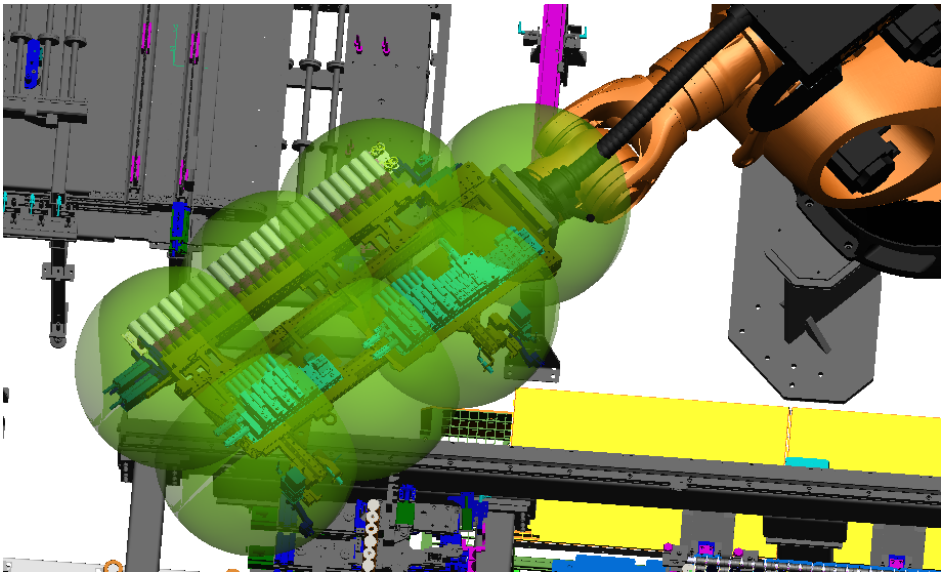


Abbildung 2.7: Greifer mit den umschließenden Werkzeugkugeln

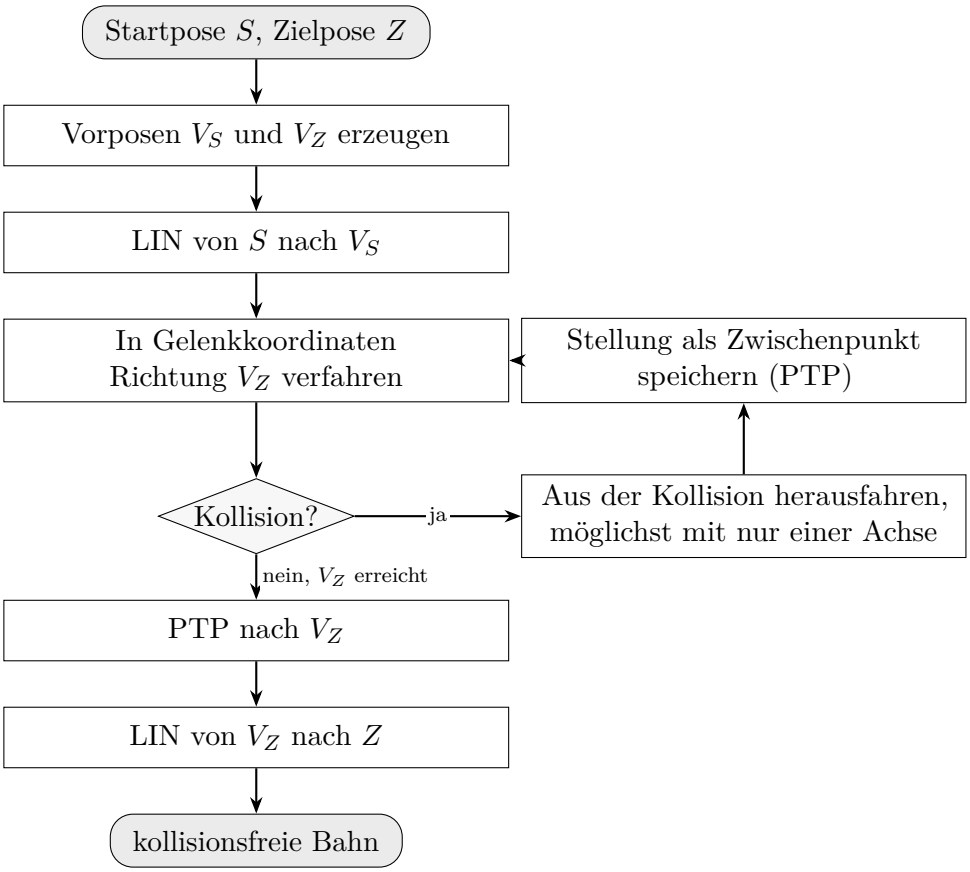


Abbildung 2.8: Manuelle Bahnplanung zwischen zwei Vorposen

Belastung der Energiezuführung zum Greifer. Außerdem bleiben die Bahnen für den Inbetriebnehmer nachvollziehbar.

2.5 Vorgehen im Projekt

Die betrachtete Zelle ist bereits seit einigen Jahren umgesetzt. Deshalb wurde zuerst das bestehende Projekt analysiert. Die Taktzeiten der dort umgesetzten Bahnen dienen als Vergleichsbasis für alle neu erzeugten Bahnen.

Anschließend wurde ein leeres Bahnmodul angelegt. Es enthält nur die Start- und Zielposen und die zugehörigen Vorposen, aber keine Zwischenpunkte. Alle Werkzeuge zur automatischen Bahnplanung erhalten dasselbe Bahnmodul als Ausgangspunkt. Dadurch sind ihre Ergebnisse untereinander und mit der bestehenden Bahn vergleichbar.

Zusätzlich hat die Projektgruppe selbst Bahnen nach dem manuellen Vorgehen aus Abschnitt 2.4 erstellt. Diese Bahnen zeigen, welches Ergebnis mit wenig Erfahrung in der Bahnplanung erreicht wird, und dienen als weiterer Vergleich. Gleichzeitig hat die Gruppe dabei das Programm und die Bahnplanung selbst kennengelernt.

Damit ergeben sich drei Vergleichswerte für eine automatisch erzeugte Bahn: die bestehende, von erfahrenen Planern erstellte Bahn, die eigene manuelle Bahn und die Bahnen der anderen Werkzeuge.

Falls vorhanden: Taktzeiten der bestehenden Bahn, der eigenen manuellen Bahn und der Bahn aus dem Robotic Automatic Path Planner als Tabelle ergänzen.

2.6 Werkzeuge zur automatischen Bahnplanung

Für die automatische Erzeugung der Bahnen waren zwei Werkzeuge vorgesehen. Als dritte Möglichkeit sollten Verfahren betrachtet werden, die nicht in Process Simulate enthalten sind.

Robotic Automatic Path Planner. Der Robotic Automatic Path Planner ist in Process Simulate integriert. Er erzeugt aus einer vorhandenen Bahn mit Start- und Zielpose eine kollisionsfreie Bahn und berücksichtigt dabei die Kollisionssets. Der Anwender kann zwischen mehreren Optimierungszielen wählen, zum Beispiel einer möglichst kurzen Taktzeit.

Das Werkzeug wurde auf das leere Bahnmodul angewendet, und Bahnen konnten erfolgreich automatisch erstellt werden. Bei der Optimierung auf die Taktzeit fiel jedoch auf, dass das Werkzeug auch die programmierten Geschwindigkeiten ändert. Betroffen war zum Beispiel die geradlinige Bewegung von der Vorpose in die Zielpose. Diese Bewegung wird bewusst langsam programmiert, damit das Werkstück sicher abgelegt oder gegriffen wird. Wird sie beschleunigt, kann das Werkstück verrutschen, oder Vorrichtung und Greifer werden stärker belastet. Eine automatisch erzeugte Bahn muss deshalb darauf geprüft werden, ob solche festen Vorgaben erhalten bleiben.

Weitere Optimierungsziele des Robotic Automatic Path Planner nennen (z. B. Abstand zu Hindernissen, Achsbewegung), falls bekannt.

Realtime Robotics. Als zweites Werkzeug war eine Lösung der Firma Realtime Robotics vorgesehen. Die Lizenz stand bis zum Ende der Zusammenarbeit nicht zur Verfügung, deshalb konnte das Werkzeug nicht getestet werden.

Alternative Verfahren.

Kommilitone: Alternative Verfahren zur Bahnplanung außerhalb von Process Simulate beschreiben (Ansätze, betrachtete Werkzeuge, Einschätzung). Neue Quellen nur mit geprüfter DOI.

2.7 Bedeutung für das vorliegende Projekt

Die Aufgabe des vorliegenden Projekts unterscheidet sich vom Vorprojekt. Im Mittelpunkt steht nicht die Planung von Bahnen in der Simulation, sondern das Erkennen, Greifen und Prüfen eines Bauteils am realen Roboter. Mehrere Erkenntnisse aus dem Vorprojekt sind trotzdem übertragbar:

Vorpose und geradlinige Anfahrt Die Greif- und Ablegesequenz des Demonstrators folgt demselben Muster wie bei GROB: Eine Vorpose wird mit einer Gelenkbewegung angefahren, das Ziel dann geradlinig, auf den letzten Millimetern langsam (Abschnitt 6.3). Die Erfahrung mit dem Robotic Automatic Path Planner zeigt, dass diese langsame Endanfahrt als feste Vorgabe gelten muss, auch wenn Bahnen später automatisch erzeugt oder optimiert werden.

Sichere Bereiche Die Robotersteuerung des Demonstrators prüft vor jeder Bewegung, ob Zielpose und Vorposition in einem festgelegten Quader liegen (Abschnitt 6.6). Das entspricht in vereinfachter Form einem Arbeitsraum. Geprüft wird dabei nur der TCP. Eine mögliche Erweiterung wäre, Greifer und Bauteil wie bei

den Werkzeugkugeln durch Kugeln anzunähern und zusätzlich Schutzräume um Kameras und Prüfstation festzulegen.

Kollisionsprüfung in der Simulation Im klassischen Ablauf des Demonstrators werden die Posen geteacht, und die Wege dazwischen führen durch freien Raum über dem Tisch. Eine Kollisionsprüfung ist dort nicht nötig. Würde die Zelle erweitert, könnten die Bahnen vorab in einer Simulation mit einem Zellmodell und Kollisionssets geprüft werden. Für den eingesetzten Roboter bietet der Hersteller dafür eine Offline-Simulation an [1].

Randbedingungen für KI-Verfahren Im Vorprojekt sollte eine KI die Bahnen erzeugen, im Demonstrator steuert optional eine gelernte Policy den Roboter beim Greifen (Kapitel 8). In beiden Fällen gilt: Das Ergebnis der KI muss die Randbedingungen der Anlage einhalten, also kollisionsfrei sein, in den erlaubten Bereichen liegen und feste Vorgaben wie Geschwindigkeiten beibehalten. Da die Policy keine Zielpose liefert, sondern laufend Gelenk-Sollwerte über eine eigene Verbindung sendet, prüft sie diese Randbedingungen selbst: Jeder Sollwert wird auf Geschwindigkeit und Achsgrenzen geprüft, und ein Kollisionsmodell aus Quadern mit Prüfpunkten entlang des Arms überwacht den Abstand zu Tisch und Hindernissen (Abschnitt 8.7). Das entspricht in einfacher Form den Kollisionssets und Werkzeugkugeln aus dem Vorprojekt. Die Bewegungen der Robotersteuerung prüft diese weiterhin gegen ihre Arbeitsraumgrenzen.

3 Grundlagen

Dieses Kapitel stellt die Verfahren vor, auf denen die Umsetzung in den Kapitel 5 bis 7 aufbaut. Es beschreibt dabei auch Alternativen, gegen die sich die Umsetzung entschieden hat; die jeweilige Begründung folgt in den Modulkapiteln.

3.1 Digitale Bildverarbeitung

3.1.1 Digitale Bilder

Ein digitales Grauwertbild ist eine Matrix $I(u, v)$, deren Elemente (Pixel) die Helligkeit an der Stelle mit Spalte u und Zeile v angeben, üblicherweise als ganze Zahl von 0 (schwarz) bis 255 (weiß). Ein Farbbild besteht aus drei solchen Matrizen, den Farbkanälen. Der Ursprung liegt in der linken oberen Ecke, die v-Achse zeigt nach unten. Für die Verarbeitung wird im Projekt die Bibliothek OpenCV [2] eingesetzt, die die im Folgenden beschriebenen Verfahren bereitstellt.

3.1.2 Filterung

Bildrauschen erschwert viele nachfolgende Schritte. Es wird meist durch eine Glättung verringert, bei der jeder Pixel durch einen gewichteten Mittelwert seiner Nachbarschaft ersetzt wird. Mathematisch ist das eine Faltung des Bildes mit einem Filterkern. Beim Gaußfilter folgen die Gewichte einer zweidimensionalen Normalverteilung, sodass nahe Pixel stärker eingehen als entfernte. Die Glättung verringert das Rauschen, macht aber auch Kanten unschärfer; die Größe des Filterkerns ist deshalb ein Kompromiss.

3.1.3 Schwellwertverfahren

Eine Segmentierung trennt Bildbereiche, die zu einem Objekt gehören, vom Hintergrund. Das einfachste Verfahren ist ein Schwellwert T: Pixel mit $I(u, v) > T$ werden dem

Objekt zugeordnet, alle anderen dem Hintergrund. Das Ergebnis ist ein Binärbild (Maske).

Ein fester Schwellwert ist empfindlich gegenüber Änderungen der Beleuchtung. Das Verfahren von Otsu [3] bestimmt den Schwellwert deshalb automatisch aus dem Histogramm des Bildes. Es wählt denjenigen Wert T, der die Varianz zwischen den beiden entstehenden Klassen maximiert:

$$\sigma_B^2(T) = \omega_0(T)\,\omega_1(T)\,\Big(\mu_0(T) - \mu_1(T)\Big)^2. \tag{3.1}$$

Dabei sind ω_0 und ω_1 die Anteile der Pixel unterhalb und oberhalb von T und μ_0, μ_1 deren mittlere Grauwerte. Das Verfahren funktioniert gut, wenn das Histogramm zwei deutlich getrennte Häufungen aufweist, also Objekt und Hintergrund ähnlich große Flächen einnehmen und sich klar unterscheiden. Nimmt das Objekt nur einen kleinen Teil des Bildes ein, ist seine Häufung im Histogramm schwach ausgeprägt, und der Schwellwert orientiert sich überwiegend an den Schwankungen des Hintergrunds.

Neben globalen Verfahren gibt es adaptive Schwellwerte, die für jeden Pixel aus seiner lokalen Umgebung berechnet werden, sowie Schwellwerte, die relativ zu einer bekannten Referenz wie der Helligkeit des Hintergrunds festgelegt werden.

3.1.4 Morphologische Operationen

Binärmasken nach einer Schwellwertbildung enthalten oft einzelne Störpixel oder kleine Lücken. Morphologische Operationen bearbeiten die Form der Bereiche mithilfe eines Strukturelements, etwa eines kleinen Quadrats:

- Die **Erosion** setzt einen Pixel nur dann auf Objekt, wenn das gesamte Strukturelement an dieser Stelle im Objekt liegt; Bereiche schrumpfen.
- Die **Dilatation** setzt einen Pixel auf Objekt, sobald das Strukturelement irgendwo das Objekt berührt; Bereiche wachsen.
- Das **Öffnen** (Erosion, dann Dilatation) entfernt Strukturen, die kleiner als das Strukturelement sind, und erhält die übrigen weitgehend in ihrer Form.
- Das **Schließen** (Dilatation, dann Erosion) füllt Lücken und Einbuchtungen, die kleiner als das Strukturelement sind.

3.1.5 Konturen und Bildmomente

Die zusammenhängenden Bereiche einer Binärmaske lassen sich durch ihre Randlinien, die Konturen, beschreiben. OpenCV bestimmt sie mit dem Randverfolgungsverfahren nach Suzuki und Abe [4], das auch die Verschachtelung von äußeren Rändern und Löchern liefert. Aus einer Kontur lassen sich Fläche, Umfang und umschließende Formen wie das kleinste umschließende Rechteck oder der kleinste umschließende Kreis berechnen.

Lage und Ausrichtung eines Bereichs werden über Bildmomente beschrieben [5]. Für einen Bereich mit den Pixeln (u, v) sind die Momente

$$m_{pq} = \sum_{u} \sum_{v} u^p \, v^q \, I(u, v), \tag{3.2}$$

wobei $I(u, v)$ in einer Binärmaske 1 für Objektpixel und 0 sonst ist. Das Moment m_{00} ist die Fläche, und der Flächenschwerpunkt ergibt sich zu

$$\bar{u} = \frac{m_{10}}{m_{00}}, \qquad \bar{v} = \frac{m_{01}}{m_{00}}. \tag{3.3}$$

Die zentralen Momente zweiter Ordnung μ_{20}, μ_{02} und μ_{11} werden auf den Schwerpunkt bezogen berechnet und beschreiben die Streuung des Bereichs. Aus ihnen folgt der Winkel der Hauptachse

$$\theta = \tfrac{1}{2} \operatorname{atan2}\Big(2\mu_{11},\ \mu_{20} - \mu_{02}\Big). \tag{3.4}$$

Die Hauptachse und die lange Seite des kleinsten umschließenden Rechtecks geben bei länglichen Objekten die Orientierung an, allerdings nur bis auf 180°: Eine Achse hat keine Richtung. Für eine eindeutige Orientierung braucht es ein zusätzliches, asymmetrisches Merkmal des Objekts.

3.2 Farbräume

Kameras liefern Farbbilder meist im RGB-Farbraum mit je einem Kanal für Rot, Grün und Blau. In diesem Raum sind Farbe und Helligkeit miteinander verknüpft: Wird ein Objekt heller beleuchtet, steigen alle drei Kanäle, obwohl sich seine Farbe nicht ändert. Für eine Segmentierung nach Farbe ist das ungünstig.

Der Hue, Saturation, Value (HSV)-Farbraum [6] trennt diese Eigenschaften. Er beschreibt eine Farbe durch drei Größen:

Farbton (H) die Farbart als Winkel auf einem Farbkreis, z. B. Rot, Gelb, Grün oder Violett;

Sättigung (S) wie rein die Farbe ist, von grau (S = 0) bis zur reinen Farbe;

Hellwert (V) die Helligkeit.

Mit $M = \max(R, G, B)$ und $m = \min(R, G, B)$ gilt

$$V = M, \qquad S = \begin{cases} (M - m)/M, & M > 0 \\ 0, & M = 0 \end{cases} \tag{3.5}$$

und der Farbton ergibt sich daraus, welcher Kanal am größten ist und wie sich die beiden anderen zueinander verhalten. In OpenCV wird der Farbton bei 8-Bit-Bildern auf den Bereich 0 bis 180 abgebildet, Sättigung und Hellwert auf 0 bis 255.

Für die Bildverarbeitung ergeben sich daraus zwei Vorteile. Ein farbiges Objekt lässt sich über ein Intervall des Farbtons finden, das weitgehend unabhängig von der Beleuchtung ist. Und unbunte Flächen wie eine graue Platte oder ein weißes Etikett unterscheiden sich von farbigen Objekten durch ihre geringe Sättigung, auch wenn sie ähnlich hell sind.

Bei dunklen Pixeln ist die Sättigung allerdings wenig aussagekräftig. Da sie den Abstand zwischen größtem und kleinstem Kanal auf den Hellwert bezieht, wiegt ein kleiner Farbstich umso schwerer, je dunkler das Pixel ist: Bei $M = 20$ und $m = 10$ ergibt sich bereits $S = 0{,}5$, obwohl die Farbe kaum wahrnehmbar ist. Sensorrauschen wirkt bei geringer Helligkeit auf die gleiche Weise.

Der CIELAB-Farbraum [7] hat diese Schwäche nicht. Er ist so aufgebaut, dass gleiche Abstände näherungsweise gleich wahrgenommenen Farbunterschieden entsprechen, und beschreibt eine Farbe durch die Helligkeit L^* und die beiden Gegenfarbachsen a^* (Grün–Rot) und b^* (Blau–Gelb). Die Buntheit (Chroma) ist der Abstand von der unbunten Achse,

$$C^*_{ab} = \sqrt{a^{*2} + b^{*2}}, \tag{3.6}$$

und wird anders als die Sättigung nicht auf die Helligkeit bezogen. Ein schwacher Farbstich auf einer dunklen Fläche ergibt deshalb auch nur eine geringe Buntheit. OpenCV speichert a^* und b^* bei 8-Bit-Bildern um 128 verschoben und skaliert L^* auf 0 bis 255.

3.3 Kameramodell und Kalibrierung

Die Abbildung eines Punkts im Raum auf das Kamerabild wird meist mit dem Lochkameramodell beschrieben [8]. Ein Punkt $\mathbf{X} = (X, Y, Z)^\top$ im Welt-Koordinatensystem (KS) wird auf den Bildpunkt (u, v) abgebildet gemäß

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} \mid \mathbf{t} \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \qquad \mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \tag{3.7}$$

Die *extrinsischen* Parameter $\mathbf{R}$ und $\mathbf{t}$ beschreiben Lage und Ausrichtung der Kamera im Raum. Die *intrinsischen* Parameter in der Kameramatrix $\mathbf{K}$ sind die Brennweiten f_x, f_y in Pixeln und der Hauptpunkt (c_x, c_y), an dem die optische Achse die Bildebene trifft. Der Faktor λ berücksichtigt, dass alle Punkte auf einem Sehstrahl auf denselben Bildpunkt fallen.

Reale Objektive weichen von diesem Modell ab. Am stärksten wirkt die radiale Verzeichnung, bei der gerade Linien zum Bildrand hin gekrümmt erscheinen. Sie wird durch ein Polynom im Abstand r von der Bildmitte modelliert,

$$x_{\text{verz}} = x\bigl(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\bigr), \tag{3.8}$$

und analog für y; hinzu kommen kleinere tangentiale Anteile.

Bei der Kalibrierung werden die intrinsischen Parameter und die Verzeichnungskoeffizienten bestimmt. Verbreitet ist das Verfahren nach Zhang [9]: Ein ebenes Muster mit bekannter Geometrie, etwa ein Schachbrett, wird aus mehreren Blickwinkeln aufgenommen. Aus den Punktkorrespondenzen jeder Aufnahme wird zunächst eine geschlossene Lösung berechnet und diese anschließend durch eine nichtlineare Optimierung des Rückprojektionsfehlers verfeinert. Mit den ermittelten Parametern lässt sich jedes Bild der Kamera entzerren.

3.4 Fiducial Marker und Homographie

3.4.1 ArUco-Marker

Fiducial Marker sind künstliche Muster, die in einem Bild zuverlässig gefunden und eindeutig identifiziert werden können. ArUco-Marker [10] sind quadratisch und bestehen

aus einem schwarzen Rand und einem inneren Gitter aus schwarzen und weißen Feldern, das eine Kennung codiert. Die zulässigen Codes sind in Wörterbüchern zusammengefasst; ein Wörterbuch wie `DICT_4X4_50` enthält 50 Marker mit 4×4 Datenbits, die so gewählt sind, dass sie sich möglichst stark voneinander unterscheiden. Dadurch lassen sich einzelne falsch gelesene Bits erkennen oder korrigieren.

Die Detektion läuft in mehreren Schritten ab [10]: Das Graustufenbild wird mit einem adaptiven Schwellwert binarisiert, Konturen werden gesucht und durch Polygone angenähert, und nur konvexe Vierecke bleiben als Kandidaten übrig. Jeder Kandidat wird perspektivisch auf ein Quadrat entzerrt, in das Gitter der Marker unterteilt und bitweise ausgelesen. Stimmt das Bitmuster, gegebenenfalls in einer der vier Drehlagen, mit einem Marker des Wörterbuchs überein, gilt er als erkannt. Die Ecken des Markers sind dann im Bild subpixelgenau bekannt, und aus der Drehlage ergibt sich, welche Ecke welche ist.

Eine Voraussetzung ist, dass der schwarze Rand des Markers sich klar von seiner Umgebung abhebt. Marker werden deshalb üblicherweise mit einem weißen Rand gedruckt.

3.4.2 Homographie

Liegen alle betrachteten Punkte in einer Ebene, etwa der Tischebene, vereinfacht sich das Kameramodell aus Gleichung (3.7). Wählt man das Welt-KS so, dass die Ebene $Z = 0$ ist, entfällt die dritte Spalte von $\mathbf{R}$, und die Abbildung zwischen einem Punkt (x, y) der Ebene und seinem Bildpunkt (u, v) ist eine Homographie [8]:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{H} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \qquad \mathbf{H} \in \mathbb{R}^{3\times 3}. \tag{3.9}$$

Da $\mathbf{H}$ nur bis auf einen Faktor bestimmt ist, hat sie acht Freiheitsgrade. Jede Punktkorrespondenz liefert zwei Gleichungen, sodass vier Korrespondenzen, von denen keine drei auf einer Geraden liegen, zur Bestimmung genügen. Beim Direct Linear Transform (DLT) werden die Gleichungen aller Korrespondenzen zu einem linearen Gleichungssystem $\mathbf{A}\mathbf{h} = \mathbf{0}$ für die neun Einträge $\mathbf{h}$ von $\mathbf{H}$ zusammengefasst. Die Lösung ist der rechte Singulärvektor von $\mathbf{A}$ zum kleinsten Singulärwert [8]. Bei mehr als vier Korrespondenzen liefert dasselbe Vorgehen eine Lösung im Sinne der kleinsten Quadrate.

Mit der Umkehrung von $\mathbf{H}$ lässt sich jedes Kamerabild in eine Draufsicht der Ebene umrechnen, in der Längen und Winkel unabhängig von der Kameraperspektive sind.

Das gilt jedoch nur für Punkte in der Ebene selbst. Punkte oberhalb der Ebene, etwa die Oberseite eines Objekts, werden in der Draufsicht versetzt abgebildet (Parallaxe), und zwar umso stärker, je höher sie liegen und je weiter sie von dem Punkt direkt unter der Kamera entfernt sind.

3.5 Koordinatentransformationen

Lagen und Bewegungen in der Robotik werden in verschiedenen Koordinatensystemen angegeben, zwischen denen umgerechnet werden muss. Eine Starrkörpertransformation besteht aus einer Drehung $\mathbf{R}$ und einer Verschiebung $\mathbf{t}$ und erhält Längen und Winkel. In homogenen Koordinaten, bei denen einem Punkt eine zusätzliche 1 angehängt wird, lässt sie sich als eine einzige Matrixmultiplikation schreiben. In der Ebene gilt

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\varphi & -\sin\varphi & t_x \\ \sin\varphi & \cos\varphi & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}. \tag{3.10}$$

Mehrere Transformationen hintereinander ergeben sich als Produkt der Matrizen. Im Raum hat die Matrix die Größe 4×4 mit einer 3×3-Drehmatrix.

Ist die Transformation zwischen zwei Koordinatensystemen nicht bekannt, lässt sie sich aus Punkten bestimmen, deren Koordinaten in beiden Systemen gemessen wurden. Gesucht sind $\mathbf{R}$ und $\mathbf{t}$, die die Summe der quadratischen Abstände

$$E(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^{n} \left\| \mathbf{R}\,\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i \right\|^2 \tag{3.11}$$

zwischen den transformierten Punkten $\mathbf{p}_i$ und ihren Gegenstücken $\mathbf{q}_i$ minimieren. Die Lösung lässt sich geschlossen angeben [11]: Beide Punktmengen werden um ihre Schwerpunkte $\bar{\mathbf{p}}$ und $\bar{\mathbf{q}}$ zentriert, und die Kovarianzmatrix $\mathbf{C} = \sum_i (\mathbf{q}_i - \bar{\mathbf{q}})(\mathbf{p}_i - \bar{\mathbf{p}})^\top$ wird per Singulärwertzerlegung $\mathbf{C} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ zerlegt. Dann ist

$$\mathbf{R} = \mathbf{U}\,\mathbf{D}\,\mathbf{V}^\top, \qquad \mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\,\bar{\mathbf{p}}, \tag{3.12}$$

wobei die Diagonalmatrix $\mathbf{D} = \mathrm{diag}\big(1, \det(\mathbf{U}\mathbf{V}^\top)\big)$ (in der Ebene) sicherstellt, dass eine Drehung und keine Spiegelung entsteht.

3.6 Industrierobotik und Greifen

3.6.1 Kinematik und Koordinatensysteme

Ein Industrieroboter mit sechs Drehgelenken ist eine offene kinematische Kette aus Gliedern, die über Gelenke verbunden sind [12]. Die Stellung aller Gelenke, der Gelenkwinkelvektor, legt eindeutig fest, wo sich das Werkzeug befindet. Die Berechnung dieser Lage aus den Gelenkwinkeln heißt *Vorwärtskinematik*. Für Aufgaben wie das Greifen ist meist die umgekehrte Frage wichtig: Welche Gelenkwinkel führen das Werkzeug an eine gewünschte Lage? Diese *inverse Kinematik* hat im Allgemeinen mehrere Lösungen, und die Steuerung wählt eine davon, meist die nächstgelegene zur aktuellen Stellung.

Für die Beschreibung von Lagen werden mehrere KS verwendet. Das Basis-KS ist fest mit dem Sockel des Roboters verbunden. Das Werkzeug-KS liegt im TCP, dem Arbeitspunkt des Werkzeugs, bei einem Greifer typischerweise zwischen den Greiferbacken. Eine Pose des Werkzeugs besteht aus der Position des TCP im Basis-KS und der Orientierung des Werkzeug-KS, die häufig durch drei Drehwinkel (Roll, Nick, Gier) angegeben wird.

3.6.2 Bewegungsarten

Steuerungen bieten im Wesentlichen zwei Arten an, eine Zielpose anzufahren:

Point-to-Point (PTP)-Bewegung Alle Gelenke fahren von ihrer Start- zu ihrer Zielstellung. Der Weg des TCP im Raum ist dabei nicht festgelegt, dafür ist die Bewegung schnell und frei von Problemen durch singuläre Stellungen. Sie eignet sich für Wege durch freien Raum.

LIN-Bewegung Der TCP bewegt sich auf einer Geraden zur Zielpose. Die Steuerung berechnet dafür laufend die inverse Kinematik. Die Bewegung ist vorhersehbar und eignet sich zum Anfahren und Verlassen von Greif- und Ablagepositionen.

3.6.3 Greifer und Greifstrategie

Industrielle Greifer halten Werkstücke mechanisch mit Backen, über Unterdruck mit Saugnäpfen oder magnetisch [12]. Ein Parallelgreifer hat zwei Backen, die sich parallel zueinander schließen, und hält ein Werkstück an zwei gegenüberliegenden Flächen.

Für einen Griff müssen Greifposition und Greiforientierung festgelegt werden. Bei flach liegenden Teilen und einem von oben greifenden Werkzeug reduziert sich das auf die

Position in der Ebene, die Greifhöhe und die Drehung des Greifers um die senkrechte Achse. Die Anfahrt erfolgt üblicherweise über eine Vorposition oberhalb des Teils, von der aus der Greifer geradlinig abgesenkt wird, damit er das Teil nicht seitlich verschiebt.

3.6.4 Kollaborative Roboter

Kollaborative Roboter (kollaborativer Roboter (Cobot)s) sind dafür ausgelegt, ohne trennende Schutzeinrichtung in der Nähe von Menschen zu arbeiten. Sie begrenzen dazu Kräfte und Geschwindigkeiten und erkennen Kollisionen, etwa über Drehmomentsensoren in den Gelenken [12]. Die Anforderungen an solche Anwendungen sind in Normen festgelegt; ob eine Anwendung sicher ist, hängt aber nicht allein vom Roboter ab, sondern auch von Werkzeug, Werkstück und Bewegungsablauf und muss für jede Anwendung beurteilt werden.

Normen zu Industrie- und kollaborativen Robotern (ISO 10218, ISO/TS 15066) zitieren, falls gewünscht (keine DOI, als Norm aufnehmen).

3.7 Optische Messtechnik und Prüfmerkmale

3.7.1 Beleuchtung

Für die optische Prüfung ist die Beleuchtung oft wichtiger als der Algorithmus. Bei Auflicht befindet sich die Lichtquelle auf der Seite der Kamera und beleuchtet die sichtbare Oberfläche; diffuses Licht vermeidet dabei Glanzstellen und harte Schatten. Bei Durchlicht liegt das Objekt zwischen Lichtquelle und Kamera, und seine Umrisse und Öffnungen erscheinen mit hohem Kontrast. Eine gleichbleibende Beleuchtung macht die Ergebnisse reproduzierbar.

3.7.2 Kantendetektion

Kanten sind Stellen, an denen sich die Helligkeit stark ändert. Sie werden über den Gradienten des Bildes gefunden, der meist mit kleinen Differenzfiltern wie dem Sobel-Operator berechnet wird. Der Kantendetektor nach Canny [13] verbindet mehrere Schritte zu einem robusten Verfahren: Glättung mit einem Gaußfilter, Berechnung von Betrag und Richtung des Gradienten, Ausdünnen der Kanten auf eine Breite von einem Pixel (Non-Maximum Suppression) und eine Schwellwertbildung mit zwei Schwellen

(Hysterese). Starke Kanten werden dabei immer übernommen, schwache nur, wenn sie an eine starke Kante anschließen.

3.7.3 Detektion von Kreisen und Ellipsen

Die Hough-Transformation [14] findet geometrische Formen, indem jeder Kantenpunkt für alle Formen „stimmt“, auf denen er liegen könnte. Für einen Kreis $(x-a)^2+(y-b)^2 = r^2$ ist der Parameterraum dreidimensional; die Einträge eines Akkumulators über (a, b, r) werden für jeden Kantenpunkt erhöht, und Maxima entsprechen den gesuchten Kreisen. Das Verfahren ist robust gegenüber Lücken und Störungen in den Kanten, seine Genauigkeit ist aber durch die Auflösung des Akkumulators begrenzt.

Ist die Kontur eines Kreises oder einer Ellipse bereits bekannt, lässt sich die Form genauer durch Anpassung an alle Konturpunkte bestimmen. Eine allgemeine Kegelschnittgleichung

$$ax^2 + bxy + cy^2 + dx + ey + f = 0 \tag{3.13}$$

wird dabei so gewählt, dass die Summe der quadrierten Abweichungen der Punkte minimal wird. Das Verfahren nach Fitzgibbon, Pilu und Fisher [15] verwendet dafür die Nebenbedingung $4ac - b^2 = 1$, die garantiert, dass das Ergebnis immer eine Ellipse ist, und führt die Anpassung auf ein verallgemeinertes Eigenwertproblem zurück. Da ein kreisrundes Loch bei leicht schräger Ansicht als Ellipse erscheint, eignet sich eine Ellipsenanpassung auch zur Messung von Kreisen.

3.7.4 Formmerkmale und Template-Matching

Um zu entscheiden, ob ein gefundener Bereich eine bestimmte Form hat, werden einfache Formmerkmale herangezogen. Die Rundheit setzt die Fläche eines Bereichs ins Verhältnis zur Fläche seines kleinsten umschließenden Kreises und ist für einen idealen Kreis gleich 1. Die Rechteckigkeit setzt die Fläche ins Verhältnis zur Fläche des kleinsten umschließenden Rechtecks und ist für ein ideales Rechteck gleich 1.

Ein anderer Ansatz ist das Template-Matching [16]. Ein Referenzbild des gesuchten Merkmals wird über das Bild geschoben und an jeder Position mit dem darunter liegenden Bildausschnitt verglichen, etwa über die normierte Kreuzkorrelation. Die Position mit der höchsten Übereinstimmung ist der Fundort, und die Höhe der Übereinstimmung zeigt, ob das Merkmal vorhanden ist. Das Verfahren ist einfach, reagiert aber auf Änderungen von Größe, Drehung und Beleuchtung gegenüber der Referenz.

3.7.5 Umrechnung in metrische Größen

Längen im Bild werden in Pixeln gemessen. Für eine Angabe in Millimetern wird ein Maßstab benötigt, der aus einer Kalibrierung mit einem Referenzobjekt bekannter Größe stammt, etwa einer Messplatte, oder aus einer bekannten Abmessung des Prüfobjekts selbst. Der Maßstab gilt nur für Objekte im gleichen Abstand zur Kamera, und bei schräger Ansicht unterscheidet er sich in verschiedenen Richtungen. Die erreichbare Genauigkeit hängt von der Auflösung, also der Anzahl Pixel pro Millimeter, und davon ab, wie genau die Kanten lokalisiert werden. Verfahren, die viele Konturpunkte gemeinsam auswerten, wie die Ellipsenanpassung, erreichen dabei Genauigkeiten unterhalb eines Pixels.

3.8 Texterkennung (OCR)

Die optische Zeichenerkennung (Optical Character Recognition (OCR)) wandelt Text in einem Bild in maschinenlesbare Zeichen um. Eine OCR-Pipeline umfasst typischerweise die Vorverarbeitung des Bildes (Kontrastverbesserung, Binarisierung, Ausrichtung), die Lokalisierung der Textbereiche, die Erkennung der Zeichen und eine Nachbearbeitung, etwa den Abgleich mit einem erwarteten Format oder Wörterbuch.

Klassische Systeme wie Tesseract [17] binarisieren das Bild, suchen zusammenhängende Bereiche, fassen sie zu Zeilen und Wörtern zusammen und erkennen die Zeichen anschließend mit einem Klassifikator. Sie sind auf gut lesbaren, dunklen Text auf hellem Grund, wie in gescannten Dokumenten, ausgelegt.

Lernbasierte Verfahren trennen Textdetektion und Texterkennung in zwei neuronale Netze. Für die Detektion bestimmt etwa CRAFT [18] für jeden Bildpunkt, wie wahrscheinlich er zur Mitte eines Zeichens gehört und wie wahrscheinlich er zwischen zwei zusammengehörigen Zeichen liegt, und setzt daraus Wortbereiche zusammen. Für die Erkennung verarbeitet ein Convolutional Recurrent Neural Network (CRNN) [19] den Textausschnitt zunächst mit einem faltenden Netz (Convolutional Neural Network (CNN)), liest die entstehenden Merkmale anschließend mit einem rekurrenten Netz von links nach rechts und gibt die Zeichenfolge aus, ohne dass die einzelnen Zeichen vorher getrennt werden müssen. Die Bibliothek EasyOCR [20] kombiniert diese beiden Verfahren mit vortrainierten Modellen. Solche Verfahren kommen auch mit Text in natürlichen Szenen, unterschiedlichen Schriftarten und ungleichmäßiger Beleuchtung zurecht.

Ist der erwartete Text bekannt, etwa eine Seriennummer aus einer festen Anzahl von Ziffern, lässt sich die Erkennung auf die zulässigen Zeichen beschränken und das Ergebnis mit einem regulären Ausdruck prüfen. Das verringert Verwechslungen, zum Beispiel zwischen der Ziffer 0 und dem Buchstaben O.

3.9 Lernbasiertes Greifen

3.9.1 Greifposen und Policies

Lernbasierte Greifverfahren lassen sich grob danach unterscheiden, was das neuronale Netz ausgibt. *Greifposenbasierte* Verfahren schätzen aus einem Bild, wo und wie gegriffen werden soll. Das Generative Grasping CNN (GG-CNN) [21] bestimmt zum Beispiel für jeden Bildpunkt eines Tiefenbildes eine Greifqualität, einen Greifwinkel und eine Öffnungsweite. Die beste Greifpose wird anschließend klassisch angefahren. Das Netz ersetzt damit die Objekterkennung, die Bewegung bleibt programmiert.

Eine *Policy* (Steuerungsstrategie) bildet dagegen eine Beobachtung direkt auf eine Bewegung ab. Sie erhält in jedem Takt die Kamerabilder und den Zustand des Roboters und gibt die nächsten Sollwerte der Gelenke und des Greifers aus. Ein solches Netz übernimmt Erkennung und Bewegungsplanung gemeinsam und kann während der Fahrt auf Änderungen in der Szene reagieren. Policies, die aus Bildern Bewegungen erzeugen, heißen auch *visuomotorisch*.

3.9.2 Imitation Learning und Behavior Cloning

Eine Policy kann aus Vorführungen gelernt werden, ohne dass eine Belohnungsfunktion formuliert werden muss. Dieses Lernen aus Demonstrationen (Imitation Learning) ist für die Robotik ausführlich beschrieben [22], [23]. Die einfachste Form ist das Behavior Cloning (BC). Aus vorgeführten Fahrten entstehen Paare aus Beobachtung o_t und der dabei ausgeführten Aktion a_t, und die Policy π_θ wird wie bei einer gewöhnlichen Regression darauf trainiert, die Aktion aus der Beobachtung vorherzusagen:

$$\theta^* = \arg\min_\theta \sum_{(o_t, a_t)} \ell\Big(\pi_\theta(o_t),\, a_t\Big). \tag{3.14}$$

Das Verfahren hat eine bekannte Schwäche. Die Policy sieht im Training nur Zustände, die in den Demonstrationen vorkommen. Weicht sie bei der Ausführung etwas ab, gerät sie in Zustände, für die sie keine Beispiele hat, und der Fehler kann sich von Takt zu Takt

aufschaukeln. Ross, Gordon und Bagnell [24] zeigen, dass der erwartete Fehler dadurch im ungünstigen Fall quadratisch mit der Länge der Aufgabe wächst, und schlagen mit DAgger ein Verfahren vor, bei dem ein Experte die von der Policy tatsächlich besuchten Zustände nachträglich mit der richtigen Aktion versieht. Laskey et al. [25] gehen mit DART einen anderen Weg: Schon während der Demonstration wird Rauschen auf die Bewegung gegeben, sodass der Roboter von der Idealbahn abweicht. Die Aktionen, die den Roboter zurückführen, landen im Datensatz, und die Policy lernt dadurch auch, Abweichungen zu korrigieren.

Für ein Rauschen, das sich glatt über die Zeit ändert statt in jedem Takt zu springen, eignet sich der Ornstein-Uhlenbeck-Prozess [26]:

$$\mathrm{d}x = -\frac{x}{\tau}\,\mathrm{d}t + \sigma\,\mathrm{d}W. \tag{3.15}$$

Die Abweichung x wird mit der Zeitkonstante τ zur Null zurückgezogen und durch den Zufallsanteil $\sigma\,\mathrm{d}W$ (Wiener-Prozess) immer wieder ausgelenkt. Das Ergebnis ist eine langsame, schwankende Auslenkung mit begrenzter Amplitude.

3.9.3 Diffusionsmodelle

Diffusionsmodelle [27] sind generative Modelle, die eine Probe schrittweise aus reinem Rauschen erzeugen. Im Training wird einer Probe x_0 (zum Beispiel einem Bild) in K Schritten immer mehr Gaußsches Rauschen hinzugefügt, bis nur noch Rauschen übrig ist. Ein neuronales Netz lernt, diesen Vorgang umzukehren, also aus einer verrauschten Probe x_k und dem Schritt k die Probe einen Schritt weniger verrauscht zu schätzen. Zur Erzeugung beginnt man bei zufälligem Rauschen und wendet das Netz K-mal an. Das Netz kann dabei entweder das enthaltene Rauschen vorhersagen (Vorhersageziel ε) oder direkt die unverrauschte Probe x_0.

Viele Schritte machen die Erzeugung langsam. Denoising Diffusion Implicit Models (DDIM) [28] verwenden ein Netz, das mit K Schritten trainiert wurde, und überspringen bei der Erzeugung Zwischenschritte. Mit zehn statt hundert Schritten verkürzt sich die Rechenzeit etwa um den Faktor zehn.

3.9.4 Diffusion Policy

Die Diffusion Policy [29] wendet dieses Prinzip auf Robotersteuerungen an. Die erzeugte Probe ist kein Bild, sondern eine Folge zukünftiger Aktionen, und das Netz wird zusätzlich mit den letzten Beobachtungen gespeist. Die Kamerabilder werden dafür von

einem CNN wie ResNet [30] in Merkmalsvektoren übersetzt, mit dem Roboterzustand verbunden und an ein Netz übergeben, das die Aktionsfolge schrittweise entrauscht.

Gegenüber einer direkten Regression nach Gleichung (3.14) hat das zwei Vorteile. Erstens kann ein Diffusionsmodell mehrdeutige Lösungen darstellen. Führen zwei Demonstrationen auf unterschiedlichen Wegen zum selben Ziel, wählt es einen davon, während eine Regression den Mittelwert beider Wege lernen würde, der unter Umständen durch ein Hindernis führt. Zweitens sagt das Modell eine ganze Folge von Aktionen voraus statt nur der nächsten, was die Bewegung über mehrere Takte gleichmäßig macht.

Ausgeführt wird jeweils nur der vordere Teil der vorhergesagten Folge. Danach wird mit einer neuen Beobachtung erneut vorhergesagt. Dieses Vorgehen mit überlappenden Aktionsblöcken (Action Chunking) beschreiben Zhao et al. [31]; überlappende Vorhersagen für denselben Takt werden dort gemittelt, was ruckartige Übergänge zwischen zwei Blöcken glättet.

Eine frei verfügbare Umsetzung der Diffusion Policy und weiterer Verfahren enthält die Bibliothek LeRobot [32]. Sie legt außerdem ein Datensatzformat fest, in dem Zustände, Aktionen und Kamerabilder einer Demonstration zeitlich zusammengehörig gespeichert werden.

4 Systemkonzept

Dieses Kapitel beschreibt das Gesamtsystem, bevor die Kapitel 5 bis 8 die einzelnen Module im Detail behandeln. Ausgehend von den Anforderungen werden der Hardwareaufbau, die Softwarearchitektur mit ihren Schnittstellen, die verwendeten Koordinatensysteme und die schrittweise Inbetriebnahme vorgestellt.

4.1 Anforderungen

Das Projekt ist erfolgreich, wenn ein Bauteil, das von Hand an beliebiger Stelle in den Arbeitsbereich gelegt wurde, ohne weiteren Eingriff erkannt, gegriffen, geprüft und entsprechend dem Prüfergebnis abgelegt wird. Die Prüfmerkmale dienen dabei der Demonstration; es werden keine industriellen Toleranzen angestrebt. Aus dieser Zielsetzung ergeben sich die funktionalen Anforderungen in Tabelle 4.1 und die nichtfunktionalen Anforderungen in Tabelle 4.2.

Tabelle 4.1: Funktionale Anforderungen

Nr.	Anforderung
F1	Das System erkennt ein Bauteil im Arbeitsbereich und bestimmt dessen Position und Orientierung in der Tischebene.
F2	Der Roboter greift das Bauteil an der erkannten Pose und legt es an einer festen Prüfposition ab.
F3	An der Prüfposition werden die Seriennummer ausgelesen, der Lochdurchmesser bestimmt und das Vorhandensein der Kerbe geprüft.
F4	Aus den Prüfmerkmalen wird entschieden, ob ein Gut- oder ein Schlechtteil vorliegt; bei einem Schlechtteil werden die Gründe festgehalten.
F5	Der Roboter nimmt das Bauteil an der Prüfposition wieder auf und legt es auf der Ablage für Gut- bzw. Schlechtteile ab.
F6	Der gesamte Ablauf aus F1 bis F5 läuft ohne manuellen Eingriff als Zyklus durch.
F7	Tritt ein Fehler auf, stoppt der Roboter und das System wartet auf eine Quittierung durch den Bediener.

Tabelle 4.2: Nicht-funktionale Anforderungen

Nr.	Anforderung
N1	**Robustheit:** Erkennung und Prüfung funktionieren bei normalem Raumlicht und leichten Schatten.
N2	**Modularität:** Erkennung, Robotersteuerung und Prüfung sind voneinander getrennt und über definierte Schnittstellen verbunden; ein Modul lässt sich austauschen, ohne die übrigen zu ändern.
N3	**Sicherheit:** Der Roboter bewegt sich mit reduzierter Geschwindigkeit und nur innerhalb eines festgelegten Arbeitsraums; ein Stopp ist jederzeit möglich.
N4	**Konfigurierbarkeit:** Aufbauabhängige Werte wie Posen, Kameraeinstellungen und Prüfgrenzen stehen in einer Konfigurationsdatei und nicht im Programmcode.
N5	**Nachvollziehbarkeit:** Jeder Zyklus wird mit seinem Ergebnis protokolliert.
N6	**Einfachheit:** Es werden bewusst einfache, nachvollziehbare Verfahren der klassischen Bildverarbeitung eingesetzt.

4.2 Hardwareaufbau

Der Demonstrator besteht aus einem Roboter mit Greifer, zwei fest montierten Kameras, einem durch Marker begrenzten Arbeitsbereich und drei Ablagepositionen. Eine dritte Kamera am Greifer nutzt nur das lernbasierte Greifen aus Kapitel 8. Gesteuert wird der Demonstrator von einem Rechner, der über Local Area Network (LAN) mit der Robotersteuerung verbunden ist. Abbildung 4.1 zeigt den Aufbau, Tabelle 4.3 fasst die Komponenten zusammen.

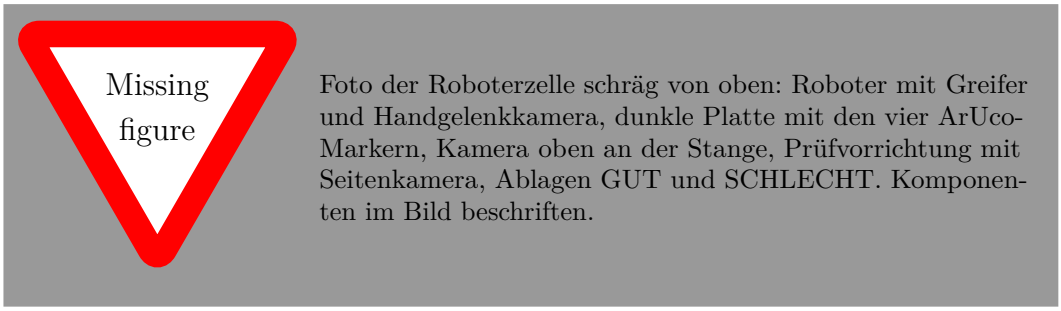


Abbildung 4.1: Aufbau des Demonstrators

Die Fase des Bauteils ist für die Erkennung wichtig: Ohne sie wäre die Draufsicht des Quaders punktsymmetrisch und die Orientierung nur bis auf 180° bestimmbar. Mit der Fase ist die Lage eindeutig, sodass das Bauteil an der Prüfposition immer mit derselben Seite zur Seitenkamera abgelegt werden kann.

Die Form des Bauteils ist bewusst einfach gehalten. Die beiden langen Seitenflächen des Quaders sind parallel, sodass der Parallelgreifer das Bauteil mit seinen Greifbacken direkt an diesen Flächen fassen kann. Eigene Greifbacken oder ein eigener Greifer

Tabelle 4.3: Komponenten des Demonstrators

Komponente	Beschreibung
Roboter	Neura Robotics LARA, sechs Achsen, Steuerungssoftware Version 5.0.8, Ansteuerung über die Python-Schnittstelle NeuraPy [33]
Greifer	Elektrischer 2-Backen-Parallelgreifer Zimmer Group MATCH LWR50L-23-00004-A: Hub 16 mm je Backe, Greifkraft einstellbar bis 500 N, Schließzeit 0,3 s, Wiederholgenauigkeit ±0,02 mm, Masse 1,4 kg, Ansteuerung über IO-Link [34]. In der Steuerung als Werkzeug „ZimmerLWR50“ mit dem TCP 197,5 mm vor dem Flansch angelegt (Greifer mit Backen 169,5 mm, Adapter 28 mm); Sollwerte der Last 1,75 kg, Schwerpunkt etwa 81 mm vor dem Flansch Lastdaten am Bedienhandgerät mit den Sollwerten vergleichen.
Adapter	Zimmer Group MATCH-Robotermodul LWR50F-13-05-A zwischen Roboterflansch und Greifer: Schnellwechsler mit integrierter elektrischer Energieübertragung, Anschlussflansch nach EN ISO 9409-1 (TK 50), Masse 0,35 kg [35]
Top-down-Kamera	Daheng VEN-161, 1440 × 1080 Pixel, Global Shutter, Universal Serial Bus (USB)3 Vision; an einer Stange schräg über dem Arbeitsbereich montiert, zugleich Szenenkamera von Modul 4 Montagehöhe und Objektiv angeben
Seitenkamera	Universal Serial Bus (USB)-Kamera an der Prüfposition; die Testbilder haben 3840 × 2160 Pixel Kameramodell, Abstand, Beleuchtung
Handgelenkkamera	Daheng VEN-161-61U3C mit Fischaugenobjektiv am Greifer, nur für Modul 4 (Tabelle 8.1)
Arbeitsbereich	dunkle Platte mit vier ArUco-Markern auf Papier (Kantenlänge 50 mm, Wörterbuch `DICT_4X4_50`, IDs 0 bis 3), Abstand der Markermitten 400 mm; der Arbeitsbereich ist das Quadrat von 330 × 330 mm zwischen den Markern
Bauteil	3D-gedruckter Quader, 60 × 40 × 15 mm, mit einer abgeschrägten Ecke (Fase); an einer langen Seitenfläche ein aufgeklebtes Etikett mit fünfstelliger Seriennummer, ein Durchgangsloch mit 10 mm Durchmesser und eine kleine rechteckige Öffnung (Kerbe)
Ablagen	3D-gedruckte Prüfvorrichtung (Auflage 38 mm über der Platte), Ablage GUT und Ablage SCHLECHT; alle drei als feste Posen hinter der oberen Markerkante
Rechner	Windows-Laptop, LAN-Verbindung zur Robotersteuerung; Python 3.11 für Bedienoberfläche und Module 1 bis 3, eine zweite Umgebung mit Python 3.12 für Modul 4

mussten deshalb nicht entwickelt werden. Voraussetzung ist nur, dass die Breite des Bauteils in den Hub der Greifbacken passt.

Bauteilbreite (40 mm) gegen die Öffnungsweite der verwendeten Greifbacken prüfen (Hub 16 mm je Backe).

Der Arbeitsbereich liegt 35 mm innerhalb der Markermitten. So bleibt das weiße Papier der Marker außerhalb und wird nicht als Bauteil erkannt.

Prüfvorrichtung beschreiben: Führt sie das Bauteil seitlich (Anschlag), oder ergibt sich die Lage nur aus der Ablegepose des Roboters?

4.3 Softwarearchitektur

4.3.1 Modulstruktur

Die Software ist in Python geschrieben und in vier Module sowie einen zentralen Orchestrator gegliedert (Abb. 4.2). Der Orchestrator steuert den Ablauf; die Module kennen sich gegenseitig nicht und tauschen ausschließlich festgelegte Datenobjekte aus. Gemeinsam genutzte Funktionen wie Konfiguration, Kamerazugriff und Koordinatentransformationen liegen in einem Paket `common`.

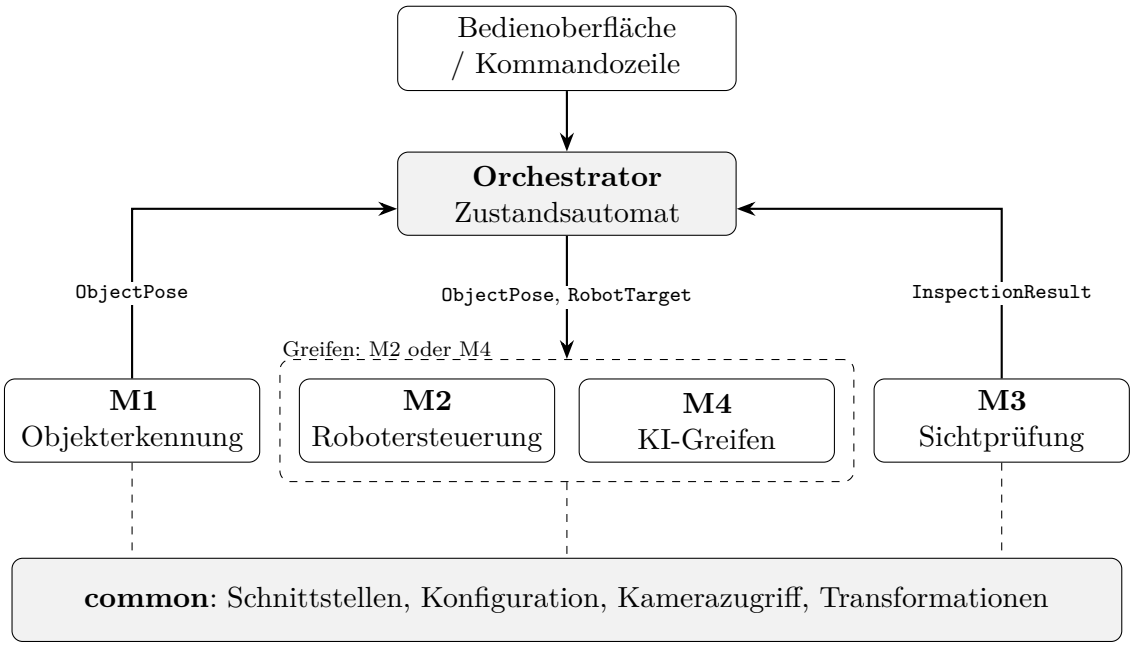


Abbildung 4.2: Modulstruktur der Software. Das Greifen übernimmt wahlweise die Robotersteuerung (M2) oder das lernbasierte Greifen (M4); alle übrigen Bewegungen führt M2 aus.

Für jedes Modul gibt es eine abstrakte Schnittstelle und mehrere Implementierungen, darunter immer eine Mock-Implementierung, die zufällige, aber plausible Ergebnisse liefert, ohne auf Hardware zuzugreifen. Welche Implementierung verwendet wird, legt die Konfiguration fest. Dadurch lässt sich jedes Modul isoliert entwickeln, während die übrigen gemockt werden. Auf dieselbe Weise ist das Greifen austauschbar: Statt der Robotersteuerung aus Kapitel 6 kann das lernbasierte Greifen aus Kapitel 8 das Bauteil aufnehmen und an die Prüfposition bringen, ohne dass sich am Ablauf etwas ändert.

4.3.2 Ablaufsteuerung

Der Orchestrator bildet einen Arbeitszyklus als endlichen Zustandsautomaten ab (Abb. 4.3). Jeder Zustand ruft genau eine Methode eines Moduls auf; die Zuordnung zeigt Tabelle 4.4. Wird im Zustand `DETECT` kein Bauteil gefunden, kehrt der Automat ohne Fehler nach `IDLE` zurück. Jede Ausnahme eines Moduls führt in den Zustand `ERROR`, in dem der Roboter gestoppt wird. Erst nach einer Quittierung durch den Bediener geht es von dort zurück nach `IDLE`. Details zur Fehlerbehandlung und zur Bedienung folgen in Kapitel 9.

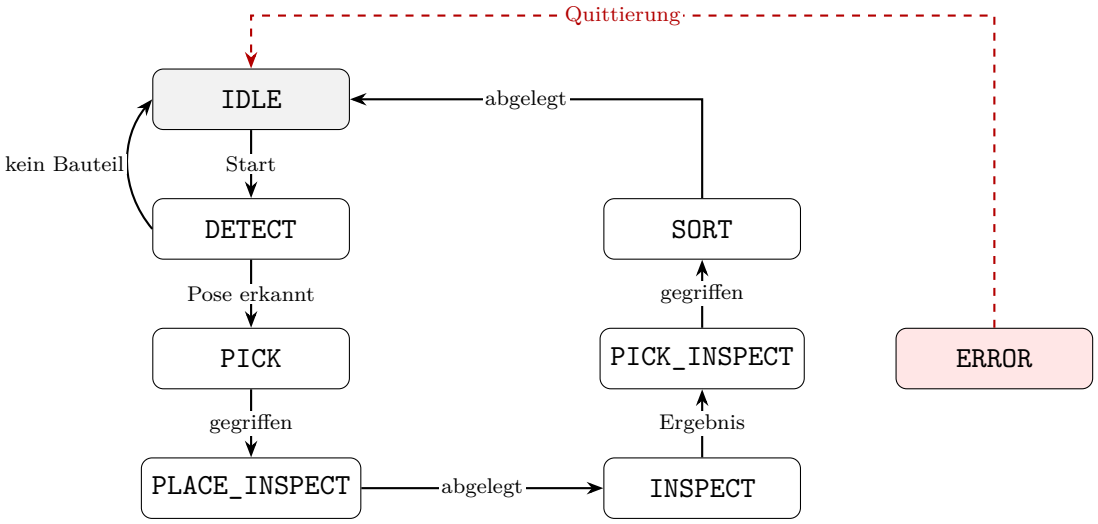


Abbildung 4.3: Zustandsautomat eines Arbeitszyklus. Aus jedem Zustand außer `IDLE` führt ein Fehler in den Zustand `ERROR` (nicht eingezeichnet).

4.3.3 Schnittstellen

Zwischen den Modulen werden drei Datenobjekte ausgetauscht (Tabelle 4.5). Längen werden durchgehend in Millimetern, Winkel in Grad angegeben; die Umrechnung in die Einheiten der Roboterschnittstelle übernimmt die Robotersteuerung.

Tabelle 4.4: Zustände des Arbeitszyklus und beteiligte Module

Zustand	Aktion	Modul
`IDLE`	Warten auf Start bzw. nächsten Zyklus	–
`DETECT`	Bild aufnehmen, Pose des Bauteils bestimmen	M1
`PICK`	Bauteil greifen: M2 an der erkannten Pose, M4 mit der gelernten Policy bis zum Übergabepunkt über der Prüfposition	M2 / M4
`PLACE_INSPECT`	An der Prüfposition ablegen, auf Anfahrhöhe über der Prüfvorrichtung außerhalb des Bilds der Seitenkamera warten	M2
`INSPECT`	Seriennummer, Lochdurchmesser und Kerbe prüfen	M3
`PICK_INSPECT`	Bauteil an der Prüfposition greifen	M2
`SORT`	Auf Ablage GUT oder SCHLECHT ablegen	M2
`ERROR`	Roboter stoppen, Fehler protokollieren, auf Quittierung warten	alle

Tabelle 4.5: Zwischen den Modulen ausgetauschte Datenobjekte

Datenobjekt	Richtung	Inhalt
`ObjectPose`	M1 → M2 (M4)	Position x, y, z und Drehwinkel θ um die Hochachse im Roboter-KS, Konfidenz, Zeitstempel, optional ein Debug-Bild; M4 protokolliert die Pose nur
`RobotTarget`	Konfiguration → M2	Name und kartesische Pose (Position und drei Drehwinkel) einer festen Roboterposition, z. B. Prüfposition oder Ablage
`InspectionResult`	M3 → Orchestrator	gelesene Seriennummer, gemessener Lochdurchmesser, Kerbe vorhanden, Gesamturteil gut/schlecht und Liste der Gründe für ein Schlechtteil

Jedes Modul implementiert eine abstrakte Schnittstelle mit wenigen Methoden (Tabelle 4.6). Fehler meldet ein Modul über eine eigene Ausnahme (Bildverarbeitungs-, Roboter- bzw. Prüffehler), auf die der Orchestrator mit dem Wechsel in den Zustand `ERROR` reagiert.

Tabelle 4.6: Schnittstellen der Module

Schnittstelle	Modul	Methoden
`ObjectDetector`	M1	`detect()`: liefert eine `ObjectPose` oder „kein Bauteil“
`RobotController`	M2	`connect()`, `disconnect()`, `home()`, `pick(pose)`, `place(target)`, `stop()`, `reset()`
`Grasper`	M2, M4	`pick(pose)`, `place(target)`, `stop()`: Greifen und Ablegen an der Prüfposition; Varianten `RobotGrasper` (M2) und `BcGrasper` (M4, legt über M2 ab)
`Inspector`	M3	`inspect()`: liefert ein `InspectionResult`

4.4 Koordinatensysteme

Damit der Roboter ein Bauteil greifen kann, das die Top-down-Kamera erkannt hat, muss dessen Lage vom Kamerabild in das Koordinatensystem des Roboters überführt werden. Dafür werden drei Koordinatensysteme unterschieden:

Bild-KS Pixelkoordinaten (u, v) im Bild der Top-down-Kamera.

Arbeitsbereich-KS Metrisches KS in der Tischebene mit Ursprung in der linken unteren Ecke des Arbeitsbereichs (aus Kamerasicht), x-Achse nach rechts, y-Achse nach oben. Die Mittelpunkte der vier ArUco-Marker liegen in diesem KS bei $(-35, -35)$, $(365, -35)$, $(365, 365)$ und $(-35, 365)$ mm.

Roboter-Basis-KS Das KS der Robotersteuerung, in dem alle Bewegungsbefehle angegeben werden.

Da das Bauteil flach auf dem Tisch liegt, genügt für den Übergang vom Bild in den Arbeitsbereich eine Homographie zwischen Bildebene und Tischebene (Abschnitt 3.4). Sie wird bei jeder Aufnahme aus den vier erkannten Markern berechnet. Der Übergang vom Arbeitsbereich in das Roboter-KS ist eine ebene Starrkörpertransformation aus Drehung und Verschiebung (Abschnitt 3.5), ergänzt um die bekannte Tischhöhe. Die Objekterkennung liefert die `ObjectPose` bereits im Roboter-KS, sodass die Robotersteuerung keine Kenntnis über die Kamera benötigt.

Kalibrierung zwischen Arbeitsbereich und Roboter

Drehung und Verschiebung zwischen Arbeitsbereich und Roboter werden mit dem Roboter selbst eingemessen. Der Bediener setzt die Greiferspitze nacheinander auf die Mittelpunkte der vier Marker, deren Lage im Arbeitsbereich bekannt ist, und übernimmt jeweils die aktuelle TCP-Position. Aus den vier Punktpaaren wird nach der Methode der kleinsten Quadrate die ebene Starrkörpertransformation bestimmt [11]. Für jeden Marker wird anschließend der Restfehler zwischen gemessener und transformierter Position ausgegeben, zusammen mit dem quadratischen Mittelwert (Root Mean Square (quadratischer Mittelwert) (RMS)). Liegt dieser über 3 mm, warnt das Werkzeug; Ursache sind dann meist vertauschte Marker, falsch eingetragene Markerpositionen oder ein nicht mittig gesetzter TCP. Die Kalibrierung setzt voraus, dass der TCP des Werkzeugs in der Steuerung stimmt.

Geprüft wird das Ergebnis mit einem Zeigetest: Die Objekterkennung bestimmt die Pose eines Bauteils, der Roboter fährt den Greifer 30 mm über diese Pose, ohne zu greifen, und der Bediener notiert den sichtbaren Versatz. Kalibrierung und Zeigetest stehen in der Bedienoberfläche und als Kommandozeilenwerkzeug zur Verfügung (Abschnitt 6.7).

Ergebnis der Kalibrierung im Labor eintragen (Restfehler je Marker, RMS) und Versatz aus dem Zeigetest an mehreren Stellen des Arbeitsbereichs.

4.5 Test- und Inbetriebnahmestrategie

Die Module wurden parallel entwickelt, während Kameras und Roboter nur zeitweise zur Verfügung standen. Die Architektur erlaubt deshalb, jedes Modul in mehreren Stufen zu betreiben, die sich schrittweise der realen Anlage annähern (Tabelle 4.7). Die Stufe wird pro Modul in der Konfiguration oder in der Bedienoberfläche gewählt, sodass sich beliebige Kombinationen bilden lassen, etwa eine reale Kamera zusammen mit einem simulierten Roboter.

In der Mock-Stufe läuft der vollständige Zyklus ohne jede Hardware und prüft das Zusammenspiel von Orchestrator und Schnittstellen. In der Wiedergabe-Stufe verarbeiten Erkennung und Prüfung gespeicherte Fotos des echten Aufbaus. Da die Bildverarbeitung in beiden Modulen vom Kamerazugriff getrennt ist, durchläuft ein Testfoto denselben Programmcode wie ein Live-Bild. In der Simulationsstufe spricht die Robotersteuerung mit einer Nachbildung der Steuerung, die dasselbe Kommunikationsprotokoll verwendet. So wird der komplette Programmpfad der Robotersteuerung einschließlich der unveränderten Herstellerbibliothek ausgeführt, ohne dass sich ein Roboter bewegt. Die

Tabelle 4.7: Betriebsstufen der Module

Stufe	Erkennung (M1)	Roboter (M2)	Prüfung (M3)
Mock	zufällige Pose im Arbeitsbereich	Bewegungen werden nur protokolliert, Arbeitsraumgrenzen werden geprüft	zufälliges Prüfergebnis, überwiegend Gutteile
Wiedergabe	gespeicherte Testfotos	–	gespeicherte Testfotos
Simulation	–	eigene Nachbildung der Robotersteuerung mit gleichem Protokoll	–
virtuelle Maschine (VM)	–	offizielle Simulation der Steuerung als virtuelle Maschine des Herstellers	–
Real	Live-Bild der Top-down-Kamera	realer Roboter	Live-Bild der Seitenkamera

eigene Nachbildung kennt keine Kinematik. Die virtuelle Maschine (VM) des Herstellers rechnet dagegen mit dem echten Robotermodell und der echten Steuerungssoftware, sodass Erreichbarkeit, Bewegungsarten und Werkzeugdaten vor dem ersten Test am Roboter geprüft werden können. Das lernbasierte Greifen aus Kapitel 8 bringt eigene Stufen mit: eine eigene Simulation des Roboters, die VM und den realen Roboter, jeweils wahlweise mit Platzhalterbildern statt echter Kameras.

Damit lassen sich große Teile der Software ohne Roboter testen. Das ersetzt jedoch nicht die Erprobung an der realen Anlage, bei der Einflüsse wie Beleuchtung, Greifverhalten und Positioniergenauigkeit erst sichtbar werden.

5 Objekterkennung mit der Top-down-Kamera

Die Objekterkennung bestimmt aus einem Bild der Top-down-Kamera, wo das Bauteil im Arbeitsbereich liegt und wie es gedreht ist. Das Ergebnis übergibt sie als `ObjectPose` im Roboter-KS an den Orchestrator (Abschnitt 4.3.3). Da das Bauteil flach auf dem Tisch liegt, genügen dafür die Position (x, y) in der Tischebene und der Drehwinkel θ um die Hochachse; die Höhe ist durch die Tischhöhe festgelegt.

Die Verarbeitung ist als Kette von Einzelschritten aufgebaut (Abb. 5.1). Sie arbeitet ausschließlich auf dem übergebenen Bild und greift selbst nicht auf die Kamera zu. Dadurch lässt sie sich mit gespeicherten Fotos genauso ausführen wie mit Live-Bildern (Abschnitt 4.5).

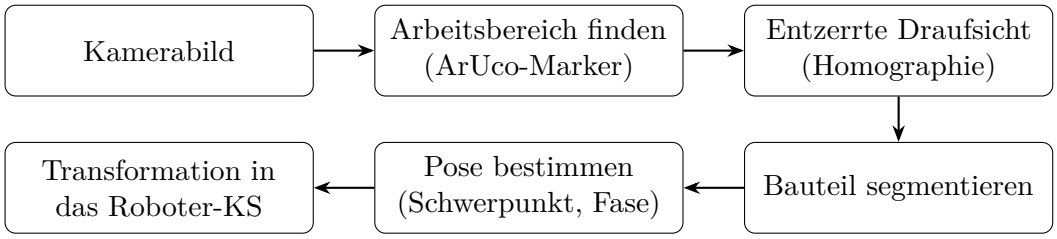


Abbildung 5.1: Verarbeitungskette der Objekterkennung

5.1 Bildaufnahme und Kamerakalibrierung

Die Top-down-Kamera, eine Daheng VEN-161 mit 1440×1080 Pixeln und Global Shutter, ist an einer Stange schräg über dem Arbeitsbereich montiert. Sie wird über das Software-Paket des Herstellers angesprochen und über ihre Seriennummer ausgewählt, da am selben Rechner eine baugleiche Handgelenkkamera für Modul 4 hängt. Dieselbe Kamera dient Modul 4 als Szenenkamera; vor einer Fahrt von Modul 4 gibt die Objekterkennung sie frei und öffnet sie beim nächsten Bild wieder (Abschnitt 8.9). Die bisherigen Testbilder stammen von einer Smartphone-Kamera aus derselben Perspektive.

Für reproduzierbare Ergebnisse soll die Belichtung fest eingestellt werden. Derzeit regelt die Kamera Belichtung und Verstärkung noch automatisch. Eine Korrektur der Linsenverzeichnung mit einer intrinsischen Kalibrierung nach Zhang (Abschnitt 3.3) ist vorgesehen, aber noch nicht umgesetzt. Die Verarbeitung arbeitet derzeit direkt auf dem unkorrigierten Bild.

Montagehöhe ergänzen, feste Belichtung festlegen. Intrinsische Kalibrierung durchführen und beschreiben, sofern sie umgesetzt wird.

5.2 Detektion des Arbeitsbereichs

Der Arbeitsbereich ist durch vier ArUco-Marker aus dem Wörterbuch `DICT_4X4_50` mit den IDs 0 bis 3 festgelegt [10]. Die Marker sind auf Papier gedruckt und auf die dunkle Platte geklebt; der weiße Rand um jeden Marker hebt dessen schwarze Kante vom dunklen Untergrund ab. Ihre Mittelpunkte liegen 400 mm auseinander und sind im Arbeitsbereich-KS in der Konfiguration hinterlegt (Abschnitt 4.4).

Die Marker werden mit dem ArUco-Detektor von OpenCV [2] im Graustufenbild gesucht; die Markerecken werden dabei subpixelgenau verfeinert. Aus den vier Ecken jedes Markers wird dessen Mittelpunkt berechnet. Fehlt einer der vier Marker, bricht die Verarbeitung mit einem Fehler ab, da die Lage des Arbeitsbereichs dann nicht eindeutig bestimmt ist.

Die vier Markermittelpunkte im Bild und ihre bekannten Positionen im Arbeitsbereich bilden vier Punktkorrespondenzen, aus denen eine Homographie zwischen Bildebene und Tischebene berechnet wird (Abschnitt 3.4). Mit ihr wird das Kamerabild in eine entzerrte Draufsicht des Arbeitsbereichs umgerechnet. Sie zeigt das Quadrat von 330×330 mm zwischen den Markern, sodass das helle Markerpapier außerhalb liegt. Die Draufsicht hat einen festen Maßstab von 2 Pixel/mm; ein Pixel entspricht also 0,5 mm auf dem Tisch, und die Perspektive der schräg montierten Kamera ist ausgeglichen. Alle weiteren Schritte arbeiten in dieser Draufsicht. Abbildung 5.3 zeigt ein Kamerabild mit den erkannten Markern und die daraus berechnete Draufsicht.

Da die Marker in jedem Bild neu gefunden werden, ist ein Verschieben der Kamera für die Objekterkennung unkritisch. Werden dagegen die Marker verschoben, müssen ihre Positionen neu vermessen und die Kalibrierung zum Roboter wiederholt werden. Der Code enthält zusätzlich einen Rückfall, der den Arbeitsbereich über die Innenkante eines weißen Rahmens findet; im aktuellen Aufbau gibt es keinen solchen Rahmen mehr, und es werden nur die Marker verwendet.

5.3 Segmentierung des Bauteils

In der entzerrten Draufsicht muss das Bauteil vom Hintergrund getrennt werden. Die Platte des Arbeitsbereichs ist dunkelgrau, das Bauteil dagegen heller oder farbiger. Die Segmentierung prüft deshalb zwei Merkmale (Abschnitt 3.2): die Helligkeit als Hellwert V des HSV-Farbraums und die Buntheit als Chroma C^*_{ab} des CIELAB-Farbraums (Gleichung (3.6)).

Anstelle fester Schwellwerte werden die Grenzen relativ zur Platte bestimmt. Da das Bauteil nur einen kleinen Teil des Arbeitsbereichs bedeckt, beschreibt der Median von Hellwert und Buntheit über die gesamte Draufsicht die Platte. Ein Pixel gilt als Bauteil, wenn sein Hellwert um mehr als 50 oder seine Buntheit um mehr als 20 über dem Wert der Platte liegt. Ändert sich das Raumlicht, verschieben sich Platte und Bauteil gemeinsam, und die Schwelle wandert mit. Außerdem funktioniert die Segmentierung unabhängig von der Farbe des Bauteils, solange es sich in Helligkeit oder Buntheit von der Platte abhebt. Ein globales Schwellwertverfahren wie das nach Otsu (Abschnitt 3.1) wäre eine Alternative, setzt aber ein deutlich zweigipfliges Histogramm voraus, das bei einem kleinen Bauteil auf großer Fläche nicht gegeben ist.

In einer ersten Fassung wurde statt der Buntheit die Sättigung S des HSV-Farbraums verwendet. Auf Testfotos des Aufbaus führte das zu Fehldetektionen. Auf der dunklen Platte spiegelte sich in einer Ecke ein bläulicher Schimmer, und weil die Sättigung bei dunklen Pixeln schon kleine Farbstiche stark gewichtet (Abschnitt 3.2), erreichte dieser Bereich Werte von 45 bis 58. Die Schwelle lag bei etwa 52 (Median der Platte plus 40), der Reflex also teilweise darüber. In einem Bild war die so segmentierte Fläche größer als das Bauteil und wurde an seiner Stelle ausgewählt; in einem anderen verschmolz sie mit dem Bauteil und verfälschte Kontur und Orientierung. In der Buntheit sind die drei Flächen dagegen klar getrennt (Tabelle 5.1): Die Schwelle (Platte plus 20, also 22 bis 23) liegt deutlich über dem Reflex und deutlich unter dem Bauteil. Mit ihr wird das Bauteil auf allen fünf Testfotos korrekt segmentiert.

Tabelle 5.1: Sättigung und Buntheit auf den Testfotos (Median je Fläche, 8-Bit-Werte, fünf Fotos)

Fläche	Sättigung S	Buntheit C^*_{ab}
Platte	12–13	2–3
bläulicher Reflex	45–58	8–9
Bauteil (violett)	76–99	40–52

Die entstehende Binärmaske wird an einem schmalen Streifen von 5 mm entlang des Rands des Arbeitsbereichs beschnitten, damit Reste des Markerpapiers nicht als Bauteil

erkannt werden, und anschließend morphologisch bereinigt: Ein Öffnen entfernt einzelne Störpixel, ein Schließen füllt kleine Lücken im Bauteil. Von den gefundenen Konturen werden nur solche mit plausibler Fläche betrachtet; die größte davon ist das Bauteil. Wird keine passende Kontur gefunden, meldet das Modul „kein Bauteil“, und der Orchestrator kehrt ohne Fehler in den Ruhezustand zurück.

5.4 Bestimmung der Pose

Die Position des Bauteils ist der Flächenschwerpunkt der Kontur, der sich aus den Bildmomenten nullter und erster Ordnung ergibt. Für die Orientierung wird das kleinste umschließende Rechteck der Kontur bestimmt; seine lange Seite gibt die Längsachse des Bauteils an.

Die Längsachse allein legt die Orientierung nur bis auf 180° fest, da ein Quader in der Draufsicht punktsymmetrisch ist. Eindeutig wird sie über die abgeschrägte Ecke des Bauteils: An der Fase fehlt dem Bauteil eine Ecke, sodass genau eine Ecke des umschließenden Rechtecks deutlich außerhalb der Kontur liegt. Für jede Rechteckecke wird der Abstand zur Kontur berechnet. Liegt die am weitesten entfernte Ecke mehr als einen festen Anteil der Bauteilbreite außerhalb, gilt die Fase als gefunden, und die Längsachse wird so ausgerichtet, dass sie zur Fase zeigt. Der Winkel ist dann im Bereich 0° bis 360° eindeutig. Wird keine Fase gefunden, wird der Winkel auf 0° bis 180° begrenzt und die Konfidenz der Pose herabgesetzt. Abbildung 5.2 fasst das Vorgehen zusammen.

5.5 Transformation in das Roboterkoordinatensystem

Schwerpunkt und Winkel liegen zunächst in Pixeln der Draufsicht vor. Wegen des festen Maßstabs ergibt sich die Position im Arbeitsbereich-KS durch Division durch den Maßstab. Da die Bildzeilen nach unten, die y-Achse des Arbeitsbereichs aber nach oben zeigt, werden dabei die y-Koordinate gespiegelt und das Vorzeichen des Winkels umgekehrt.

Anschließend wird die Pose mit der ebenen Starrkörpertransformation aus Abschnitt 4.4 in das Roboter-KS überführt: Die Position wird gedreht und verschoben, zum Winkel wird der Drehwinkel der Transformation addiert. Als Höhe wird die Tischhöhe eingetragen. Das Ergebnis wird zusammen mit der Konfidenz und einem Debug-Bild als `ObjectPose` zurückgegeben.

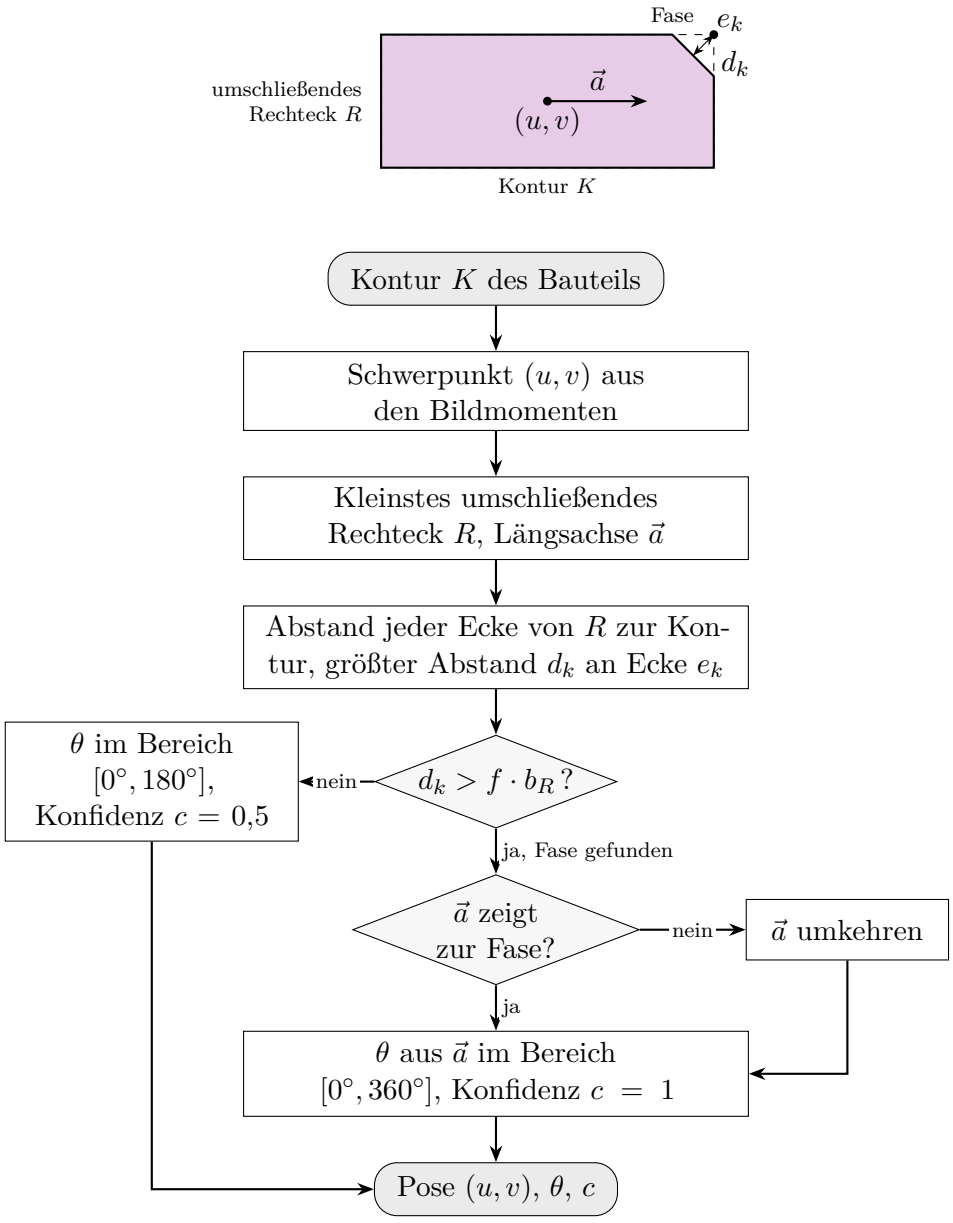


Abbildung 5.2: Posenbestimmung mit Fasenerkennung. Oben: Kontur, umschließendes Rechteck und Abstand d_k der Rechteckecke an der Fase. Unten: Ablauf; b_R ist die Breite von R, f ein fester Anteil.

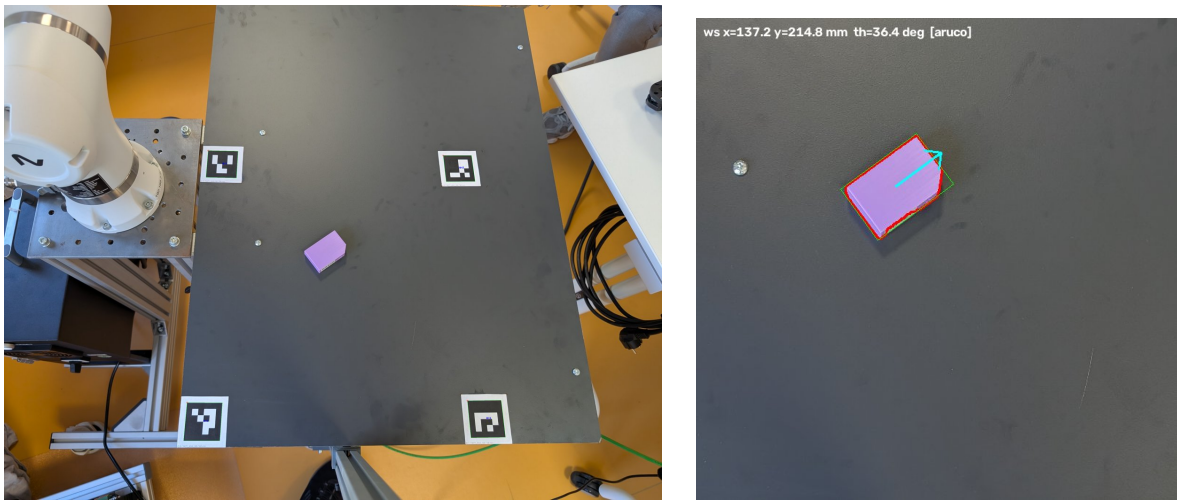


Abbildung 5.3: Zwischenergebnisse der Objekterkennung an einem Testfoto: Kamerabild mit den vier erkannten Markern (links) und entzerrte Draufsicht des Arbeitsbereichs mit Kontur (rot), umschließendem Rechteck (grün) und Orientierung (Pfeil zur Fase); oben die Pose im Arbeitsbereich-KS (rechts).

Die Transformation wird mit dem Roboter eingemessen (Abschnitt 4.4).

Parallaxe

Die Homographie bildet die Tischebene maßstabsgetreu ab. Die Oberseite des Bauteils liegt aber 15 mm über dem Tisch, und bei der schräg blickenden Kamera sind zusätzlich Seitenflächen sichtbar. Auf allen fünf Testfotos erscheint das Bauteil deshalb zu groß: 64,5 bis 69,1 mm × 43,3 bis 46,4 mm statt 60 × 40 mm, also 8 % bis 15 % mehr. Die Position ist dabei stabil; drei Fotos derselben Lage weichen um höchstens 0,4 mm voneinander ab. Sie kann aber systematisch zur Bildmitte hin oder von ihr weg verschoben sein, und zwar umso mehr, je weiter das Bauteil vom Punkt unter der Kamera entfernt liegt.

Parallaxe am Aufbau bewerten: Versatz im Zeigetest an Rand- und Mittelpositionen notieren. Falls nötig korrigieren, z. B. die Oberseite statt der Tischebene entzerren.

5.6 Zwischenfazit

Auf den Testfotos findet die Objekterkennung die Marker und das Bauteil zuverlässig und bestimmt die Orientierung über die Fase eindeutig. Die gemessene Größe des Bauteils zeigt den erwarteten Einfluss der Parallaxe. Wie stark er die Greifposition verschiebt, zeigt erst der Zeigetest am Aufbau mit der endgültigen Kamera.

Nach den Versuchen ergänzen: Ergebnisse mit der Daheng-Kamera, Versatz im Zeigetest, aufgetretene Probleme.

6 Robotersteuerung

Die Robotersteuerung setzt die Bewegungsaufträge des Orchestrators am Roboter um: Sie greift das Bauteil an der erkannten Pose, legt es an der Prüfposition ab, nimmt es dort nach der Prüfung wieder auf und legt es auf der Ablage für Gut- bzw. Schlechtteile ab. Nach außen bietet sie dafür die Schnittstelle `RobotController` aus Abschnitt 4.3.3 an. Kenntnis über Kameras oder Prüfergebnisse benötigt sie nicht.

6.1 Roboter und Programmierschnittstelle

Eingesetzt wird ein Neura Robotics LARA, ein kollaborativer Roboter (Cobot) mit sechs Achsen (Abschnitt 3.6). Die Steuerung läuft mit der Softwareversion 5.0.8. Für die Ansteuerung von einem externen Rechner stellt der Hersteller die Python-Bibliothek NeuraPy bereit [33]. Sie sendet Befehle als JSON-Nachrichten über eine TCP-Verbindung an die Steuerung und erhält deren Antworten auf demselben Weg.

Kartesische Posen werden in NeuraPy als $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ in Metern und Radiant angegeben. Innerhalb des Projekts werden dagegen durchgängig Millimeter und Grad verwendet (Abschnitt 4.3.3); die Umrechnung erfolgt ausschließlich in der Robotersteuerung.

Bevor der Roboter Bewegungen ausführt, wird er vorbereitet: Die Antriebe werden eingeschaltet, die Steuerung wird bei Bedarf vom Handbetrieb in den Automatikbetrieb umgeschaltet, das Werkzeug des Greifers wird ausgewählt, und Geschwindigkeitsbegrenzungen werden gesetzt. Abschließend wird auf der Steuerung ein Bewegungsprogramm gestartet, ohne das keine Bewegungsbefehle angenommen werden. Diese Vorbereitung läuft beim Verbinden, nach jedem Stopp und nach einer Fahrt des lernbasierten Greifens (Abschnitt 8.9) erneut ab, da dieses eigene Einstellungen an der Steuerung vornimmt.

Beim Verbinden vergleicht die Robotersteuerung außerdem den TCP des ausgewählten Werkzeugs mit dem in der Konfiguration erwarteten Wert. Weichen beide um mehr als 1 mm voneinander ab, verweigert sie jede Bewegung. Alle Posen beziehen sich auf den TCP; ein falscher Werkzeugeintrag würde jede Zielpose um die Differenz verschieben.

Für die Bewegungen werden zwei Bewegungsarten genutzt (Abschnitt 3.6):

- **PTP-Bewegung** für die großen Wege zwischen den Stationen. Aus der kartesischen Zielpose werden mit der inversen Kinematik der Steuerung die Gelenkwinkel berechnet, die dann in einer Gelenkbewegung angefahren werden.
- **LIN-Bewegung** für das Absenken auf das Bauteil und das Abheben. Der TCP bewegt sich dabei auf einer Geraden, sodass der Greifer senkrecht auf das Bauteil zufährt.

6.2 Greifpose

Aus der `ObjectPose` der Objekterkennung wird eine Zielpose für den Greifer abgeleitet. Die Position übernimmt x und y direkt; die Höhe ergibt sich aus der Tischhöhe plus einer festen Greifhöhe, die von Bauteil und Greifer abhängt. Der Greifer zeigt immer senkrecht nach unten. Dazu wird die Orientierung der Ruheposition übernommen und nur um die Hochachse des Roboters gedreht, und zwar um den erkannten Bauteilwinkel θ plus einen festen Greifer-Offset. Der Offset setzt sich aus zwei Teilen zusammen: 90° sorgen dafür, dass sich die Greiferbacken quer zur Längsachse schließen, also an den beiden langen Seiten des Bauteils anliegen. Weitere 23° gleichen aus, dass der Greifer gegenüber dem Flansch verdreht montiert ist. Der Offset beträgt damit 113°. Der resultierende Winkel wird auf den Bereich $-180°$ bis 180° normiert.

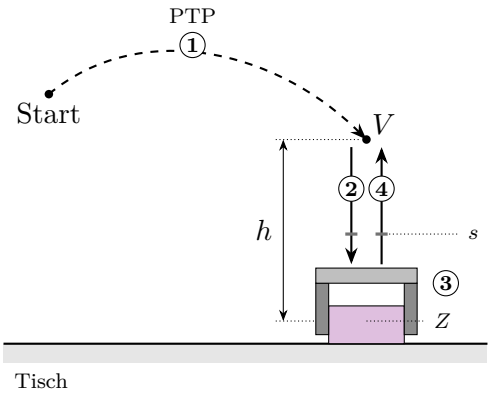
Die Drehrichtung wurde in der VM mit einem Achsentest geprüft: Eine positive Drehung um die Hochachse dreht das nach unten zeigende Werkzeug von oben gesehen gegen den Uhrzeigersinn, passend zur Formel.

Greifwinkel (Offset 113°) am realen Roboter mit dem Zeigetest an einem schräg liegenden Bauteil bestätigen.

6.3 Greif- und Ablegesequenz

Greifen und Ablegen laufen nach demselben Muster ab: Der Roboter fährt zunächst eine Vorposition oberhalb des Ziels an, senkt sich dann geradlinig ab, schließt bzw. öffnet den Greifer und hebt wieder auf die Vorposition an (Abb. 6.1). Die Vorposition liegt 80 mm über dem Ziel, sodass der Roboter bei der schnelleren Gelenkbewegung nicht mit dem Bauteil oder dem Tisch kollidiert.

Das Absenken erfolgt in zwei Stufen. Bis 20 mm über dem Ziel fährt der Roboter mit $0{,}10\,\mathrm{m\,s^{-1}}$, den Rest mit $0{,}03\,\mathrm{m\,s^{-1}}$. Beim Anheben ist es umgekehrt. So bleibt die Annäherung an Bauteil und Tisch langsam, während der größte Teil des Wegs zügig zurückgelegt wird. Nach dem Schließen und Öffnen des Greifers wird eine kurze Zeit gewartet, bis der Greifer seine Endlage erreicht hat.



	`pick`	`place`
vorab	Z aus erkannter Pose, V und Z prüfen, Greifer öffnen	Z = benannte Pose, V und Z prüfen
①	PTP nach V	
②	LIN nach Z, ab s langsam	
③	Greifer schließen, warten	Greifer öffnen, warten
④	LIN zurück nach V, bis s langsam	

Abbildung 6.1: Greif- und Ablegesequenz: Bewegung in der Seitenansicht (links) und Schritte von `pick` und `place` (rechts). Die Vorposition V liegt um $h = 80\,\mathrm{mm}$ über der Zielpose Z; ab $s = 20\,\mathrm{mm}$ über Z fährt der Roboter langsam. Geprüft wird gegen die Arbeitsraumgrenzen.

Das Greifen an der Prüfposition nutzt dieselbe Sequenz. Statt einer erkannten Pose wird dabei die fest hinterlegte Pose der Prüfposition übergeben, da das Bauteil dort in bekannter Lage liegt. Nach dem Ablegen an der Prüfposition fährt der Roboter nicht in die Ruheposition, sondern bleibt auf der Vorposition über der Prüfvorrichtung stehen. Dort ist der Greifer nicht im Bild der Seitenkamera, und nach der Prüfung kann der Roboter ohne Gelenkbewegung direkt wieder absenken.

Greifer-Rückmeldung auswerten: Nach dem Schließen prüfen, ob tatsächlich ein Bauteil gegriffen wurde.

6.4 Feste Posen

Neben der erkannten Bauteilpose fährt der Roboter vier feste Posen an (Tabelle 6.1). Die drei Ablagen wurden am 06.10.2026 am realen Aufbau angefahren (geteacht) und in die Konfiguration übernommen. Jede Pose ist die genaue Ablegepose einschließlich ihrer Höhe; an der Prüfposition liegt sie auf der Auflage der Prüfvorrichtung. Die Ruheposition ist als benannter Punkt auf der Steuerung gespeichert und wird direkt in Gelenkkoordinaten angefahren.

Tabelle 6.1: Feste Posen des Roboters (TCP im Basis-KS; Position in mm, Drehung um die Hochachse r_z in Grad)

Pose	x	y	z	r_z	Zweck
`home`	420,0	0,0	237,0	180	Ruheposition außerhalb der Kamerabilder; Start- und Endpunkt
`inspection`	520,8	199,6	42,0	−157	Ablage an der Prüfposition im Sichtfeld der Seitenkamera
`bin_good`	299,7	185,4	4,0	−157	Ablage für Gutteile
`bin_bad`	297,7	319,4	3,0	−157	Ablage für Schlechtteile

Geteacht wird über eine eigene Seite der Bedienoberfläche oder das Prüfprogramm (Abschnitt 6.7): Der Bediener fährt den Greifer im Tippbetrieb an die gewünschte Stelle, und die aktuelle TCP-Pose wird unter dem Namen der Pose übernommen.

Wiederholgenauigkeit an der Prüfposition beurteilen, da die Sichtprüfung eine reproduzierbare Lage voraussetzt.

6.5 Kalibrierung zwischen Kamera und Roboter

Die Robotersteuerung stellt für die Kalibrierung aus Abschnitt 4.4 die Werkzeuge bereit: Tippbetrieb, das Übernehmen der TCP-Position an jedem Marker und den Zeigetest, bei dem der Greifer über einem erkannten Bauteil stehen bleibt, ohne zu greifen. Die Transformation selbst rechnet ein gemeinsames Paket der Software aus, sodass Objekterkennung und Robotersteuerung dieselben Werte verwenden.

Am Labortag wurde die Transformation eingemessen. Sie besteht aus einer Drehung um 0,35°, einer Verschiebung um (205,1; −319,7) mm und einer Tischhöhe von 2,1 mm im Roboter-KS.

Restfehler der Kalibrierung (RMS) und Ergebnis des Zeigetests ergänzen.

6.6 Sicherheitskonzept

Die Anlage ist nicht sicherheitszertifiziert und wird nur unter Aufsicht betrieben. Die Robotersteuerung ergänzt die Schutzfunktionen des Cobots um mehrere Softwaremaßnahmen:

Reduzierte Geschwindigkeit Im Testbetrieb sind die globale Geschwindigkeit der Steuerung und die Gelenkgeschwindigkeit auf 20 % begrenzt. Geradlinige Bewegungen fahren höchstens $0{,}10\,\mathrm{m\,s^{-1}}$, die letzten 20 mm vor Bauteil und Tisch $0{,}03\,\mathrm{m\,s^{-1}}$.

Werkzeugprüfung Stimmt der TCP des Werkzeugs in der Steuerung nicht mit der Konfiguration überein, werden keine Bewegungen ausgeführt (Abschnitt 6.1).

Arbeitsraumgrenzen Vor jeder Bewegung wird geprüft, ob die Zielpose und die Vorposition innerhalb eines festgelegten Quaders im Roboter-KS liegen. Liegt eine Pose außerhalb, wird die Bewegung nicht ausgeführt, und die Robotersteuerung meldet einen Fehler. Dadurch führt auch eine fehlerhafte Pose der Objekterkennung nicht zu einer Bewegung außerhalb des vorgesehenen Bereichs.

Stopp mit Quittierung Ein Stopp beendet das Bewegungsprogramm auf der Steuerung. Danach werden alle Bewegungsbefehle abgelehnt, bis der Bediener den Stopp quittiert und der Roboter erneut vorbereitet wird (Abschnitt 6.1). Das Wiederanlaufen geschieht bewusst nicht automatisch.

Not-Halt Der Hardware-Not-Halt des Roboters ist jederzeit in Reichweite.

6.7 Inbetriebnahme

Für die schrittweise Inbetriebnahme am realen Roboter gibt es ein Prüfprogramm, das die Funktionen einzeln ausführt: Verbindung und Statusabfrage, Lesen der aktuellen Pose, Anfahren der Ruheposition, Öffnen und Schließen des Greifers, ein Achsentest, bei dem der Roboter nacheinander kleine Schritte entlang jeder Achse fährt, der Tippbetrieb, die Kalibrierung, der Zeigetest, das Teachen der festen Posen und schließlich ein vollständiger Greif- und Ablegevorgang an der Prüfposition. Dieselben Schritte stehen im Reiter der Robotersteuerung in der Bedienoberfläche zur Verfügung, gegliedert in die Seiten „Erste Schritte“, „Achsentest“, „Posen teachen“, „Kalibrierung“ und „Zeigetest“ (Kapitel 9). Am realen Roboter fragt jede Bewegung vorher nach; nur beim Tippen kann der Bediener diese Rückfrage für die laufende Sitzung abschalten, um viele kleine Schritte zügig fahren zu können.

Vor dem Einsatz am Roboter wurde der gesamte Programmpfad zunächst gegen die eigene Nachbildung der Steuerung und dann gegen die VM des Herstellers getestet (Abschnitt 4.5).

Die Steuerung ist im Labornetz unter der Adresse 192.168.2.13 erreichbar.

Betriebsartwechsel (Schlüsselschalter?) am realen Roboter verifizieren.

6.8 Zwischenfazit

Am Labortag am 06.10.2026 wurde die Robotersteuerung am realen Roboter in Betrieb genommen. Werkzeug, feste Posen und Kalibrierung wurden eingerichtet, und ein erster vollständiger Ablauf lief durch, noch ohne Greiferbewegung und mit Erkennung und Prüfung auf Testbildern. Dabei zeigten sich drei Punkte, die in die beschriebene Lösung eingeflossen sind: Der Greifer ist um 23° verdreht montiert (Offset 113°), eine durchgehend langsame Annäherung kostet zu viel Zeit (zweistufiges Absenken), und der Greifer ist in der Ruheposition nicht nötig, um das Bild der Seitenkamera freizugeben (Warten über der Prüfvorrichtung). Bei einem späteren Labortermin wurde auch das Öffnen und Schließen des Greifers aus dem Programm in Betrieb genommen.

Nach den Versuchen ergänzen: Wiederholgenauigkeit an der Prüfposition, aufgetretene Probleme.

7 Sichtprüfung

Die Sichtprüfung beurteilt das Bauteil an der Prüfposition anhand von drei Merkmalen auf seiner Seitenfläche: Sie liest die aufgeklebte Seriennummer, bestimmt den Durchmesser des Durchgangslochs und prüft, ob die Kerbe vorhanden ist. Aus diesen Merkmalen entscheidet sie, ob ein Gut- oder ein Schlechtteil vorliegt, und gibt das Ergebnis als `InspectionResult` an den Orchestrator zurück (Abschnitt 4.3.3).

Wie die Objekterkennung ist auch die Sichtprüfung in eine reine Bildverarbeitung und den Kamerazugriff getrennt, sodass sie sich mit gespeicherten Bildern testen lässt (Abschnitt 4.5). Abbildung 7.1 zeigt den Ablauf.

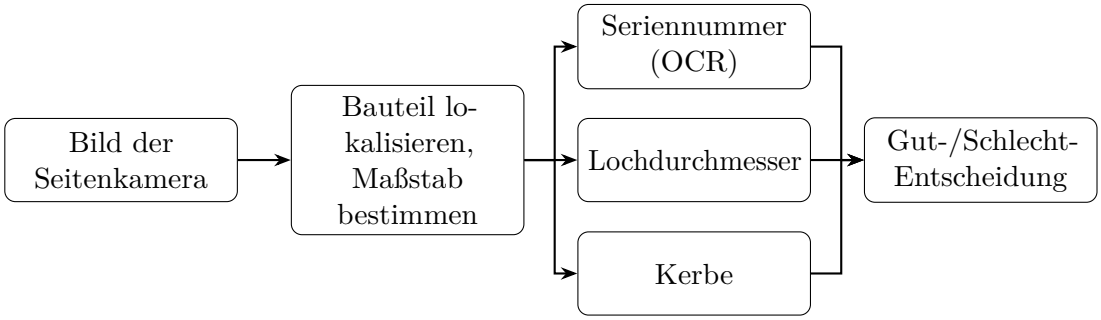


Abbildung 7.1: Ablauf der Sichtprüfung. Die drei Merkmale werden unabhängig voneinander bestimmt und gemeinsam bewertet.

7.1 Aufbau der Prüfstation

Die Seitenkamera, eine Universal Serial Bus (USB)-Kamera, ist fest neben der Prüfposition montiert; die bisherigen Testbilder haben 3840×2160 Pixel. Sie blickt möglichst senkrecht auf die Seitenfläche des Bauteils, da eine schräge Ansicht die Längen im Bild perspektivisch verzerren würde. Das Bauteil liegt auf einer 3D-gedruckten Prüfvorrichtung, deren Auflage 38 mm über der Platte liegt. Während der Prüfung wartet der Roboter über der Vorrichtung, sodass der Greifer nicht im Bild ist (Abschnitt 6.3).

Da die Objekterkennung die Orientierung über die Fase eindeutig bestimmt (Abschnitt 5.4) und der Roboter das Bauteil immer in derselben Lage relativ zum Greifer

aufnimmt, liegt es an der Prüfposition stets gleich orientiert, mit Etikett, Loch und Kerbe zur Kamera. Wird die Fase nicht gefunden, ist die Orientierung nur bis auf 180° bestimmt; das Bauteil kann dann verdreht liegen, und die Prüfmerkmale zeigen von der Kamera weg. Die Objekterkennung meldet diesen Fall mit einer verringerten Konfidenz.



Abbildung 7.2: Seitenansicht eines Testbilds mit dem Ergebnis der Sichtprüfung: Bauteil-Box (grün), Prüfbereiche für Etikett, Loch und Kerbe, angepasste Ellipse am Loch (rot) und erkannte Kerbe (blau). Oben links das Urteil mit Seriennummer, Lochdurchmesser und Kerbe.

Kameramodell, Abstand und Beleuchtung der Prüfstation ergänzen; Foto der Prüfvorrichtung mit Seitenkamera aufnehmen. Führt die Vorrichtung das Bauteil seitlich?

7.2 Lokalisierung des Bauteils und Maßstab

Zuerst wird das Bauteil im Bild gesucht. Da es sich farblich deutlich vom Hintergrund abhebt, geschieht das über eine Farbschwelle im HSV-Farbraum (Abschnitt 3.2): Pixel mit dem Farbton des Bauteils und ausreichender Sättigung und Helligkeit gehören zum Bauteil. Das weiße Etikett, das Loch und die Kerbe haben nicht die Farbe des Bauteils und erscheinen in der Maske zunächst als Lücken. Ein morphologisches Schließen mit einem Strukturelement in der Höhe des Bauteils füllt diese Lücken, sodass das umschließende Rechteck die gesamte Seitenfläche erfasst.

Die drei Prüfbereiche (Region of Interest (ROI)) für Etikett, Loch und Kerbe sind relativ zu diesem Rechteck festgelegt, jeweils als Bruchteil von dessen Breite und Höhe.

Sie passen sich damit automatisch an, wenn das Bauteil etwas näher an der Kamera oder etwas versetzt liegt.

Für die Umrechnung von Pixeln in Millimeter wird die sichtbare Höhe des Bauteils als Maßstab verwendet: Da die tatsächliche Bauteilhöhe bekannt ist, ergibt ihr Verhältnis zur Höhe des Rechtecks in Pixeln den Maßstab in Millimetern pro Pixel. Eine separate Kalibrierung mit einer Messplatte ist dadurch nicht nötig, und der Maßstab bleibt auch bei kleinen Abstandsänderungen richtig. Voraussetzung ist, dass Seitenfläche und Kamera annähernd parallel stehen.

7.3 Auslesen der Seriennummer

Die Seriennummer ist eine fünfstellige Zahl auf dem Etikett. Sie wird mit EasyOCR gelesen [20], einer lernbasierten OCR-Bibliothek, die ohne separate Installation einer OCR-Software auskommt (Abschnitt 3.8). Tesseract [17] wäre eine Alternative gewesen.

Der Etikettbereich wird ausgeschnitten und bei geringer Höhe vergrößert, da die Texterkennung auf sehr kleine Schrift schlechter anspricht. Die Erkennung ist auf Ziffern beschränkt. Aus den erkannten Textkandidaten wird derjenige mit der höchsten Konfidenz gewählt, der dem erwarteten Format entspricht; das Format ist als regulärer Ausdruck für genau fünf Ziffern in der Konfiguration hinterlegt. Liegt die Konfidenz unter einer Mindestschwelle, gilt die Seriennummer als nicht lesbar.

7.4 Vermessung des Lochdurchmessers

Im Lochbereich ist die Seitenfläche des Bauteils hell und farbig. Das Loch hebt sich davon ab, allerdings je nach Beleuchtung unterschiedlich: Die Bohrungswand erscheint dunkel, durch das Loch hindurch kann aber auch heller Hintergrund sichtbar sein. Deshalb wird nicht nach einer bestimmten Farbe des Lochs gesucht, sondern nach allem, was nicht zur hellen Seitenfläche gehört. Als Referenz für die Helligkeit der Fläche dient ein hohes Perzentil der bauteilfarbenen Pixel im Bereich.

Die so gefundenen Regionen werden morphologisch bereinigt und auf eine Mindestgröße gefiltert. Für jede Region wird ihre Rundheit bestimmt, also das Verhältnis ihrer Fläche zur Fläche des kleinsten umschließenden Kreises. Die rundeste Region ist das Loch, sofern ihre Rundheit eine Mindestschwelle überschreitet. An ihre Kontur wird eine Ellipse nach der Methode der kleinsten Quadrate angepasst (Abschnitt 3.7); der Mittelwert

der beiden Achsenlängen ergibt den Durchmesser in Pixeln, der mit dem Maßstab aus Abschnitt 7.2 in Millimeter umgerechnet wird. Die Ellipse berücksichtigt, dass ein kreisrundes Loch bei leicht schräger Ansicht elliptisch erscheint. Eine Kreisdetektion mit der Hough-Transformation (Abschnitt 3.7) wäre möglich, liefert den Radius aber nur in der Auflösung ihres Akkumulators, während der Ellipsen-Fit alle Konturpunkte nutzt.

7.5 Prüfung auf Kerbe

Die Kerbe ist eine kleine rechteckige Öffnung am Rand der Seitenfläche. Je nach Licht ist sie dunkler als die Fläche, anders gefärbt oder nur an ihren Kanten zu erkennen. Deshalb werden Kandidaten auf zwei Wegen gesucht:

- **Regionenbasiert:** Pixel, die deutlich dunkler oder deutlich anders gesättigt sind als die umgebende Fläche. Die Referenz für die Fläche ist der Median am Rand des Kerbenbereichs.
- **Kantenbasiert:** geschlossene Kantenzüge aus einer Kantendetektion, für den Fall, dass die Öffnung dieselbe Farbe wie die Fläche hat und sich nur durch ihre Ränder abzeichnet.

Aus allen Kandidaten werden nur solche betrachtet, die vollständig im Kerbenbereich liegen, eine plausible Größe haben und annähernd quadratisch bis mäßig länglich sind. Für jeden wird die Rechteckigkeit bestimmt, also das Verhältnis seiner Fläche zur Fläche des kleinsten umschließenden Rechtecks. Überschreitet der rechteckigste Kandidat eine Mindestschwelle, gilt die Kerbe als vorhanden. Ein Template-Matching gegen ein Referenzbild der Kerbe (Abschnitt 3.7) wäre eine Alternative, reagiert aber im Allgemeinen empfindlicher auf Änderungen von Beleuchtung und Lage.

7.6 Gut-/Schlecht-Entscheidung

Ein Bauteil ist ein Gutteil, wenn alle drei Merkmale erfüllt sind: Die Seriennummer ist lesbar und entspricht dem Format, das Loch wurde gefunden und sein Durchmesser liegt innerhalb der Toleranz um den Sollwert, und die Kerbe ist vorhanden. Andernfalls ist es ein Schlechtteil, und jeder nicht erfüllte Punkt wird als Grund im Prüfergebnis festgehalten (Abb. 7.3). Die Toleranz des Lochdurchmessers dient der Demonstration und entspricht keiner Fertigungsvorgabe.

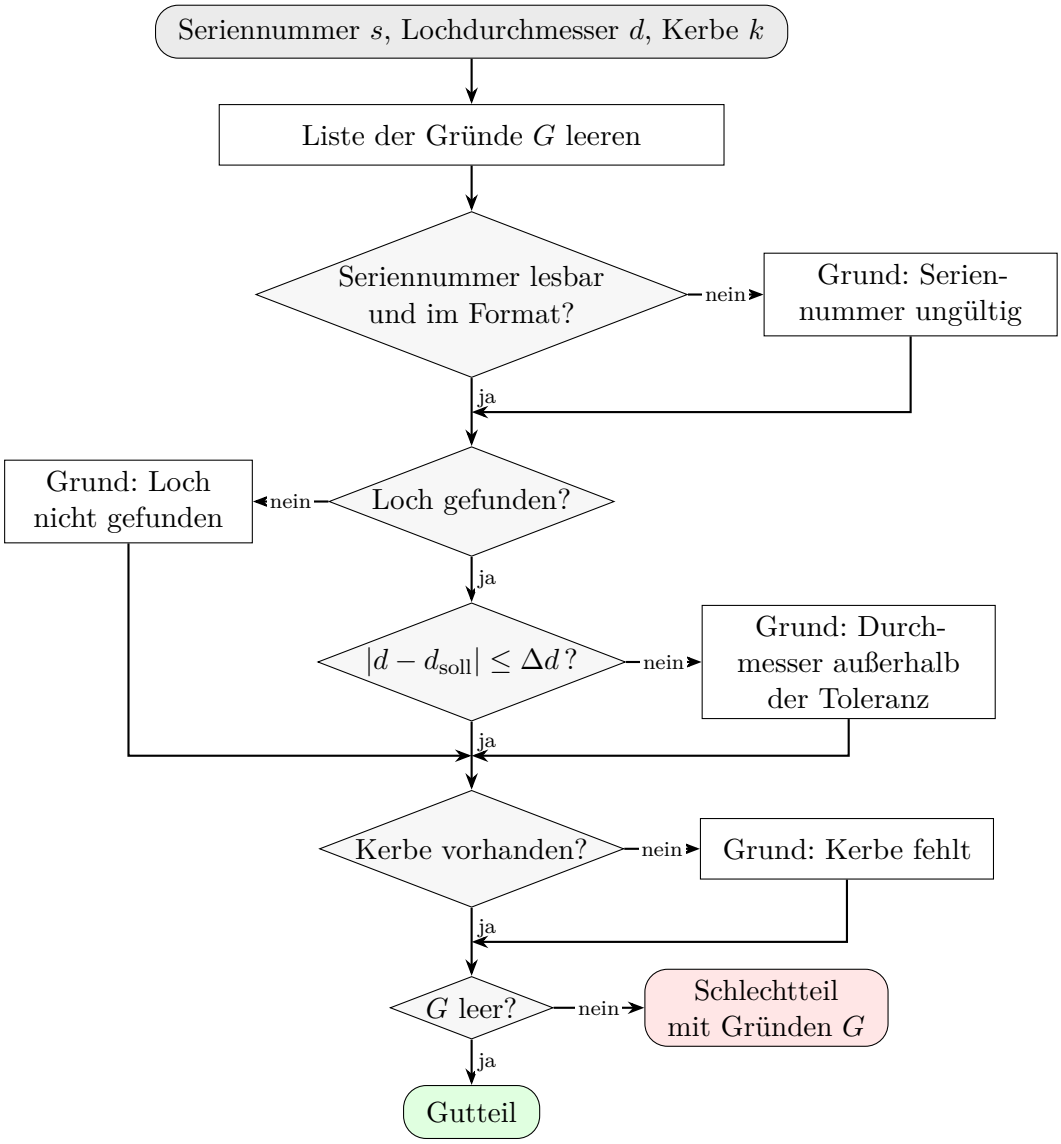


Abbildung 7.3: Gut-/Schlecht-Entscheidung. Jedes nicht erfüllte Merkmal wird als Grund in G festgehalten.

Für das Testbauteil gelten eine Bauteilhöhe von 15 mm als Maßstab, ein Soll-Durchmesser des Lochs von 10 mm und eine Toleranz von ±0,4 mm. Die Toleranz ist ein Demonstrationswert und keine Fertigungstoleranz.

7.7 Zwischenfazit

Auf vier Testbildern eines Gutteils wird die Seriennummer „69420“ jedes Mal richtig gelesen. Das Verhältnis von Lochdurchmesser zu Bauteilhöhe streut auf den ersten drei Bildern um ±1,2 % (0,647; 0,663; 0,649), was etwa 9,7 mm bis 9,9 mm entspricht. Künstlich erzeugte Fehler, bei denen Kerbe oder Loch im Bild übermalt sind, werden als Schlechtteil erkannt.

Nach den Versuchen ergänzen: Ergebnisse mit der Seitenkamera am Aufbau, echte Fehlerteile, aufgetretene Probleme.

8 KI-basiertes Greifen

Modul 4 greift das Bauteil nicht über eine programmierte Bahn, sondern mit einer gelernten Policy (Abschnitt 3.9). Eine Diffusion Policy erhält in jedem Takt die Bilder einer Kamera am Greifer und der Kamera über dem Arbeitsbereich sowie den Zustand des Roboters und gibt die nächsten Gelenk-Sollwerte und den Greiferbefehl aus. Gelernt wird aus Demonstrationen, die der Roboter selbst fährt (BC, Abschnitt 3.9.2). Das Modul ist eine Alternative zum klassischen Greifen von Modul 2 und für den automatischen Ablauf nicht erforderlich.

8.1 Aufgabe und Abgrenzung

Im Ablauf übernimmt Modul 4 den Zustand PICK (Tabelle 4.4). Die Policy startet in einer festen Ausgangsstellung, sucht das Bauteil in den Kamerabildern, greift es und fährt es an einen Übergabepunkt etwa 9 cm über der Prüfvorrichtung. Dort endet ihre Fahrt mit dem Bauteil im Greifer. Das Ablegen auf der Prüfvorrichtung (PLACE_INSPECT) und alle weiteren Bewegungen führt wieder Modul 2 aus. Die Pose aus der Objekterkennung wird an Modul 4 übergeben, aber nur protokolliert; die Policy bestimmt die Lage des Bauteils selbst aus den Bildern.

Die gelernte Fahrt besteht aus sechs Abschnitten, die beim Aufzeichnen aus geteachten Punkten geplant werden:

1. eine Ausgangsstellung, in der der Arm das Bild der Draufsichtkamera freigibt,
2. eine Anflugposition 10 cm über der Vorgreifposition (PTP),
3. die Vorgreifposition 5 cm über dem Greifpunkt (LIN),
4. den Greifpunkt, an dem der Greifer schließt (LIN),
5. die Rückkehr zur Vorgreifposition (LIN) und
6. den Übergabepunkt über der Prüfvorrichtung (PTP).

Zwischen den Abschnitten wird überschliffen, damit der Arm an den Durchfahrpunkten nicht anhält.

Der Greifer lässt sich über die Programmierschnittstelle nur öffnen und schließen; eine Rückmeldung der Backenstellung gibt es nicht. Der Greiferzustand geht deshalb als binärer Befehl in Zustand und Aktion ein. Nach jedem Greiferbefehl wartet die Bahn eine feste Totzeit von 0,5 s, damit der Arm erst anhebt, wenn die Backen geschlossen sind.

Jeremy: Totzeit des Greifers (0,5 s) an der Anlage messen (Video) und hier bestätigen.

8.2 Hardware und Software

Tabelle 8.1 fasst die von Modul 4 genutzte Hardware zusammen. Roboter und Greifer sind dieselben wie in Modul 2 (Tabelle 4.3).

Tabelle 8.1: Hardware und Software von Modul 4

Komponente	Ausführung
Handgelenkkamera	Daheng VEN-161-61U3C, Sony IMX296, 1440×1080 Pixel, Global Shutter, bis 61 fps, USB3 Vision
Objektiv	Fischauge, Brennweite 1,85 mm
Szenenkamera	Draufsichtkamera von Modul 1 (Kapitel 5)
Trainingsrechner	NVIDIA GeForce RTX 5070 Ti, 16 GB Grafikspeicher
Software	Python 3.12, PyTorch 2.11, LeRobot 0.6.1 [32], NeuraPy 5.0.8

Die Handgelenkkamera sitzt am Greifer und bewegt sich mit dem Arm. Ihre Lage zum Greifer ist fest, sodass ein Verrutschen der Szene sie nicht betrifft. Ein Global-Shutter-Sensor belichtet alle Pixel gleichzeitig; Bilder aus der Bewegung werden dadurch nicht verzerrt. Für das Objektiv wurde ein Fischauge mit kurzer Brennweite gewählt, weil es den Arbeitsbereich vom Anflug bis zum Greifpunkt scharf abbildet. Eine Unschärfe würde Information unwiederbringlich verlieren, die starke, aber feste Verzeichnung des Fischauges lernt das Netz dagegen mit. Die Kamera wird nicht über die übliche Videoschnittstelle angesprochen, sondern über das Software-Paket des Herstellers (Galaxy SDK). Beide Kameras müssen direkt an den USB-3-Anschlüssen des Rechners hängen; über einen Verteiler erreichten sie in einem Test nur 23 bis 30 fps.

Das Training läuft vollständig lokal. Die Daten werden nicht in einen Cloud-Dienst hochgeladen, und ein Datensatz kann zum Training auf einen anderen Rechner kopiert werden.

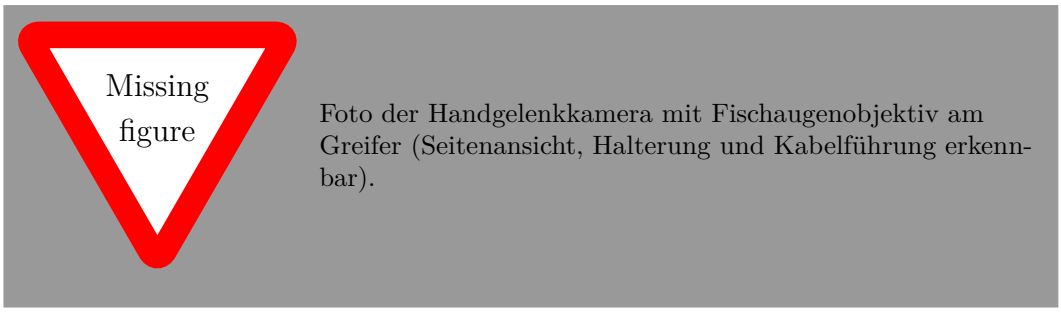


Foto der Handgelenkkamera mit Fischaugenobjektiv am Greifer (Seitenansicht, Halterung und Kabelführung erkennbar).

Abbildung 8.1: Handgelenkkamera am Greifer

8.3 Beobachtung und Aktion

Aufzeichnung und Ausführung laufen mit einem festen Takt von 15 Hz. In jedem Takt wird eine Beobachtung gebildet und eine Aktion ausgeführt (Tabelle 8.2).

Tabelle 8.2: Beobachtung und Aktion der Policy

Größe	Werte	Inhalt
Zustand	14	sechs Gelenkwinkel in rad, TCP-Position in m, TCP-Orientierung als Quaternion, Greiferbefehl (0 = offen, 1 = geschlossen)
Bilder	2	Handgelenk- und Szenenkamera, je 240 × 320 Pixel, RGB
Aktion	7	sechs Gelenkwinkel für den nächsten Takt (absolut), Greiferbefehl (Schwelle 0,5)

Die Aktion ist eine absolute Gelenkstellung, nicht eine Änderung. Weil der Takt fest ist, steckt die Geschwindigkeit des Roboters im Abstand zweier aufeinanderfolgender Aktionen. Die Policy lernt also auch das Tempo der Aufzeichnung und fährt bei der Ausführung genauso schnell. Deshalb müssen Takt, Bildgröße, Kameras, Geschwindigkeitsvorgaben und der Override der Steuerung bei Aufzeichnung und Ausführung übereinstimmen. Weicht einer dieser Werte ab, bricht die Ausführung ab, bevor sich der Roboter bewegt.

Zwischen den 15 Hz-Takten interpoliert das Modul je drei Zwischenwerte und sendet die Gelenk-Sollwerte mit 60 Hz an die Steuerung (Befehl `servo_j`).

8.4 Datenaufnahme mit eingespeistem Rauschen

Demonstrationen werden nicht von Hand vorgeführt. Ein Bediener teacht nur die Punkte der Bahn, mit einem Gamepad oder durch Handführung. Die eigentliche Aufzeichnung fährt der Roboter selbst: Ein Bahnplaner verbindet die Punkte mit PTP- und LIN-Abschnitten konstanter Geschwindigkeit und prägt der Bahn dabei gezielt Rauschen auf

(Abb. 8.2). Dadurch ist das Tempo in allen Demonstrationen gleich, und es entstehen keine Pausen durch Zögern des Bedieners, die die Policy als „stehen bleiben“ lernen würde.

8.4.1 Rauschen und asymmetrische Datenpaare

Würde der Roboter immer genau die ideale Bahn fahren, enthielte der Datensatz kein einziges Beispiel dafür, wie man von einer Abweichung zurückfindet (Abschnitt 3.9.2). Das Modul folgt deshalb dem Gedanken von DART [25]: Während der Fahrt wird die Bahn kartesisch ausgelenkt, und zwar mit einem Ornstein-Uhlenbeck-Prozess (Gleichung (3.15)) mit einer Zeitkonstante von 0,8 s. Die Auslenkung ändert sich dadurch langsam und sieht im Bild wie ein Schlingern aus, nicht wie ein Zittern. Die größte Amplitude liegt bei 15 mm in der Position und 0,05 rad in der Drehung. Sie wurde bewusst so groß gewählt, dass die Verschiebung im Kamerabild deutlich sichtbar ist.

Gespeichert werden die Daten asymmetrisch. Als Beobachtung dienen der tatsächliche, ausgelenkte Zustand und die dabei aufgenommenen Bilder, als Aktion dagegen der nächste Punkt der *idealen* Bahn. Jedes Datenpaar zeigt der Policy also, wohin sie aus einer abweichenden Lage steuern muss. Das Rauschen selbst taucht in den Aktionen nie auf und wird deshalb nicht mitgelernt.

Kurz vor dem Greifpunkt muss die Bahn genau sein. Das Rauschen wird deshalb ab einem Abstand von 15 cm zum Greifpunkt gedämpft und ist ab 2 cm ganz abgeklungen (Abb. 8.3). Nach unten, zum Tisch hin, ist die Auslenkung zusätzlich begrenzt. Die Rauschstärke wird für jede Episode zufällig zwischen null und der vollen Amplitude gezogen. So enthält der Datensatz sowohl ruhige Fahrten nahe der Idealbahn als auch Fahrten mit großen Korrekturen.

8.4.2 Umrechnung und Kollisionsprüfung

Das Rauschen ist kartesisch festgelegt, weil sich nur so eine Grenze in eine feste Richtung, etwa zum Tisch hin, sicher einhalten lässt. Die verrauschte Bahn wird vor der Fahrt mit der inversen Kinematik der Robotersteuerung in Gelenkwinkel umgerechnet. Als Startwert dient jeweils die Lösung des vorherigen Schritts, sodass die Lösung nicht zwischen verschiedenen Armstellungen springt. Ein Vortest an drei realen Posen ergab, dass alle Punkte im Korridor von ± 20 mm erreichbar sind und die Bahn weit von singulären Stellungen entfernt liegt.

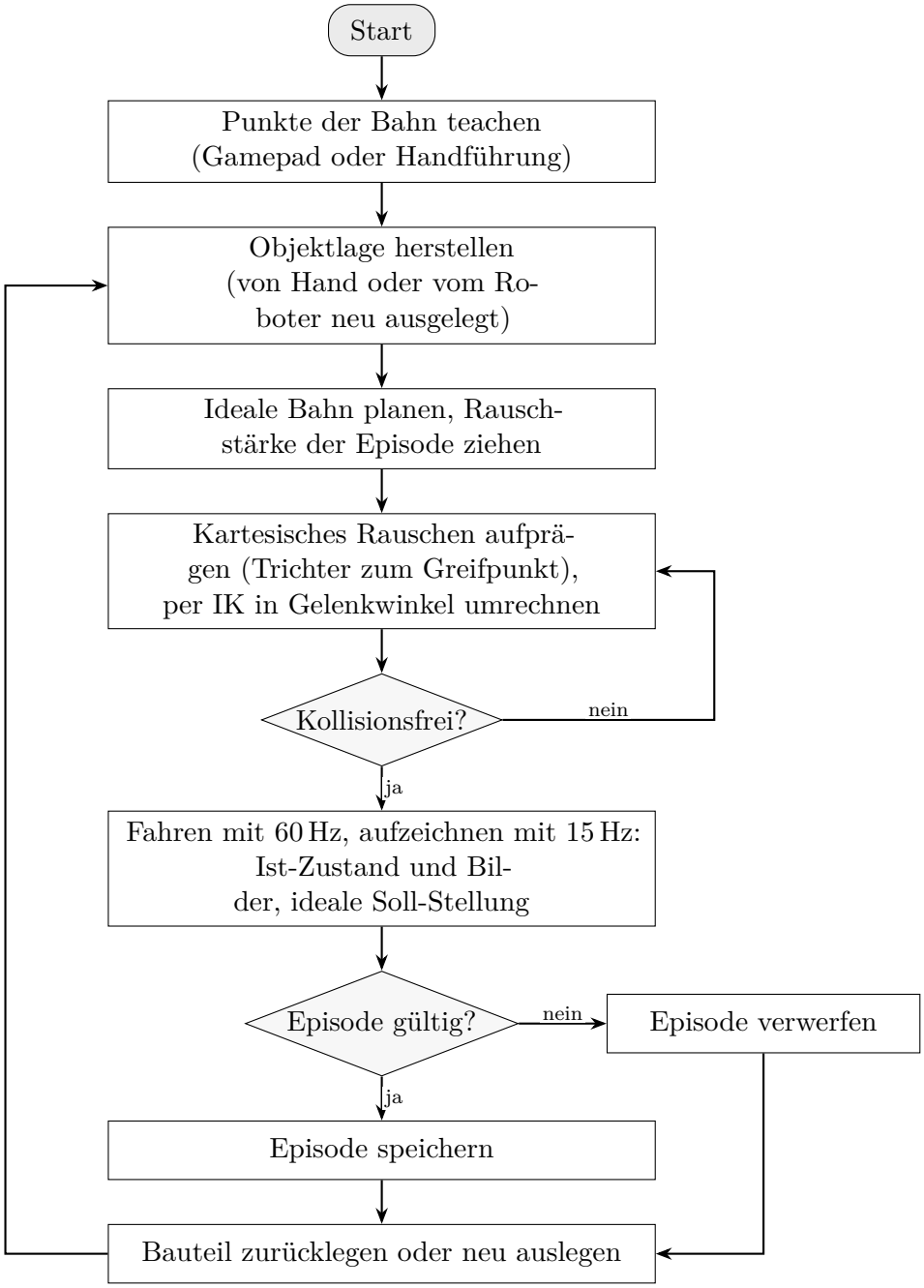


Abbildung 8.2: Ablauf der automatischen Datenaufnahme. Eine Episode wird verworfen, wenn ein Schutzstopp auslöst, der Arm der Soll-Bahn zu weit hinterherläuft oder Kamerabild und Roboterzustand zeitlich nicht zusammenpassen.

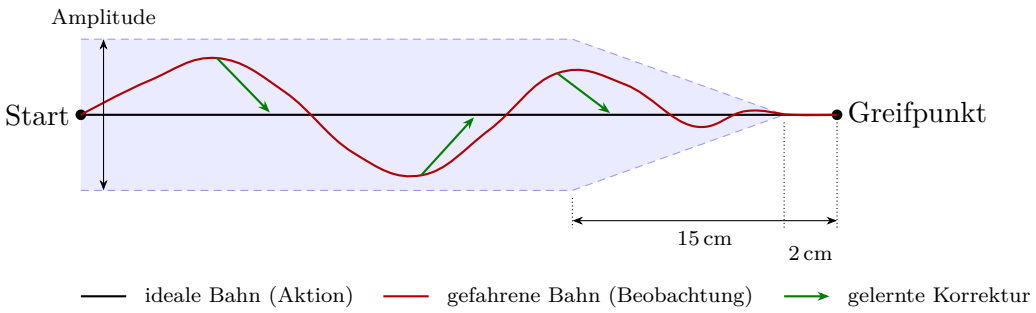


Abbildung 8.3: Rauschkorridor um die ideale Bahn (schematisch). Die Auslenkung ist bis 15 cm vor dem Greifpunkt voll wirksam, wird danach gedämpft und ist 2 cm vor dem Greifpunkt null. Aus der Paarung von ausgelenktem Zustand und idealer Soll-Stellung lernt die Policy, zur Bahn zurückzusteuern.

Anschließend wird die gesamte Bahn gegen ein einfaches Kollisionsmodell geprüft. Tisch und feste Hindernisse sind als Quader beschrieben, und für mehrere Punkte entlang des Arms wird der Abstand zu diesen Quadern berechnet. Unterschreitet ein Punkt einen Sicherheitsabstand von 3 cm, wird das Rauschen neu gezogen.

8.4.3 Objektlagen

Damit die Policy das Bauteil an beliebigen Stellen findet, muss der Datensatz viele verschiedene Objektlagen enthalten. Für jede Lage werden die Greifpunkte nachgeteacht. Um diesen Aufwand zu verringern, legt der Roboter das Bauteil nach jeder Episode selbst wieder ab: entweder an derselben Stelle, um die Bahn mit neuem Rauschen erneut aufzuzeichnen, oder an einer neuen, zufällig gezogenen Lage innerhalb des Arbeitsbereichs mit einer Drehung bis $\pm 45°$. Die neue Lage ist dann genau bekannt und wird als Zahl in den Metadaten der Episode gespeichert. Diese Fahrten werden nicht aufgezeichnet.

Als Richtgröße für den Umfang werden einige Dutzend bis wenige Hundert Demonstrationen angesetzt. Die tatsächlich benötigte Anzahl ergibt sich erst aus den Trainingsergebnissen.

8.5 Modell und Training

Eingesetzt wird die faltungsbasierte Diffusion Policy [29] aus LeRobot (Abschnitt 3.9.4). Jede Kamera hat einen eigenen Bildkodierer, ein ResNet18 mit vortrainierten ImageNet-Gewichten [30]. Seine Merkmale werden mit dem Zustand der letzten beiden Takte

zusammengefasst und an ein eindimensionales U-Net übergeben, das eine Folge von 16 Aktionen entrauscht. Tabelle 8.3 listet die wichtigsten Einstellungen.

Tabelle 8.3: Einstellungen der Diffusion Policy

Parameter	Wert
Bildkodierer	ResNet18 je Kamera, ImageNet-Gewichte
U-Net-Kanäle	256, 512, 1024
Parameter gesamt	89,5 Millionen
Beobachtungen	2 Takte
Vorhersagehorizont	16 Aktionen, davon 8 nutzbar
Entrauschen	100 Schritte im Training, 10 Schritte (DDIM) bei der Ausführung
Vorhersageziel	Aktionsfolge direkt (nicht das Rauschen)
Bildaugmentierung	zufälliger Ausschnitt 90 %
Optimierer	AdamW, Lernrate 10^{-4}, Cosinus-Verlauf, gleitender Mittelwert der Gewichte (0,999)
Batchgröße, Schritte	64, 60 000 (Vorgabe)

Die Größe des U-Nets ist kleiner als die Vorgabe von LeRobot. So passt das Training in die 16 GB der Grafikkarte, und die Vorhersage bleibt auf kleineren Rechnern ausführbar. Auf der RTX 5070 Ti dauert ein Trainingsschritt mit Batchgröße 64 etwa 0,21 s; die vorgegebenen 60 000 Schritte dauern damit rund 3,5 h.

Für das Vorhersageziel wurde die Aktionsfolge selbst gewählt statt des enthaltenen Rauschens. Bei nur zehn Entrauschungsschritten erzeugte die Rauschvorhersage in einem Vergleich gezackte Aktionsfolgen mit Sprüngen von 0,10 bis 0,12 rad, während die Sprünge in den Trainingsdaten bei 0,015 rad lagen. Mit der direkten Vorhersage der Folge entsprechen die Sprünge denen der Trainingsdaten.

8.6 Ausführung

Vor jeder Fahrt wird der Greifer in seinen Ausgangszustand gebracht, und der Arm fährt die Ausgangsstellung der Aufzeichnung an. Danach läuft der Takt aus Abb. 8.4.

Eine Vorhersage mit zehn Entrauschungsschritten dauert auf der RTX 5070 Ti etwa 77 ms und damit länger als ein Takt von 67 ms. Sie läuft deshalb in einem eigenen Thread. Der Sollwertstrom an den Roboter läuft ungestört weiter, und jede Vorhersage wird dem Takt zugeordnet, aus dem ihre Beobachtung stammt. Teile einer Vorhersage, die bei ihrem Eintreffen schon vorbei sind, verfallen. Reicht keine Vorhersage mehr bis zum aktuellen Takt, wird die Fahrt beendet.

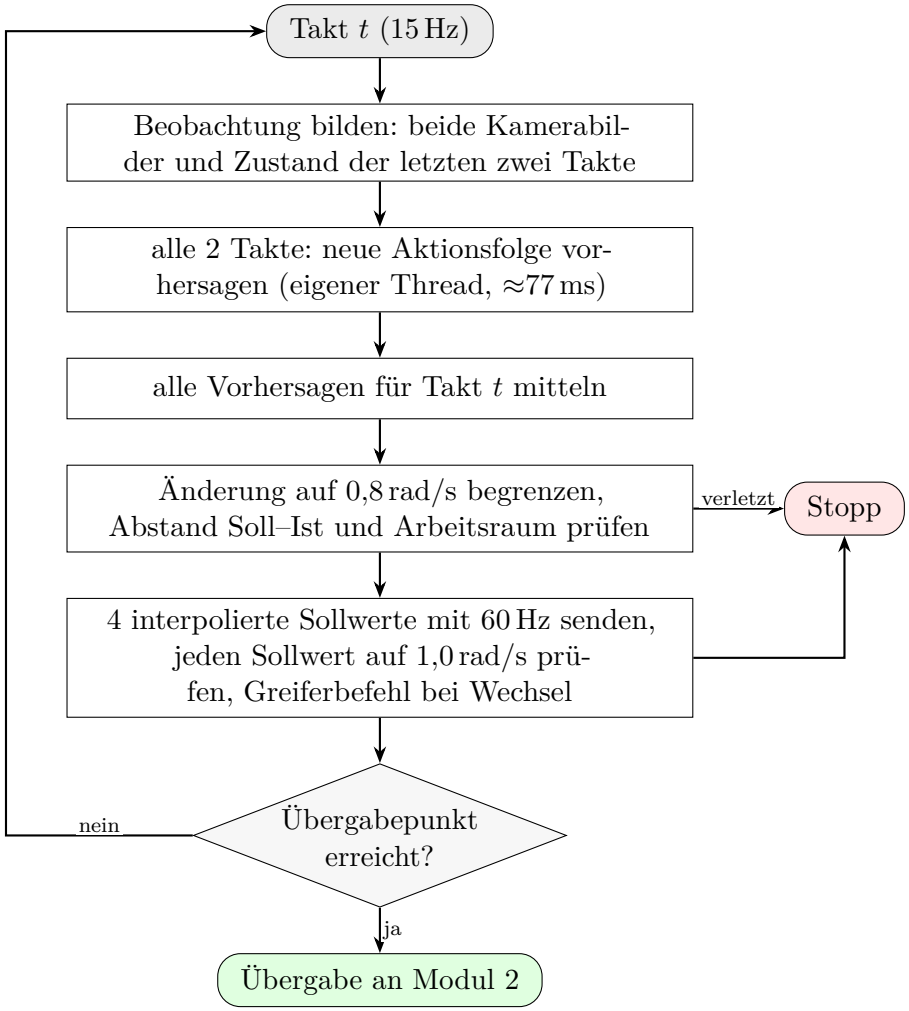


Abbildung 8.4: Ablauf eines Takts bei der Ausführung der Policy

Weil alle zwei Takte neu vorhergesagt wird und jede Vorhersage acht nutzbare Aktionen hat, liegen für einen Takt bis zu vier Vorhersagen vor. Ausgeführt wird ihr Mittelwert, wie von Zhao et al. vorgeschlagen [31]. Das glättet den Übergang, wenn eine neue Vorhersage eintrifft.

Die Fahrt gilt als beendet, wenn die Gelenke fünf Takte lang höchstens 0,02 rad von der Endstellung der Aufzeichnung entfernt sind und der Greifer geschlossen ist. Die Stellung allein reicht nicht, weil der Arm auf dem Hin- und Rückweg dieselben Stellungen durchfahren kann und sich dann nur der Greiferzustand unterscheidet.

8.7 Sicherheitsmaßnahmen

Eine gelernte Policy kann unerwartete Sollwerte ausgeben. Zusätzlich zu den Schutzfunktionen des Cobots prüft Modul 4 deshalb jeden Befehl, bevor er gesendet wird:

Weiche Begrenzung Die Änderung der Sollstellung je Takt wird auf eine Gelenkgeschwindigkeit von 0,8 rad/s begrenzt, die Richtung bleibt erhalten.

Harte Grenze Jeder einzelne 60 Hz-Sollwert wird verworfen, wenn er nicht endlich ist, außerhalb der Achsgrenzen liegt oder eine Gelenkgeschwindigkeit über 1,0 rad/s erfordert. Liegt die Sollstellung mehr als 0,35 rad von der gemessenen Stellung entfernt, wird ebenfalls nichts gesendet. In beiden Fällen stoppt der Roboter.

Arbeitsraumüberwachung Ein eigener Thread prüft laufend die Lage von Werkzeug und Arm gegen dasselbe Kollisionsmodell wie bei der Aufnahme und stoppt bei einer Verletzung.

Schleppfehler Bei der Aufnahme wird abgebrochen, wenn der Arm vier Takte lang mehr als 0,30 rad hinter der Sollstellung liegt.

Freigabe Fahrten am realen Roboter erfordern eine ausdrückliche Bestätigung.

Der Software-Stopp hält den Arm nach Messungen an der Anlage innerhalb von 96 ms an. Er ersetzt keinen Not-Halt; der Hardware-Not-Halt bleibt bei allen Fahrten in Reichweite.

8.8 Ausführungsart an der Anlage

In der virtuellen Maschine (VM) der Steuerung lief die Ausführung über `servo_j` mit 60 Hz ruhig. Am realen Roboter zitterte der Arm dagegen deutlich. Die Steuerung fährt jeden Sollwert an und wartet dann auf den nächsten, sodass bei 60 Hz eine treppenförmige Bewegung entsteht. Sie regt Arm und Aufbau bei 7 Hz bis 8 Hz an (Abb. 8.5). Gemessen wurde die Schwingung mit dem Beschleunigungssensor eines Smartphones am Unterarm des Roboters.

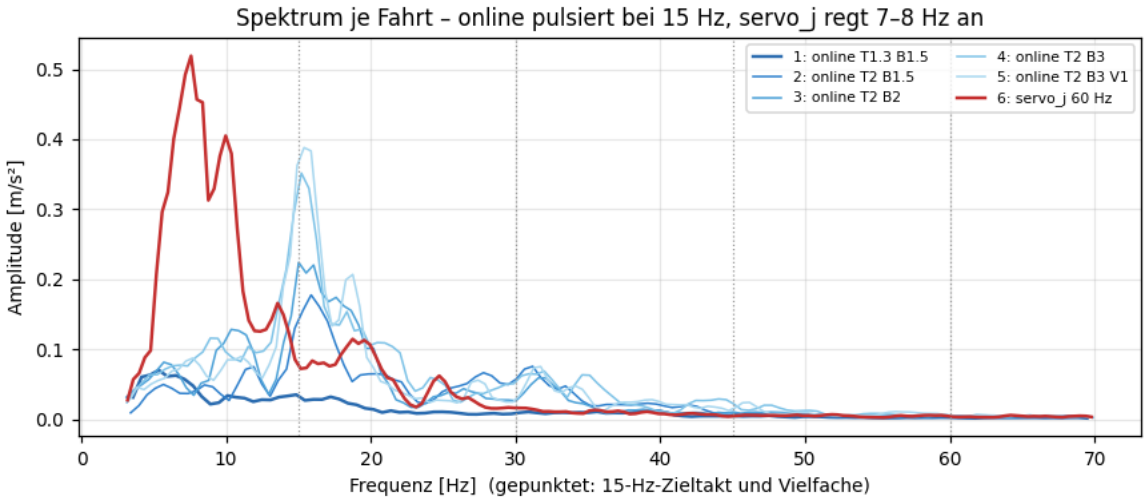


Abbildung 8.5: Spektrum der Beschleunigung am Unterarm je Fahrt (Messung 13.10.2026). Rot: `servo_j` mit 60 Hz; blau: Varianten mit Interpolation in der Steuerung (`movelinear_online`). Quelle: Unterlagen Modul 4.

Untersucht wurden zwei Abhilfen. Lässt man die Steuerung selbst zwischen Zielpunkten interpolieren (`movelinear_online`), sinkt das Zittern deutlich, der Arm weicht aber in Kurven und Drehungen um bis zu 49 mm von der geplanten Bahn ab und läuft bis zu 0,79 rad hinterher. Das widerspricht den Datenpaaren aus Abschnitt 8.4, bei denen Zustand und Sollstellung zueinander passen müssen. Wird dagegen `servo_j` beibehalten und der Override der Steuerung auf 0,5 gesetzt, führt die Steuerung die Sollwerte weicher nach. An einem Labortag lag die Schwingung damit bei 0,19 $\mathrm{m\,s^{-2}}$ bis 0,20 $\mathrm{m\,s^{-2}}$ statt 1,74 $\mathrm{m\,s^{-2}}$, also auf dem Niveau eines Programms am Bedienhandgerät des Herstellers (0,16 $\mathrm{m\,s^{-2}}$ bis 0,18 $\mathrm{m\,s^{-2}}$, Abb. 8.6). Die Bahn wich um höchstens 1,2 mm ab, und die Fahrzeit blieb mit 11,6 s gleich. Für Aufnahme und Ausführung wurde deshalb `servo_j` mit Override 0,5 festgelegt.

Jeremy: Override 0,5 mit dem Betreuer abstimmen (Sicherheitsthema) und hier vermerken.

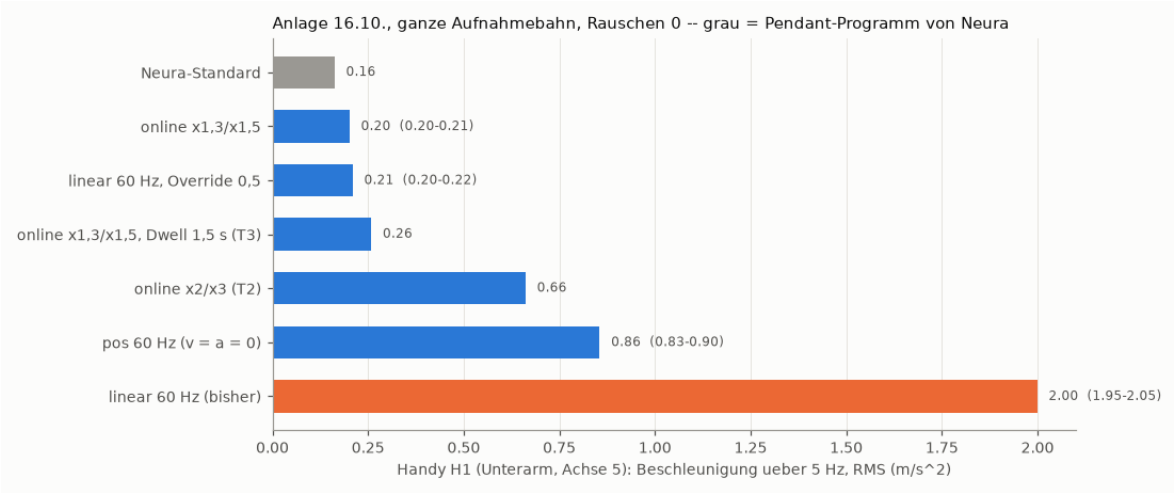


Abbildung 8.6: Schwingung am Unterarm über die ganze Aufnahmebahn je Ausführungsart (Effektivwert der Beschleunigung oberhalb 5 Hz, Labortag 16.10.2026, ohne Rauschen). Grau: Programm am Bedienhandgerät als Vergleich. Quelle: Unterlagen Modul 4.

8.9 Einbindung in das Gesamtsystem

Modul 4 ist mit der Schnittstelle `Grasper` in das System eingebunden (Tabelle 4.6). Die Variante `BcGrasper` führt beim Greifen genau eine Fahrt der Policy aus. Weil LeRobot 0.6.1 Python 3.12 voraussetzt und seine Abhängigkeiten mit denen der Texterkennung von Modul 3 kollidieren, läuft die Policy in einer eigenen Python-Umgebung als Unterprozess. Die Bedienoberfläche und die übrigen Module bleiben in ihrer Umgebung.

Ein Greifvorgang mit Modul 4 läuft so ab:

1. Die Objekterkennung gibt die Draufsichtkamera frei, damit Modul 4 sie als Szenenkamera öffnen kann. Die Robotersteuerung von Modul 2 trennt ihre Verbindung zur Steuerung.

2. Der Unterprozess startet die Policy und fährt den Roboter über eine eigene Verbindung. Bei Fahrten am realen Roboter wird die Freigabe aus dem Sicherheitsdialog des Ablaufs übernommen (Abschnitt 9.4).

3. Der Fortschritt wird aus der Ausgabe des Unterprozesses gelesen. Erreicht die Policy den Übergabepunkt nicht oder dauert die Fahrt länger als 300 s, gilt das Greifen als fehlgeschlagen, und der Ablauf geht in den Fehlerzustand.

4. Für das Ablegen verbindet sich Modul 2 wieder, setzt Werkzeug, Override und Bewegungsprogramm auf seine eigenen Werte zurück und legt das Bauteil auf der Prüfvorrichtung ab.

Dass immer nur eine Seite den Roboter und die Draufsichtkamera benutzt, regelt die Bedienoberfläche (Abschnitt 9.3). Daneben enthält sie einen eigenen Reiter für Modul 4 mit Systemcheck, Aufnahme, Training und Betrieb.

8.10 Stand und Ergebnisse

Bisher wurde die Kette aus Aufnahme, Training und Ausführung in zwei Umgebungen erprobt: in einer eigenen Simulation des Roboters und in der VM der Neura-Steuerung. In beiden Fällen lieferten die Kameras nur Platzhalterbilder ohne Bezug zur Szene. Die Policy konnte also nur aus dem Zustand lernen; geprüft wurde die Kette, nicht die spätere Greifqualität.

Tabelle 8.4: Erprobung der Kette mit Platzhalterbildern (17.09.2026). Erfolg: Greifer schließt höchstens 10 mm neben dem Greifpunkt der Aufzeichnung, und die Fahrt endet höchstens 10 mm neben dem Übergabepunkt. Werte als Median über die Fahrten.

Umgebung (Datensatz)	Erfolg	Greifpunkt	Bahn p95	Vorhersage
Simulation (50 Episoden)	10/10	1,6 mm	4,0 mm	117 ms
VM der Steuerung (30 Episoden)	0/5	–	5,2 mm	85 ms

In der Simulation führte die Policy die Aufgabe in allen zehn Fahrten selbst aus (Tabelle 8.4 und Abb. 8.7). In der VM folgte sie der Bahn bis zur Vorgreifposition, blieb dort aber in allen Fahrten stehen. An dieser Stelle endet in den Daten eine PTP-Fahrt im Stillstand, und die folgende LIN-Fahrt beginnt aus dem Stillstand. Im Zustand sind „angekommen“ und „losfahren“ dort nicht zu unterscheiden, und die Policy sagt „stehen bleiben“ vorher. Daraufhin wurde der Bahnplaner so geändert, dass er an der Vorgreifposition mit 10 mm überschleift (Abschnitt 8.1). Am Greifpunkt selbst bleibt ein Halt, weil der Arm dort für das Schließen des Greifers wartet. Ob die Policy diesen Halt mit echten Bildern überwindet, ist noch offen.

Ob das eingespeiste Rauschen nützt, wurde vorab in einer Ersatzstudie ohne Bilder untersucht. Dafür wurde mit derselben Planungs- und Rauschkette ein einfaches Netz trainiert und in einer Simulation mit verschobenen Objektlagen und seitlichen Stößen von 15 mm geprüft (5 Trainingsläufe mit je 24 Objektlagen). Tabelle 8.5 zeigt das Ergebnis. Ohne Rauschen gelang ein Drittel der Fahrten, mit Rauschen etwa neun von zehn. Die gewählte Einstellung, eine je Episode zufällige Rauschstärke, war bei Stößen am besten und erzeugte zugleich die ruhigsten Bewegungen. Da die Studie ohne Bilder und mit einem anderen Netz arbeitet, lässt sich das Ergebnis nur der Tendenz nach auf die Anlage übertragen.

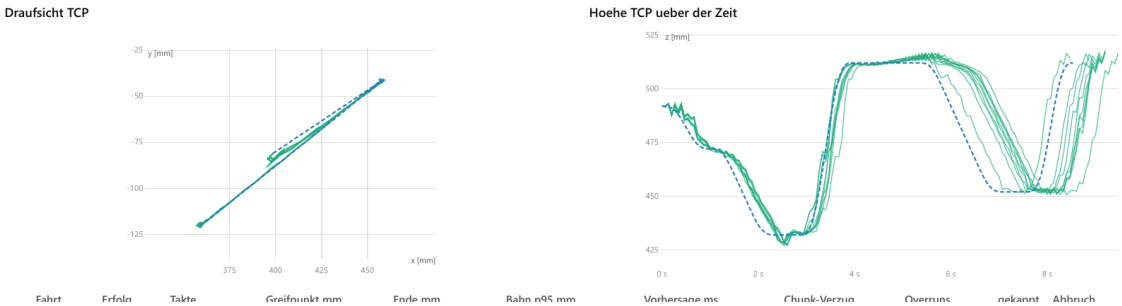
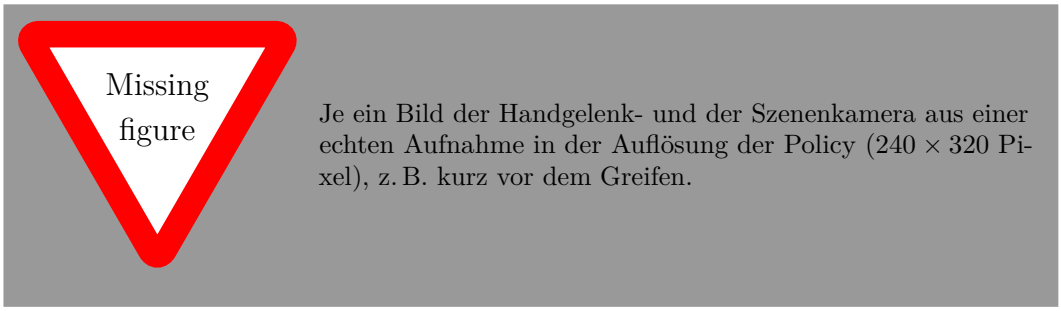


Abbildung 8.7: Zehn Fahrten der Policy in der Simulation: Draufsicht (links) und Höhe des TCP über der Zeit (rechts). Blau gestrichelt die ideale Bahn der Aufzeichnung, grün die Fahrten der Policy. Quelle: Unterlagen Modul 4.

Tabelle 8.5: Ersatzstudie zum eingespeisten Rauschen (16.09.2026, ohne Bilder). Erfolg: Greifen weniger als 5 mm neben dem Greifpunkt und Übergabe mit geschlossenem Greifer.

Rauschen	Erfolg ungestört	Erfolg mit Stößen	Greiffehler
ohne	32 %	12 %	3,4 mm
fest, 15 mm / 2,9°	90 %	92 %	0,6 mm
je Episode zufällig, 0–15 mm / 0–2,9°	92 %	93 %	0,5 mm

Am realen Roboter sind Aufnahme und Kameras in Betrieb genommen. Sechs Aufnahme-Episoden mit der endgültigen Ausführungsart, noch mit Platzhalterbildern und ohne Greiferansteuerung, liefen ohne Abbruch; der Arm lag dabei höchstens 0,06 rad bis 0,10 rad hinter der Sollstellung, weit innerhalb der Grenzen aus Abschnitt 8.7. Bei der ersten Aufnahme mit beiden echten Kameras wurden alle drei Episoden verworfen, weil die Handgelenkkamera über einen USB-Verteiler nur 23 fps lieferte und dadurch zu viele Bilder älter als 30 ms waren. Direkt am Rechner erreicht die Kamera ihre volle Bildrate.



Je ein Bild der Handgelenk- und der Szenenkamera aus einer echten Aufnahme in der Auflösung der Policy (240 × 320 Pixel), z. B. kurz vor dem Greifen.

Abbildung 8.8: Beobachtung der Policy: Handgelenkkamera (links) und Szenenkamera (rechts)

8.11 Zwischenfazit

Die Kette aus automatischer Aufnahme mit eingespeistem Rauschen, Training einer Diffusion Policy und Ausführung mit Sicherheitsprüfungen ist umgesetzt und in den automatischen Ablauf eingebunden. In der Simulation fährt die Policy die Aufgabe zuverlässig; in der VM hat die Erprobung eine Schwäche des Datendesigns an Haltepunkten gezeigt, die an der Vorgreifposition im Bahnplaner behoben ist und am Greifpunkt noch beobachtet werden muss. Am realen Roboter wurde eine Ausführungsart gefunden, die so ruhig ist wie ein Programm des Herstellers und die Bahn dabei genau einhält. Eine Aussage zur Greifqualität ist erst nach dem Training mit echten Bildern möglich.

Jeremy: Zwischenfazit nach den Versuchen mit echtem Datensatz ergänzen.

9 Integration und Bedienung

Die Kapitel 5 bis 8 haben die Module einzeln beschrieben. Dieses Kapitel zeigt, wie sie zu einem lauffähigen Gesamtsystem zusammengesetzt werden: wie ein Arbeitszyklus abläuft, wie das System konfiguriert und bedient wird, wie es auf Fehler reagiert und wie die Ergebnisse protokolliert werden.

9.1 Ablauf eines Zyklus

Ein Arbeitszyklus folgt dem Zustandsautomaten aus Abschnitt 4.3. Abbildung 9.1 zeigt die Aufrufe, die der Orchestrator dabei an die Module richtet. Zuerst liefert die Objekterkennung die Pose des Bauteils. Das Greifen übernimmt je nach Einstellung die Robotersteuerung oder das lernbasierte Greifen; in beiden Fällen liegt das Bauteil danach auf der Prüfposition, und der Roboter wartet auf Anfahrhöhe darüber, außerhalb des Bilds der Seitenkamera. Danach prüft die Sichtprüfung das Bauteil. Abhängig vom Ergebnis nimmt der Roboter das Bauteil wieder auf, legt es auf der Ablage für Gut- oder Schlechtteile ab und kehrt in die Ruheposition zurück. Damit ist der Zyklus beendet, und der nächste kann beginnen.

Findet die Objekterkennung kein Bauteil, endet der Zyklus ohne Fehler, bevor sich der Roboter bewegt. Im Dauerbetrieb wiederholt der Orchestrator den Zyklus, bis er angehalten wird oder ein Fehler auftritt.

Da die Module nur über ihre Schnittstellen angesprochen werden, läuft dieser Ablauf unabhängig davon, welche Implementierung eines Moduls gewählt ist. Derselbe Zyklus lässt sich daher vollständig gemockt, mit gespeicherten Testfotos und simuliertem Roboter oder an der realen Anlage ausführen (Abschnitt 4.5).

9.2 Konfiguration

Alle Werte, die vom konkreten Aufbau abhängen, stehen in einer zentralen Konfigurationsdatei im YAML-Format. Dazu gehören:

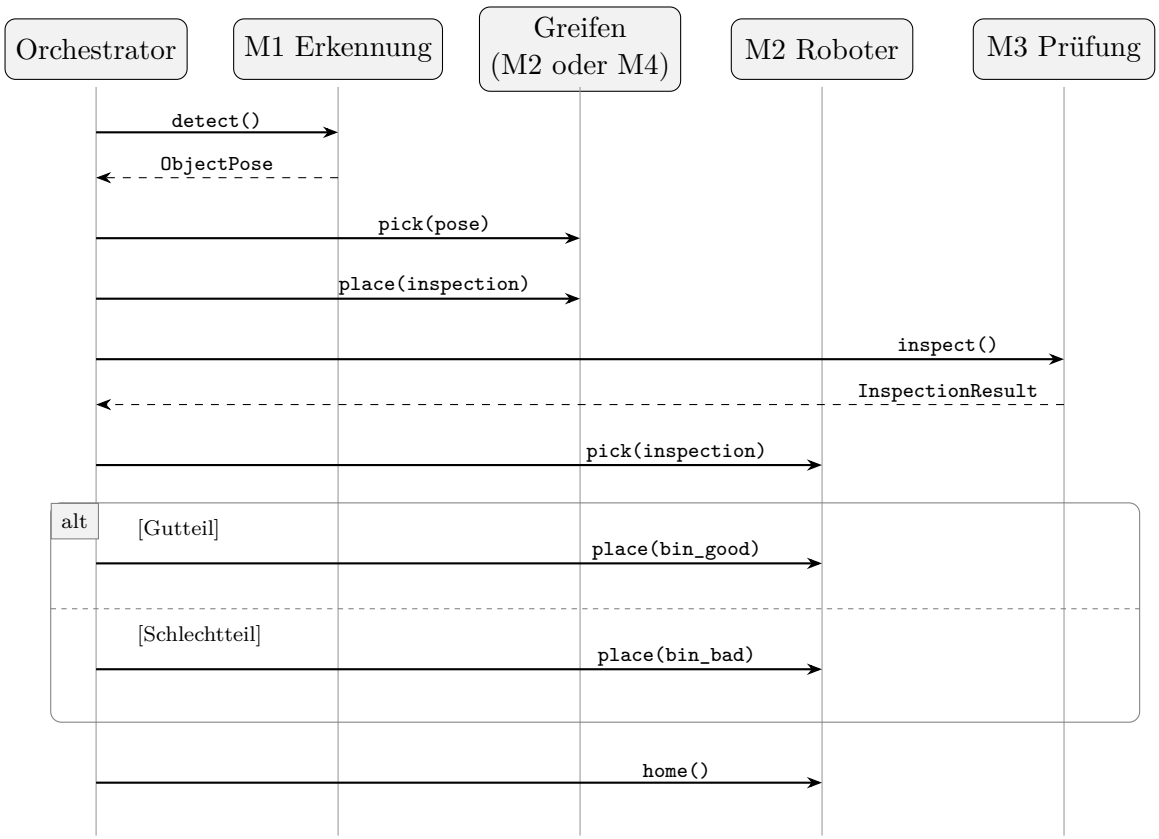


Abbildung 9.1: Aufrufe des Orchestrators an die Module während eines Arbeitszyklus. Greifen und Ablegen an der Prüfposition laufen über die Schnittstelle `Grasper`: entweder direkt über M2 oder mit der Policy von M4, die anschließend über M2 ablegt.

- Kameraquellen und Auflösungen,
- Größe des Arbeitsbereichs, Wörterbuch und Positionen der ArUco-Marker,
- Schwellwerte der Bildverarbeitung für Segmentierung und Prüfmerkmale,
- Transformation vom Arbeitsbereich in das Roboter-KS,
- Netzwerkadresse, Werkzeug, Geschwindigkeiten, Arbeitsraumgrenzen und feste Posen des Roboters,
- Sollwerte und Format der Prüfmerkmale,
- für Modul 4 das trainierte Modell, die Kameras und die Zeitgrenze einer Fahrt,
- die standardmäßig verwendete Implementierung jedes Moduls.

Die Module werden über eine gemeinsame Fabrikfunktion erzeugt, die zu einem Modul und einem Variantennamen die passende Implementierung liefert (Tabelle 9.1). Diese Funktion nutzen sowohl der Start über die Kommandozeile als auch die Bedienoberfläche, sodass beide Wege dasselbe System aufbauen.

Tabelle 9.1: Wählbare Varianten der Module

Modul	Variante	Bedeutung
Erkennung	`mock`	zufällige Pose, keine Kamera
	`replay`	Objekterkennung auf gespeicherten Testfotos
	`camera`	Objekterkennung auf dem Live-Bild der Top-down-Kamera
Roboter	`mock`	Bewegungen werden nur protokolliert
	`sim`	Robotersteuerung gegen die eigene Nachbildung der Steuerung
	`vm`	Robotersteuerung gegen die VM des Herstellers
	`real`	realer Roboter
Greifen	`robot`	M2 greift an der erkannten Pose und legt ab
	`bc`	die Policy von M4 greift bis zum Übergabepunkt, M2 legt ab (Kapitel 8)
Prüfung	`mock`	zufälliges Prüfergebnis, keine Kamera
	`replay`	Sichtprüfung auf gespeicherten Testfotos
	`camera`	Sichtprüfung auf dem Live-Bild der Seitenkamera

9.3 Bedienoberfläche

Für Inbetriebnahme und Betrieb gibt es eine grafische Bedienoberfläche (Graphical User Interface (GUI)), die mit dem Framework Qt über dessen Python-Anbindung PySide6

umgesetzt ist. Sie ist in sechs Reiter gegliedert (Tabelle 9.2). Unter den Reitern zeigt ein Protokollfenster laufend die Meldungen aller Module an.

Tabelle 9.2: Reiter der Bedienoberfläche

Reiter	Funktion
Ablauf	Auswahl der Varianten für Erkennung, Roboter, Prüfung und Greifen (M2 oder M4), einzelner Zyklus oder Dauerbetrieb mit einstellbarer Pause, Anzeige des aktuellen Zustands, der Bilder von Erkennung und Prüfung direkt nach dem jeweiligen Schritt, einer Draufsicht auf den Roboterarbeitsraum sowie einer Tabelle und Zählung der Ergebnisse; optional Speichern der Bilder und Zwischenschritte
M1 Erkennung	Bild aus Kamera, Testfoto oder Datei holen und analysieren, optional fortlaufend; Anzeige des Kamerabilds mit erkannten Markern und der entzerrten Draufsicht mit Pose
M2 Roboter	Verbinden mit Simulator, VM oder realem Roboter, Status, Greifer öffnen und schließen, Tippbetrieb, Stopp und Freigabe; Seiten für die Inbetriebnahme (Erste Schritte, Achsentest, Posen teachen, Kalibrierung, Zeigetest, Abschnitt 6.7)
M3 Prüfung	wie M1 für die Seitenkamera; Anzeige der Prüfbereiche und der erkannten Merkmale zum Einstellen der Parameter
M4 Behavior Cloning	Verbindung zu Simulation, VM oder realem Roboter mit Override und Freigabe der realen Anlage; Unterreiter Systemcheck, Aufnahme, Training, Betrieb und Konsole (Kapitel 8)
Einstellungen	alle Werte der Konfigurationsdatei als bearbeitbarer Baum; Änderungen wirken sofort, Speichern schreibt die Datei unter Erhalt der Kommentare

Screenshots mit echten Kamerabildern am Aufbau ersetzen, sobald vorhanden; für M4 z. B. den Unterreiter „Aufnahme“ mit Live-Bildern beider Kameras.

Alle Reiter greifen auf eine gemeinsame Sitzung zu, die Konfiguration, Kameras und Roboterverbindung hält. Eine Kamera oder eine Roboterverbindung wird dadurch nur einmal geöffnet, auch wenn sie in mehreren Reitern genutzt wird, und Einzeltests in den Modulreitern und der automatische Ablauf verwenden dieselbe Hardware.

Modul 4 baut eine eigene Verbindung zur Robotersteuerung auf und öffnet die Draufsichtkamera selbst als Szenenkamera. Damit nie zwei Seiten gleichzeitig den Roboter bewegen, regelt die Oberfläche den Zugriff: Wer den Roboter braucht, übernimmt ihn, und die andere Seite wird getrennt, sofern sie gerade nichts tut. Läuft auf der anderen Seite noch ein Zyklus, eine Aufnahme oder eine Fahrt, wird die Übernahme mit einer Meldung abgelehnt. Ebenso gibt die Objekterkennung die Draufsichtkamera frei, bevor Modul 4 sie öffnet. Training und Ausführung der Policy laufen als Unterprozess in einer eigenen Python-Umgebung (Abschnitt 8.9), die ein Einrichtungsskript anlegt.

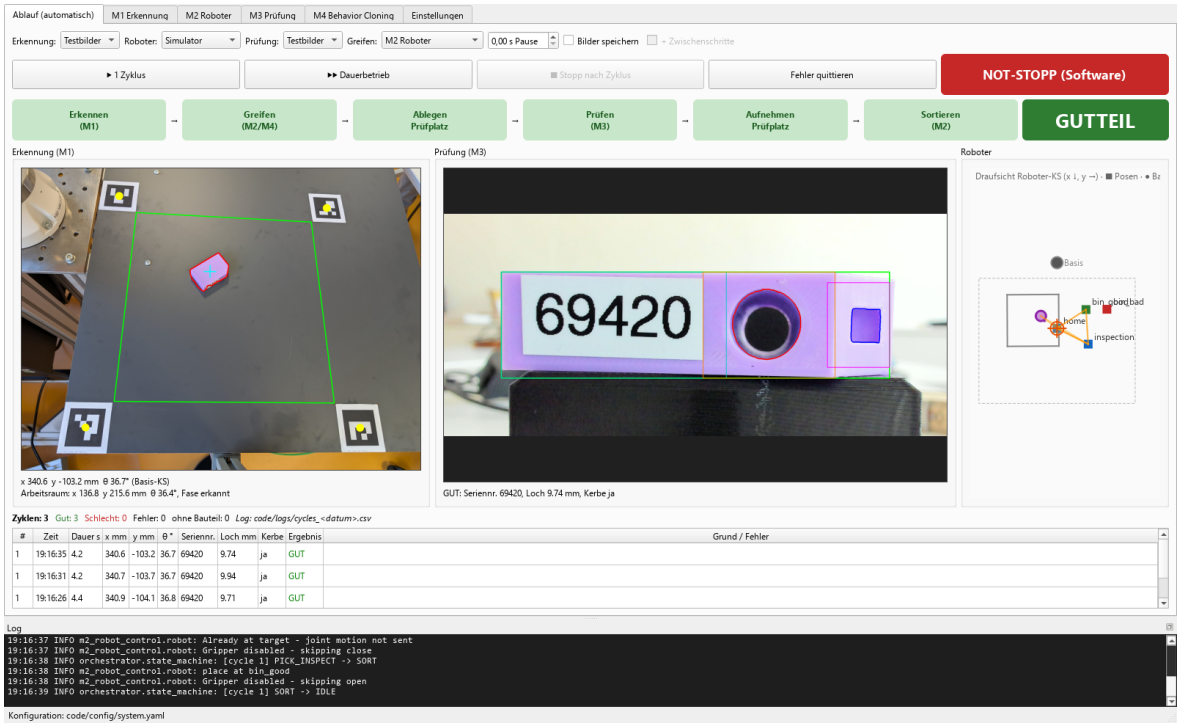


Abbildung 9.2: Bedienoberfläche, Reiter „Ablauf“ nach drei Zyklen mit Testfotos und simuliertem Roboter. Oben die Auswahl der Varianten, darunter die Ablaufkette, die Bilder von Erkennung und Prüfung, die Draufsicht auf den Roboterarbeitsraum und die Ergebnistabelle.

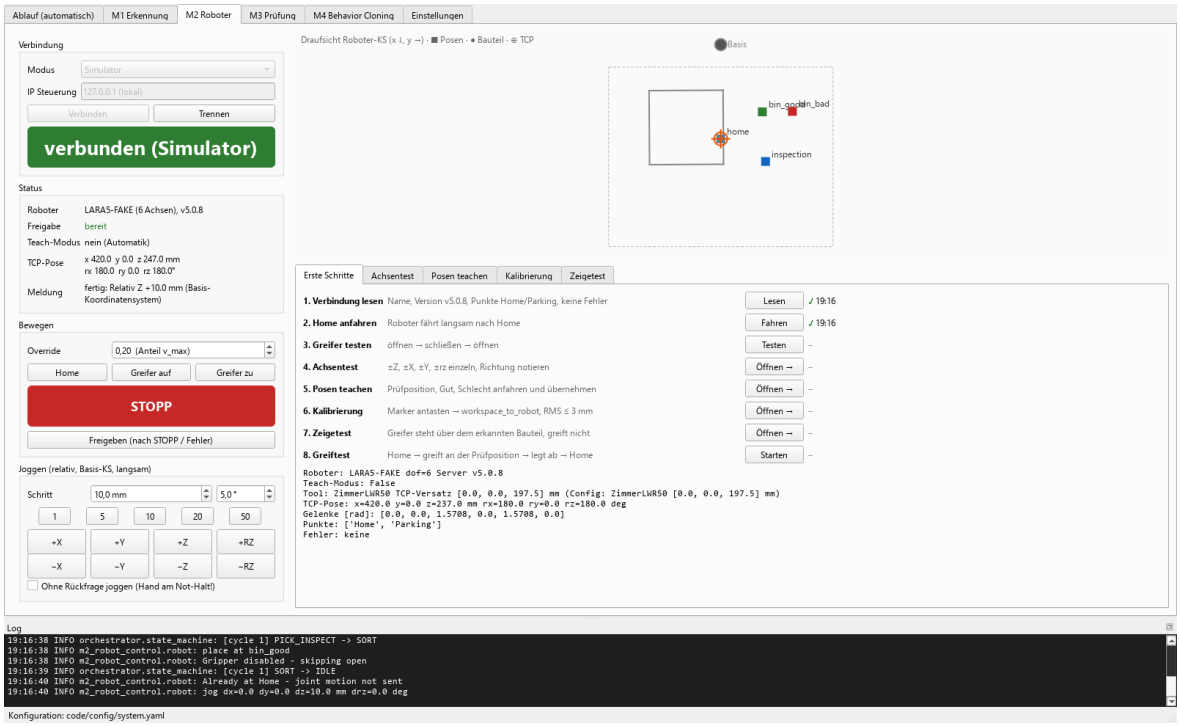


Abbildung 9.3: Bedienoberfläche, Reiter „M2 Roboter“ mit dem Simulator verbunden: links Verbindung, Status, Override und Tippbetrieb, rechts die Draufsicht und die Seiten für die Inbetriebnahme

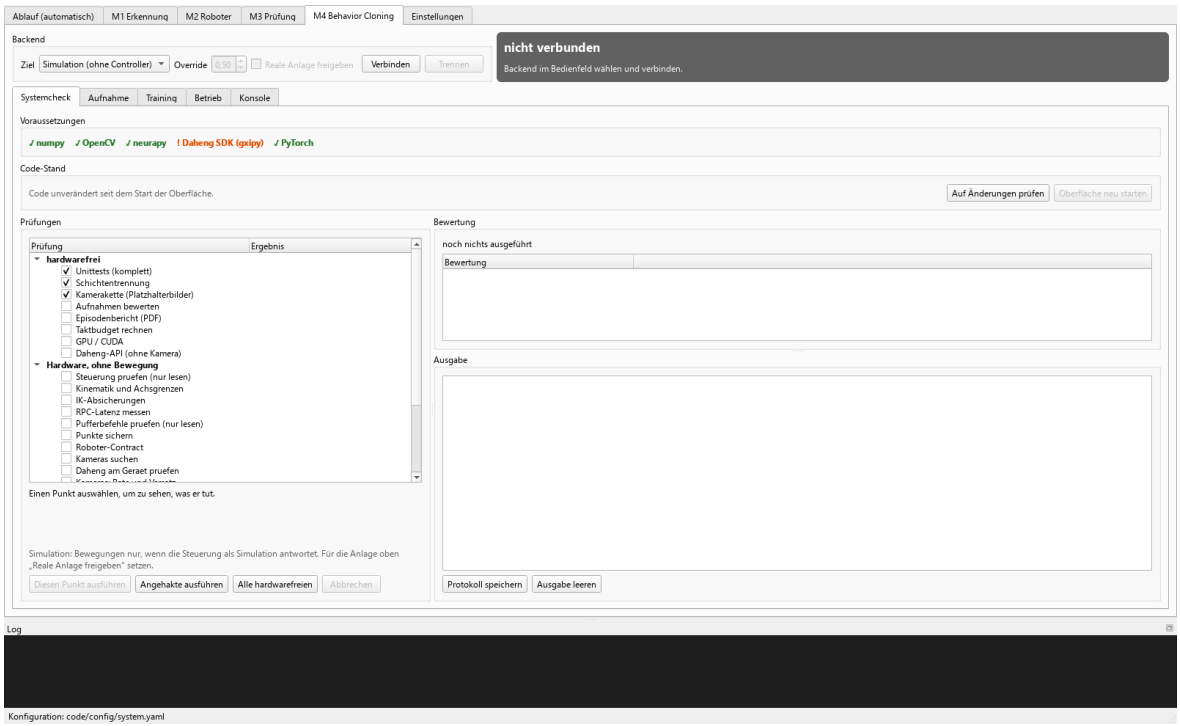


Abbildung 9.4: Bedienoberfläche, Reiter „M4 Behavior Cloning“, Unterreiter Systemcheck

Damit die Oberfläche während langer Roboterbewegungen oder Bildverarbeitungsschritte bedienbar bleibt, laufen diese Aufgaben in eigenen Hintergrund-Threads. Alle Befehle an den Roboter werden in einem einzigen Thread strikt nacheinander ausgeführt, sodass sich zwei Bewegungsbefehle nie überschneiden können. Der Zustandsautomat meldet jeden Zustandswechsel an die Oberfläche, die den aktuellen Zustand farbig in der Ablaufkette hervorhebt.

9.4 Fehlerbehandlung und Sicherheit im Ablauf

Jedes Modul meldet Fehler über eine eigene Ausnahme (Abschnitt 4.3.3). Tritt während eines Zyklus eine Ausnahme auf, gleich in welchem Modul, wechselt der Orchestrator in den Zustand `ERROR`, hält die Fehlermeldung zusammen mit dem Zustand fest, in dem sie auftrat, und stoppt den Roboter. Ein Dauerbetrieb wird dabei beendet. Weiter geht es erst, wenn der Bediener den Fehler quittiert: Dann wird der Roboter erneut vorbereitet (Abschnitt 6.1), fährt in die Ruheposition, und der Automat kehrt nach `IDLE` zurück.

Zusätzlich gibt es in der Oberfläche einen Stopp-Knopf, der den Roboter unabhängig vom Zustand des Ablaufs sofort anhält. Er läuft in einem eigenen Thread, damit

er nicht hinter einer gerade laufenden Bewegung in der Warteschlange des Roboter-Threads wartet. Greift gerade Modul 4, bricht der Stopp auch dessen Fahrt ab. Der Software-Stopp ergänzt den Hardware-Not-Halt, ersetzt ihn aber nicht.

Bevor der Roboter an der realen Anlage aus der Oberfläche heraus bewegt wird, fragt ein Dialog, ob der Arbeitsraum frei ist und der Not-Halt in Reichweite. Greift im Ablauf Modul 4, nennt der Dialog das ausdrücklich; die Bestätigung ersetzt die Freigabe, die Modul 4 für Fahrten am realen Roboter verlangt (Abschnitt 8.7). Im Dauerbetrieb kann der Ablauf außerdem so angehalten werden, dass der laufende Zyklus noch zu Ende geführt wird.

9.5 Protokollierung

Jeder Zyklus wird als Zeile in einer Comma-Separated Values (CSV)-Datei protokolliert; pro Tag entsteht eine Datei. Eine Zeile enthält Zeitpunkt und Nummer des Zyklus, ob er erfolgreich war, bei einem Fehler dessen Meldung, die Dauer, die erkannte Pose, die Prüfergebnisse (Seriennummer, Lochdurchmesser, Kerbe), das Gesamturteil und die Gründe für ein Schlechtteil. Optional werden zu jedem Zyklus die Ergebnisbilder von Objekterkennung und Sichtprüfung gespeichert, auf Wunsch zusammen mit allen Zwischenschritten der Bildverarbeitung (13 Bilder der Objekterkennung, 14 der Sichtprüfung). So lassen sich einzelne Zyklen nachträglich nachvollziehen; die Abbildungen in Kapitel 5 und 7 stammen aus dieser Funktion. Fahrten von Modul 4 werden zusätzlich von dessen eigener Protokollierung erfasst (Abschnitt 8.6).

Diese Protokolle sind die Datengrundlage für die Evaluation in Kapitel 10.

10 Evaluation

Die Evaluation soll zeigen, ob das Gesamtsystem die Zielsetzung aus Kapitel 1 erfüllt: Ein Bauteil wird ohne Eingriff eines Bedieners erkannt, gegriffen, geprüft und richtig sortiert. Sie ist als Funktionsnachweis angelegt und nicht als Messkampagne zur Genauigkeit einzelner Verfahren. Grundlage sind die Zyklusprotokolle aus Abschnitt 9.5.

10.1 Versuchsaufbau und Vorgehen

Alle Versuche finden an der realen Anlage statt, mit allen Modulen in der Betriebsstufe „Real“ (Abschnitt 4.5). Es werden zwei Versuchsreihen durchgeführt.

Als Vorstufe lief am Labortag am 06.10.2026 ein erster vollständiger Zyklus am realen Roboter. Erkennung und Prüfung arbeiteten dabei noch auf Testbildern, und der Greifer war noch nicht angesteuert (Kapitel 6). Dieser Lauf zeigte, dass Zustandsautomat, Robotersteuerung und Posen zusammenpassen, ist aber noch kein Ergebnis im Sinne der Versuchsreihen.

Versuchsreihe 1: Wiederholbarkeit. Ein Gutteil wird wiederholt von Hand an zufälliger Position und mit zufälliger Orientierung in den Arbeitsbereich gelegt, und jeweils ein vollständiger Zyklus wird gestartet. Die Reihe zeigt, ob Erkennung, Greifen, Ablegen an der Prüfposition und Prüfung für unterschiedliche Lagen zuverlässig zusammenspielen und das Teil jedes Mal als Gutteil sortiert wird.

Versuchsreihe 2: Aussortierung. Es werden gezielt fehlerhafte Bauteile gefertigt, bei denen jeweils eines der Prüfmerkmale nicht erfüllt ist (Tabelle 10.1). Jedes dieser Teile durchläuft den Zyklus wie in Versuchsreihe 1. Die Reihe zeigt, ob das System die Fehler erkennt und die Teile als Schlechtteil ablegt.

Anzahl der Durchläufe je Versuchsreihe festlegen. Festlegen, ob und welche Kennzahlen ausgewertet werden (z. B. Anteil erfolgreicher Zyklen, Fehler je Zustand, Zyklusdauer).

Tabelle 10.1: Fehlerhafte Bauteile für Versuchsreihe 2

Teil	Fehler	Erwartetes Ergebnis
A	Kerbe fehlt	Schlechtteil, Grund „Kerbe fehlt“
B	Lochdurchmesser außerhalb der Toleranz	Schlechtteil, Grund „Durchmesser außerhalb der Toleranz“
C	Etikett mit Seriennummer entfernt	Schlechtteil, Grund „Seriennummer ungültig“

10.2 Ergebnisse Versuchsreihe 1: Wiederholbarkeit

Ergebnisse nach den Versuchen ergänzen.

10.3 Ergebnisse Versuchsreihe 2: Aussortierung

Ergebnisse nach den Versuchen ergänzen.

10.4 Vergleich klassischer und KI-basierter Ansatz

Verglichen wird das Greifen: einmal mit der Objekterkennung und der programmierten Greifsequenz von Modul 2 (Kapitel 5 und 6), einmal mit der gelernten Policy von Modul 4 (Kapitel 8). Prüfung und Sortieren sind in beiden Fällen gleich. Beide Varianten lassen sich im Reiter „Ablauf“ umschalten, sodass dieselbe Versuchsreihe 1 mit beiden Varianten gefahren werden kann.

Die Übergabe zwischen beiden Modulen wurde am 28.10.2026 in der VM der Steuerung geprüft. Modul 4 fuhr dabei über eine zweite Verbindung zur Steuerung, anschließend setzte Modul 2 seine Einstellungen zurück, legte an der Prüfposition ab und fuhr in die Ruheposition, ohne dass ein Fehler auftrat. Zwei gleichzeitige Verbindungen zur Steuerung sind demnach kein Problem. Da für diesen Test kein trainiertes Modell vorlag, hielt die Policy ihre Stellung, statt zu greifen; der Übergabepunkt wurde also nicht erreicht. In der VM meldete die Ausführung außerdem viele Taktüberschreitungen (562 bei 45 Takten), die auf die Rechenleistung der VM zurückgehen dürften und an der Anlage zu prüfen sind.

Als Vergleichsgrößen bieten sich der Anteil erfolgreicher Griffe und die Robustheit gegenüber Störungen an, etwa bei veränderter Beleuchtung oder verschobener Szenenkamera. Die Zykluszeit eignet sich nur eingeschränkt: Die Policy fährt im Tempo ihrer

Aufzeichnung (Abschnitt 8.3), und im Ablauf wird ihre Ausführung für jeden Zyklus neu gestartet, einschließlich des Ladens des Modells.

Jeremy: Vergleich nach den Läufen mit trainiertem Modell ergänzen (Anzahl Versuche, Erfolgsrate beider Varianten, Verhalten bei Störungen). Dauer des Starts der Inferenz je Zyklus messen.

11 Fazit und Ausblick

11.1 Zusammenfassung

Nach den Versuchen ergänzen.

11.2 Diskussion

Einordnung der Ergebnisse aus Kapitel 10: Was hat funktioniert, wo lagen die Grenzen, welche Probleme traten auf?

11.3 Ausblick

Offen, ob und welche Punkte hier genannt werden. Kandidaten aus dem aktuellen Stand: Ansteuerung des Greifers über IO-Link und Auswertung der Greiferrückmeldung; intrinsische Kalibrierung der Top-down-Kamera; Korrektur der Parallaxe durch die Bauteilhöhe; eine dauerhaft laufende Inferenz von Modul 4, die nicht in jedem Zyklus neu startet; Werkzeugkugeln und Schutzräume für die Robotersteuerung (Abschnitt 2.7); Nutzung von Zyklen des klassischen Ablaufs als zusätzliche Demonstrationen für Modul 4; Weiterverwendung im Pflegerobotik-Projekt (Entscheidung des Teams).

Literaturverzeichnis

[1] Neura Robotics GmbH, *Offline Robot Simulation Setup*, Herstellerdokumentation, 2. Dez. 2024.

[2] G. Bradski, „The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] N. Otsu, „A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, Jg. 9, Nr. 1, S. 62–66, 1979. DOI: 10.1109/TSMC.1979.4310076

[4] S. Suzuki und K. Abe, „Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics, and Image Processing*, Jg. 30, Nr. 1, S. 32–46, 1985. DOI: 10.1016/0734-189X(85)90016-7

[5] M.-K. Hu, „Visual pattern recognition by moment invariants," *IRE Transactions on Information Theory*, Jg. 8, Nr. 2, S. 179–187, 1962. DOI: 10.1109/TIT.1962.1057692

[6] A. R. Smith, „Color gamut transform pairs," *ACM SIGGRAPH Computer Graphics*, Jg. 12, Nr. 3, S. 12–19, 1978. DOI: 10.1145/965139.807361

[7] M. D. Fairchild, *Color Appearance Models*, 3. Aufl. Chichester: Wiley, 2013, ISBN: 978-1-119-96703-3. DOI: 10.1002/9781118653128

[8] R. Hartley und A. Zisserman, *Multiple View Geometry in Computer Vision*, 2. Aufl. Cambridge: Cambridge University Press, 2004. DOI: 10.1017/CBO9780511811685

[9] Z. Zhang, „A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 22, Nr. 11, S. 1330–1334, 2000. DOI: 10.1109/34.888718

[10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas und M. J. Marín-Jiménez, „Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, Jg. 47, Nr. 6, S. 2280–2292, 2014. DOI: 10.1016/j.patcog.2014.01.005

[11] S. Umeyama, „Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 13, Nr. 4, S. 376–380, 1991. DOI: 10.1109/34.88573

[12] B. Siciliano und O. Khatib, Hrsg., *Springer Handbook of Robotics*, 2. Aufl. Cham: Springer, 2016. DOI: 10.1007/978-3-319-32552-1

[13] J. Canny, „A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. PAMI-8, Nr. 6, S. 679–698, 1986. DOI: 10.1109/TPAMI.1986.4767851

[14] R. O. Duda und P. E. Hart, „Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, Jg. 15, Nr. 1, S. 11–15, 1972. DOI: 10.1145/361237.361242

[15] A. Fitzgibbon, M. Pilu und R. B. Fisher, „Direct least square fitting of ellipses," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 21, Nr. 5, S. 476–480, 1999. DOI: 10.1109/34.765658

[16] R. Brunelli, *Template Matching Techniques in Computer Vision: Theory and Practice*. Chichester: Wiley, 2009. DOI: 10.1002/9780470744055

[17] R. Smith, „An overview of the Tesseract OCR engine," in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, Bd. 2, 2007, S. 629–633. DOI: 10.1109/ICDAR.2007.4376991

[18] Y. Baek, B. Lee, D. Han, S. Yun und H. Lee, „Character region awareness for text detection," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, S. 9357–9366. DOI: 10.1109/CVPR.2019.00959

[19] B. Shi, X. Bai und C. Yao, „An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 39, Nr. 11, S. 2298–2304, 2017. DOI: 10.1109/TPAMI.2016.2646371

[20] JaidedAI, *EasyOCR: Ready-to-use OCR with 80+ supported languages*, Version 1.7.2. besucht am 26. Sep. 2026. Adresse: https://github.com/JaidedAI/EasyOCR

[21] D. Morrison, P. Corke und J. Leitner, „Closing the Loop for Robotic Grasping: A Real-time, Generative Grasp Synthesis Approach," in *Robotics: Science and Systems XIV*, 2018. DOI: 10.15607/RSS.2018.XIV.021

[22] B. D. Argall, S. Chernova, M. Veloso und B. Browning, „A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, Jg. 57, Nr. 5, S. 469–483, 2009. DOI: 10.1016/j.robot.2008.10.024

[23] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel und J. Peters, „An Algorithmic Perspective on Imitation Learning," *Foundations and Trends in Robotics*, Jg. 7, Nr. 1–2, S. 1–179, 2018. DOI: 10.1561/2300000053

[24] S. Ross, G. J. Gordon und J. A. Bagnell, „A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,“ in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. DOI: 10.48550/arXiv.1011.0686

[25] M. Laskey, J. Lee, R. Fox, A. Dragan und K. Goldberg, „DART: Noise Injection for Robust Imitation Learning,“ in *Proceedings of the 1st Conference on Robot Learning (CoRL)*, 2017. DOI: 10.48550/arXiv.1703.09327

[26] G. E. Uhlenbeck und L. S. Ornstein, „On the Theory of the Brownian Motion,“ *Physical Review*, Jg. 36, Nr. 5, S. 823–841, 1930. DOI: 10.1103/PhysRev.36.823

[27] J. Ho, A. Jain und P. Abbeel, „Denoising Diffusion Probabilistic Models,“ in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. DOI: 10.48550/arXiv.2006.11239

[28] J. Song, C. Meng und S. Ermon, „Denoising Diffusion Implicit Models,“ in *International Conference on Learning Representations (ICLR)*, 2021. DOI: 10.48550/arXiv.2010.02502

[29] C. Chi u. a., „Diffusion policy: Visuomotor policy learning via action diffusion,“ *The International Journal of Robotics Research*, Jg. 44, Nr. 10–11, S. 1684–1704, 2024. DOI: 10.1177/02783649241273668

[30] K. He, X. Zhang, S. Ren und J. Sun, „Deep Residual Learning for Image Recognition,“ in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, S. 770–778. DOI: 10.1109/CVPR.2016.90

[31] T. Z. Zhao, V. Kumar, S. Levine und C. Finn, „Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,“ in *Robotics: Science and Systems XIX*, 2023. DOI: 10.15607/RSS.2023.XIX.016

[32] R. Cadene u. a., *LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch*, Version 0.6.1, 2024. besucht am 30. Okt. 2026. Adresse: https://github.com/huggingface/lerobot

[33] Neura Robotics GmbH, *NeuraPy, Release v5.0.8*, Herstellerdokumentation der Python-API, 11. Juli 2025.

[34] Zimmer Group, *2-Backen-Parallelgreifer LWR50L-23-00004-A*, Produktdatenblatt, Version 1. Adresse: https://www.zimmer-group.com

[35] Zimmer Group, *MATCH-Robotermodul LWR50F-13-05-A*, Produktdatenblatt, Version 1. Adresse: https://www.zimmer-group.com